Source: docs/DESIGN-NOTES.md · revision 6f7537e8c268 · SHA-256
c5f1afdbc83479121e6cf8f6328e2f8a641682fe7372792fe821681edb64caee

MHFE Design Notes and Analysis

This supplement accompanies version 0.7.0 of the MHFE specification in `README.md`. It is not normative and does not define the format; it explains in more detail why MHFE is built the way it is and what is known about its security. The specification's Rationale explains the design decisions; this supplement develops their analysis.

Part I analyses suite 3. A section after it examines what the narrower state of the same-length suite 4 changes. Part II is the detailed design discussion. It was first written for suite 2 and has since been brought up to date; it describes suite 3. `README.md` defines suites 3 and 4; where the documents differ, `README.md` takes precedence. Words such as "must" and "should" in this supplement describe the design; the requirements themselves are those of the specification. The archive keeps the text of suite 2 as released, marked as historical; only its links to `mhfe` commits were updated.

Contents

- Part I. Analysis for suite 3
  - Why a state-derived salt and a Feistel network
  - Security model
  - Scope of the security results
  - Ciphertext-only guessing
  - Known pairs
  - What a guess costs
  - Composition with the BIP39 passphrase
  - Deniability
  - Parameter rationale
  - Final-word-preserving cycle walking (research idea)
  - Open questions for review
- Suite 4: what the narrower state changes
- Part II. Detailed design discussion
  - Motivation
  - Notes on the construction
  - Rationale
  - Backward Compatibility
  - Security Considerations
  - Related Work and Alternatives
  - Theoretical Investigation and Alternative Designs
  - Reference Implementation
  - Test Vectors
  - Known Limitations
- Research directions
  - Container check words: a hash of the container (planned)
  - Argon2i on the rounds with a public salt
  - A second memory-hard function
  - A hidden wallet behind an honest disclosure
  - Check words for derived wallets in the current suites
  - Derived wallets of a chosen length
  - Nested containers
  - A check for new 24-word and suite 4 sources by choosing the entropy
  - A private check word for generated passwords
  - Alternative formats for multiple password openings with recovery checks

- Appendix: proofs of the guessing bounds
  - A.1 Model
  - A.2 Theorem 3: one container
  - A.3 Proof of Theorem 3
  - A.4 Consequences
  - A.5 Theorem 4: a known pair
  - A.6 Theorem 5: a 24-word source with an unknown BIP39 passphrase
  - A.7 What the proofs do not cover
  - A.8 Related proofs and techniques

Part I. Analysis for suite 3

Why a state-derived salt and a Feistel network

A constant salt would work, but it would help the attacker. Encrypting 256 bits into 256 bits without storing anything is easy on its own: derive one key as $\mathrm{Argon2id}(\mathrm{password}, \mathrm{constant})$ and use any 256-bit block cipher, such as Threefish-256 or a wide-block mode like HCTR2. Decryption needs only the container, the password and the constant from the specification. The weakness is that the constant is the same for every user. An attacker computes Argon2id once for each password in a large dictionary and then tests that list of keys against every container in the world with a cheap cipher operation. The expensive work is paid once and reused everywhere.

The only unique material is the container itself. MHFE stores no salt, so the one thing that differs between two users' backups is the backup. The Argon2id salt must therefore come from the data being encrypted, and decryption must be able to recompute it from the container. This requires a value that is available to encryption before a step and to decryption after that step.

A Feistel network is a simple way to meet this requirement. In each round the right half passes through unchanged and becomes the next left half. The salt of every round is derived from that half, so decryption can recompute a different salt for every round. The last round's salt comes from the left half of the container, which anyone holding the container can read. Containers of independently generated sources therefore have different salt chains, except when branch values repeat or salts collide; these exceptions are negligible under the model below. An attacker cannot ordinarily reuse a password's KDF work across such containers. SLIP-0039 also derives each round's salt from the unchanged half, with PBKDF2 [19].

Other structures considered here also offer a value that one round leaves unchanged, but none is simpler:

- An unbalanced Feistel network, called Direction B in Part II, splits the state unevenly. Its published idealized bounds [21] favor the imbalance only when enough rounds are available. With few rounds the balanced network has the stronger bound, and every extra round costs an Argon2id call.
- The Lai-Massey scheme [22] keeps the XOR of its two halves unchanged within a round, so the salt could be derived from that value. Without a mixing step between rounds, the same value would survive every round and pass the XOR of the source halves straight into the container. The scheme therefore needs such a step. Vaudenay proves security with an orthomorphism or an almost orthomorphism [23], which adds complexity and is less studied in this setting.

- The swap-or-not shuffle [24] has a round invariant too, but it decides one swap per round. Its published bounds call for hundreds of rounds or more, depending on the domain and the security target. These bounds are sufficient rather than proven minimums, but with one Argon2id call per round they would make a recovery many times slower.

A Feistel network needs few rounds, has a well-studied theory, and keeps the construction easy to implement and check.

Security model

The main threat is simple: someone obtains the 24-word container, for example by finding or photographing a metal backup, and tries to recover the original seed phrase by guessing the password offline. Part I analyses this threat first. Threat models T1 to T15 in Part II cover the others, some of which are serious in practice, such as a substituted container phrase or leaked intermediate values.

The attacker is assumed to know the suite, the MHFE format and, unless the owner keeps them secret, the PIM and the memory level. The attacker may also be able to check a candidate mnemonic against the blockchain, because a wallet with funds or history reveals itself. Rarer situations, such as knowing an original seed phrase together with its container (a "known pair"), are treated separately below.

The arguments below use the random-oracle model, the standard idealization for analysing password hashing:

- Argon2id with fixed parameters is modelled as a random function $A(P, S)$ of password and salt. Distinct input pairs have independent outputs; repeated pairs reuse the same output. In this model each query costs one KDF evaluation; the model does not prescribe the attacker's memory allocation.
- BLAKE2b, SHA-256 and HMAC-SHA-256 are modelled as random functions.
- For Corollary G10, the derivation of wallet data from the phrase is modelled as a further random function.
- For Theorem 5, the BIP39 seed of a phrase and passphrase, the wallet-check digest and the wallet data derived from the seed are modelled as further random functions. Seed computations have their own budget, as the second expensive resource.
- The password is drawn from some distribution; $p_1 \geq p_2 \geq \ldots$ are the probabilities of the most likely passwords.

Treating Argon2id outputs for different salts as independent is the assumption Part II calls A1. The random-oracle model does not capture how expensive each evaluation is in memory and time. That question belongs to the Argon2id analysis itself and is addressed under What a guess costs.

In this model the guessing bounds below are theorems, proved in the appendix: Theorem 3 for one container, Theorem 4 for a known pair and Theorem 5 for a 24-word source with an unknown BIP39 passphrase. They count Argon2id evaluations and, with separate budgets, BIP39 seed computations and the cheap functions, whose number matters too. The scope tables identify the information, success events and conditions for each result; A.1 states the model's assumptions. Other statements of Part I are arguments unless they say otherwise.

The model also omits side channels. Argon2id uses data-independent memory addresses only during the first half of its first pass; the remaining addresses are data-dependent [4]. This hybrid design does not prove that an MHFE implementation resists timing or cache observation. Platform-specific analysis and careful handling of intermediate states remain necessary even on an offline computer.

Scope of the security results

The deniability results compare disclosure experiments. The guessing bounds limit recovery of a specified secret for given query budgets. Neither kind of result implies the other. These tables point the reader to the statements and their hypotheses; they do not replace them.

Deniability results

Both experiments disclose one container, one password, the phrase it recovers and that wallet's record. They require freshly uniform source entropy, independent of the functions, the passwords and the adversary's prior information. They also require the stated password-sampling rules and the same wallet-usage scenario for the disclosed wallets. That scenario is the abstract record generator U defined below. It makes no cryptographic oracle queries, receives no hidden password or round keys, and gives distinct wallets independent records.

Result	Source lengths	Question answered	Scope
Theorem 1 and its suite 4 extension	Suite 3: 24 words. Suite 4: every supported length.	Do the random functions together with the disclosed data have exactly the same joint distribution in the honest and prepared experiments?	Exact equality, even for fixed functions. Fixed-point refusal and decoy-password redrawing are omitted; discovery of the additional real wallet is outside this equality.
Theorem 2 and its suite 4 extension	Suite 3: 12 to 24 words. Suite 4: every supported length.	How well can an adversary distinguish the two experiments using the functions and wallet-record lookups?	In the random-oracle model, an advantage bound in terms of k, q, l, p_1 and ENT, including refusal, redrawing and discovery of the real wallet.

Here k counts distinct passwords submitted to Argon2id, q counts HMAC queries and l counts wallet-record lookups; p_1 is the largest password probability and ENT is the source entropy length in bits. For suite 3, Theorem 2 compares against an honest 24-word source. For a shorter real source, it assumes no information about the source length beyond the experiment. The suite 4 extension compares sources of the same length. Neither theorem covers sources selected by the optional wallet check.

Guessing bounds

These games concern one suite 3 container, a known source length and fixed known settings, with classical oracle queries. P is the MHFE password, E the source entropy, $X = \mathrm{pack}(E)$ the state encrypted by MHFE and Y the container state. Here Q is the BIP39 passphrase; in the deniability experiments above, the same letter denotes the decoy password.

k_A counts calls to the ideal Argon2id function; q_B, q_H and q_G count salt, mask and source-hash queries respectively. The wallet-reference budget is q_Ω. Theorem 5 also counts BIP39 seed computations as k_S and wallet-check queries as q_T.

Result	Information available to the adversary	Success event	Budgets
Theorem 3	One container of a uniform 12- to 21-word source.	Output the actual password P.	k_A, q_B, q_H, q_G.
Corollary G6	Theorem 3's information.	Output the source entropy E.	Theorem 3's budgets; the output counts as one additional source-hash input.
Corollary G7	Theorem 3's information, with queries submitted in batches.	Recover P with at most eleven batches containing Argon2id queries.	Theorem 3's total budgets, plus the batch restriction.
Theorem 4	A fixed known state X and its container Y, at any source length.	Output P.	k_A, q_B, q_H.

Result	Information available to the adversary	Success event	Budgets
Corollary G10	One container of a uniform source at any length and a wallet reference idealized as $\Omega(E)$, with an empty or known BIP39 passphrase.	Output P.	$k_A, q_B, q_H, q_G, q_\Omega$.
Theorem 5, parts (a) and (b)	One container of a 24-word source generated with the wallet check, and the target wallet reference $\omega^* = \Omega(\text{seed}^*)$. The adversary knows that the profile was used.	(a) Output seed^*; (b) output the actual pair (P, Q).	$k_A, k_S, q_B, q_H, q_T, q_\Omega$.
Corollary W5	Theorem 5's information.	Output the actual phrase entropy and passphrase (E, Q).	Theorem 5's budgets; the output counts as one additional seed query.
Corollary W6	One container of an ordinary uniform 24-word source and $\omega^* = \Omega(S(E, Q))$, with an unknown independent passphrase.	The seed, secret-pair and phrase-with-passphrase recovery events above.	Theorem 5's budgets, without a programmed wallet-check function.

Source and password conditions. The password distribution is independent of the functions. In Theorem 3 and Corollaries G6, G7 and G10, the source is uniform and independent of the password and functions. Theorem 4 instead fixes X independently of the password and round functions; it does not require uniform X. Corollary G10 requires the stated uniformity and independence of the wallet-reference function; its width is not the length of serialized fields. Corollary G10 also excludes sources selected by the wallet check and conditions the password probabilities on the known passphrase; Theorem 5's known-passphrase case conditions them on the leak. Under either conditioning, the password must remain independent of the functions, and the stated source and oracle assumptions must still hold.

The wallet-check case. For unknown Q, Theorem 5 requires P and Q to be independent of each other and of all the modelled functions. Its stated generation and non-disclosure conditions apply, with no additional phrase-identifying data and no funded opening under another MHFE password. All funds of the original phrase stay under Q: a funded wallet of that phrase under another BIP39 passphrase is excluded, even if that passphrase remains unknown to the adversary. Empty or known Q is included with $s_1 = 1$; all these exclusions remain in force, and the target reference ω^* remains allowed. Corollary W5 inherits these conditions. Corollary W6 uses ordinary uniform source entropy, so the generation-by-selection error and the check-query term disappear; the remaining restrictions still apply.

Reading the bounds. The error terms of each bound must be compared with its main term at the chosen budgets. The main term in Theorem 3 accounts for $\lfloor k_A \,/\, 12 \rfloor$ password candidates and one final guess. Theorem 4's main term uses eleven calls per fully tested candidate and one final guess. The fractional credit in Theorem 4 comes from the proof's averaging; no attack is known that uses a partial chain. For $m = \lfloor k_A \,/\, 11 \rfloor$, Proposition K5 bounds the relative false-match loss by $m \cdot (2^{-128} + 11 \cdot (m+1) \cdot 2^{-256})$. The 128-bit filter in K5 therefore has a small loss, about $m \cdot 2^{-128}$, while $m \ll 2^{128}$; this loss is present even when the fractional credit is zero.

An untested password guess does not itself provide the source in G6. G7 concerns depth for an unknown source, not a known pair. Every parallel query in G7 still counts towards k_A. Remark G9 adds 2^{-256} for creation refusal to Theorems 3 and 4 and Corollaries G6, G7 and G10; Theorem 5 and its corollaries already include that term. Propositions G8, K5 and W7 give concrete attacks for comparison, with their stated false-match losses. For G8 and W7, narrow tests can leave a substantial gap. In W7 the test may be the 16-bit wallet check alone, a wallet reference or both.

These bounds hold in ideal-function models and count cost in oracle calls, not in proven amounts of physical memory or elapsed time. The guessing bounds do not cover suite 4, multiple containers, multiple known pairs or additional funded openings; the remaining cases are listed in A.7.

Ciphertext-only guessing

What counts as a password test depends on the source. Ordinary recovery performs twelve Argon2id evaluations. For a short source, the recovered candidate can be screened by its verifier. For a 24-word source, recovery alone cannot confirm a password. Confirmation requires external information, such as a known address, and needs the full recovered mnemonic. A phrase created with the optional wallet check `MHFE-WALLET-CHECK-SEED-1` adds a 16-bit filter, evaluated on every 24-word reading, as the wallet check explains. With the empty BIP39 passphrase, this filter screens MHFE password guesses alone; a wrong guess still passes in about one case in 65,536. This check also needs the full recovered mnemonic.

Theorem 3 (statement and proof). For a 12- to 21-word source in suite 3 with uniform entropy independent of the password, an adversary that holds one container and makes k_A Argon2id evaluations and q_G, q_B and q_H SHA-256, BLAKE2b and HMAC evaluations names the password with probability at most

$$\operatorname{top}\left(\left\lfloor \frac{k_A}{12} \right\rfloor + 1\right) + \varepsilon, \qquad \varepsilon = \frac{q_G + q_B + 12 p_1 k_A + 66}{2^{128}} + \frac{q_H}{2^{256}}.$$

where $\operatorname{top}(m) = p_1 + \ldots + p_m$. The bound matches the strategy of testing the most likely passwords, each with the twelve evaluations of a recovery: all cheaper partial work together gains at most ε, and that strategy falls short of the main term only through false verifier passes (Proposition G8). For recovering the source, the bound is $\operatorname{top}(\lfloor k_A \,/\, 12 \rfloor) + \varepsilon + 2^{-128}$ (Corollary G6). Theorems 1 and 2 are the deniability results below; Observation 2 keeps its earlier number.

The proof replaces the container by a uniform value and keeps the container's relation to the source in a single programmed SHA-256 point, the source entropy e^* behind the container. The password then influences what the attacker sees only through a SHA-256 query at e^*. To query SHA-256 at e^*, the attacker must in effect follow the inverse computation round by round: the half L_{i+1} that feeds the salt of round i is hidden behind the mask M_{i+1} until $A(P, S_{i+1})$ has been evaluated. A skipped round therefore leaves the next salt, and finally e^*, uniformly random up to the events counted in ε. After the eleven evaluations for rounds 11 to 1, the right half $R_0 = L_1$ is known. For a 12-word source it is $\mathrm{Trunc}_{128}(\text{SHA-256}(E))$, and testing whether it has a preimage needs SHA-256 at the unknown L_0, about 2^{128} evaluations, the first term of ε. For 15 to 21 words such a screen filters essentially nothing. The proof uses completed-chain counting, a technique also used in the analysis of iterated password hashing by Bellare, Ristenpart and Tessaro [47]. The state-derived salts and the Feistel dependencies require the separate argument of the appendix.

For a uniformly generated 24-word source, the core permutation's container is exactly independent of the password (Lemma 2); the creation refusal contributes the exceptional term of Remark G9. Every password gives a valid phrase, and a test needs external data, such as a known address or the master key fingerprint, derived from the full recovered mnemonic. Corollary G10 extends Theorem 3 to such tests for every source length, with the empty or a known BIP39 passphrase. The bounds of G10 match the twelve-call recovery strategy up to the stated error terms and false-match losses, with the q_Ω evaluations of the derivation counted beside q_G. With an unknown BIP39 passphrase, the wallet data also depends on the passphrase. Theorem 5 covers that case for 24-word sources (see Composition).

Many containers. The salt of the last round comes from the container's own left half. In the described direct guessing procedure, an Argon2id result for one password and salt is reused for another container only when both the password candidate and the salt match. Containers with the same left half, or salts that collide after truncation to 128 bits, can share that evaluation. For suite 3 containers of independently generated sources under the stated model, accidental salt collisions are expected to be negligible; uniqueness is probabilistic, not guaranteed. This salt derivation avoids the direct reuse of one fixed-salt password table across all containers.

Theorem 3 concerns one container; a bound for many containers needs its own game. In a composable model, per-session confidentiality of password-based encryption is unachievable when salting bounds guesses only globally rather than per session [48]. Suite 4's narrower salt diversity is discussed in its analysis. If several containers use the same password, finding it opens all of them; that is inherent to any password scheme.

Blockchain checks. A 24-word source has no internal verifier, but a funded wallet provides an external check: the attacker can derive addresses from each candidate and look them up. This check needs the full candidate mnemonic, so in the ordinary inverse it comes after all twelve evaluations. The check means that "no verifier" does not by itself stop a guesser when the wallet has no BIP39 passphrase (see Composition).

Known pairs

A known pair is an original seed phrase together with its container, for example after the seed phrase leaked through another channel. With a known pair the attacker can test a password guess by evaluating the rounds from both ends and checking that they meet.

Observation 2. With a known pair, a password guess can be tested with 11 of the 12 Argon2id evaluations.

Argument. Compute rounds $0 \ldots k-1$ forward from X and rounds $11 \ldots k+1$ backward from Y.

- If exactly one round k is skipped, the Feistel relation $L_{k+1} = R_k$ compares two known 128-bit values without that round's mask. This is a valid filter with 11 evaluations.
- If two adjacent rounds k and $k+1$ are skipped, the known values satisfy $L_{k+2} = L_k \oplus M_k$ and $R_{k+2} = R_k \oplus M_{k+1}$. Each relation contains a mask that was never computed and is uniformly random, so neither relation says anything about the password.
- Skipped rounds that are not adjacent split the chain so that the part between them can be computed from neither side.

So the filter saves one round out of N, the number of rounds. **Theorem 4** (statement and proof) bounds every attack in the model. It states that with one known pair of any suite 3 length, an adversary that makes k_A Argon2id evaluations and q_B and q_H BLAKE2b and HMAC evaluations names the password with probability at most $T(1 + k_A \,/\, 11) + \varepsilon_K$. Here $T(x) = \mathrm{top}(\lfloor x \rfloor) + (x - \lfloor x \rfloor) \cdot p_{\lfloor x \rfloor + 1}$ credits one partly tested password, and $\varepsilon_K = (24 \cdot p_1 \cdot k_A + 2 \cdot q_B + 419) \,/\, 2^{128} + (2 \cdot q_H + 12) \,/\, 2^{256}$. The bound matches the eleven-call filter up to that credit and the small terms. It does not cover several known pairs or preprocessing with retained advice. With four rounds, as in SLIP-0039, the same filter would save one quarter; with twelve rounds it saves one twelfth. This is one concrete reason for the round count (see Parameter rationale).

What a guess costs

The figures in this section come from one simple cost model. They are estimates, not measurements of a complete wallet attack.

At the default settings (PIM 0, memory level 0) one guess performs 12 Argon2id calls with 2 GiB and 12 passes each: about 288 GiB of Argon2id memory processing. Computing each 1 KiB block logically reads two blocks and writes one. From the second pass on, version 1.3 also reads the old content of the block it overwrites (RFC 9106, section 3.2). That is 3 to 4 KiB of logical memory access per KiB processed, roughly 0.9 to 1.2 TB per guess. Caches and registers can serve part of it, so the actual traffic to main memory depends on the implementation.

At 0.9 to 1.2 TB per guess, a graphics card limited by memory bandwidth manages on the order of one guess per second for every 1 TB/s of bandwidth. The model uses exactly one guess per second per card as a round figure, not as a measured attack speed. For comparison, the model assumes 1.5 million guesses per second per card for the PBKDF2-

HMAC-SHA512 step with 2,048 iterations that protects a BIP39 passphrase. This rate comes from the extrapolation below. Times are for finding the password after searching half of the space. The model ignores address derivation and lookups. RFC 9106 also discusses memory-time trade-offs that let an attacker use less memory at the price of recomputation. For both reasons, real costs may be lower or higher.

The PBKDF2 rate is a rounded extrapolation from the published RTX 4090 Hashcat 6.2.6 benchmark [7]: about 3.12 million PBKDF2-HMAC-SHA512 candidates per second at about 1,000 iterations scales to about 1.52 million at 2,048 if time scales linearly. This is a generic primitive benchmark. Its candidate varies the PBKDF2 password, whereas a BIP39 passphrase varies the salt while the mnemonic is fixed. Kernel optimizations, input lengths and preprocessing can therefore differ. The two rates are model inputs, not measurements of a complete MHFE-versus-BIP39 wallet attack on one card.

Word-based passwords in the table use independent uniform choices from the EFF list of 7,776 words [6]. Two words illustrate a weak password, not a recommendation. The 30- and 40-bit rows describe uniform random choices from spaces of 2^{30} and 2^{40} possibilities. Passwords that people invent are not uniform, and their length alone does not tell which row they belong to.

Password	Bits	MHFE default, one card	MHFE default, 1,000 cards	BIP39 passphrase, one card
Two random words, a weak example	~25.8	~350 days	~8 hours	~20 seconds
Uniform 30-bit random password	30	~17 years	~6 days	~6 minutes
Three random words	~38.8	~7,500 years	~7.5 years	~1.8 days
Uniform 40-bit random password	40	~17,000 years	~17 years	~4 days
Four random words	~51.7	~58 million years	~58,000 years	~39 years
Five random words	~64.6	~4.5 x 10^11 years	~4.5 x 10^8 years	~300,000 years

The ratio between the two schemes is about 1.5 million, or about 20 bits: in this model an MHFE password is as expensive to find as a BIP39 passphrase about 20 bits stronger. The password's entropy itself does not change. One random word from a 7,776-word list carries about 12.9 bits, so the default settings are worth roughly one and a half extra words. An attacker limited by memory capacity rather than bandwidth, such as a custom chip, is hit harder in this model: in the full-memory implementation it assumes, each active guess uses about 2 GiB of Argon2 working memory.

The relative cost in bits is $\log_2(\,\mathrm{BIP39\ rate}\ /\ \mathrm{MHFE\ rate}\,)$, so it is sensitive to both assumed rates. With the BIP39 rate held at 1.5 million per second, MHFE rates of 0.1, 1 and 10 guesses per second give about 23.8, 20.5 and 17.2 bits of relative cost respectively. These are sensitivity examples, not additional measurements or changes to password entropy.

This is a supporting argument for MHFE, not its main one. The main arguments are that MHFE protects the mnemonic itself, which a passphrase does not, and that it can be applied to an existing wallet without moving funds. The gain in bits comes on top, and for a cold-storage backup it costs the owner little. Measured end to end on the author's laptop at the default settings, a recovery took about 70 seconds on the command line, about 90 seconds in a browser's fast mode and about four minutes in a browser's standard mode; an encryption with its check took roughly twice as long (measurement record).

Farms and botnets. Two larger attackers, in the same model:

- A hypothetical farm of about 25 data-centre graphics cards with 80 GB each, 2 TB of memory in total, is assumed to have an aggregate bandwidth of 90 TB/s and to scale ideally. At the model's rounded 1 TB per guess, the cards together make roughly 90 guesses per second. Total memory alone does not determine this throughput.
- A botnet of one million ordinary computers, each able to spare 2 GiB and each finishing one guess in about two minutes, makes roughly 8,000 guesses per second together. Devices unable to provide that memory are excluded from this particular estimate. That leaves out many older phones and small devices, but not current phones with 8 GB or more. Implementations using time-memory trade-offs are outside the model.

Password	Farm, 2 TB of card memory	Botnet, one million computers
Uniform 30-bit random password	~2 months	~18 hours
Four random words, ~51.7 bits	~640,000 years	~7,000 years
Five random words, ~64.6 bits	~5 billion years	~54 million years

Password length versus work factor. At a fixed memory level, this model treats cost as approximately proportional to the number of passes, so PIM k multiplies the per-guess cost by $k + 1$. Increasing memory has a different, model-dependent effect. Doubling it roughly doubles cost for a bandwidth-limited attacker, or quadruples area-time cost under the capacity-limited assumptions in Parameter rationale. Memory-time trade-offs can change these estimates.

Under the pass-scaling assumption, PIM 1023 multiplies the cost by 1,024 (about 10 bits of cost) and makes a recovery take roughly 17 to 34 hours. Each additional independent random word multiplies the search space by 7,776 (about 12.9 bits), without increasing the Argon2id work factor. The work factor is therefore a margin on top of a good password, not a substitute for one.

The practical advice follows directly: a password that a person invents without a method can fall to a well-funded attacker within days, whereas four or five words chosen with dice, for example from the EFF large wordlist of 7,776 words, put the container out of reach in this model.

Composition with the BIP39 passphrase

MHFE protects the mnemonic; the BIP39 passphrase protects the wallet derived from it. They are applied one after the other: recovery restores the original seed phrase, and the passphrase is then used as before. The estimates below assume a known source length, independent uniformly chosen secrets (N_1 MHFE passwords costing C_1 each; N_2 passphrases costing C_2 each), and no separate check that identifies the original seed phrase. They describe sequential searches. For 24 words, Theorem 5 also proves a matching lower bound in the random-oracle model; for 12 to 21 words no lower bound is proved.

12 to 21 words: the costs add up when false matches are rare. The recovery verifier tests the MHFE password on its own, so an attacker first searches the MHFE passwords and then attacks the passphrase through the blockchain. A wrong MHFE password passes an r-bit verifier with probability about 2^{-r}, and each such false match needs its own full passphrase search. The expected cost is about

$$\frac{N_1 C_1}{2} + \frac{N_1 N_2 C_2}{2^{r+1}} + \frac{N_2 C_2}{2}$$

Relative to the MHFE work, the middle term's ratio is $N_2 \cdot C_2 \,/\, (2^r \cdot C_1)$; relative to the final passphrase search, it is $N_1 \,/\, 2^r$. So the middle term is negligible overall whenever N_1 is much smaller than 2^r, but no source length guarantees this for arbitrary secrets. With 21 words ($r = 32$), searching half of a four-word dice-password space yields

about 426,000 false matches. With a five-word dice passphrase, the passphrase searches for these false matches cost about 4,400 times the MHFE work in the model above. Even then, this pair-search term is smaller by a factor of 2^{32} than for a 24-word source with the same secrets.

24 words: the search runs over pairs. With no separate mnemonic check, the attacker runs one MHFE recovery per MHFE password and then tries passphrases on the resulting phrase, for an expected cost of about

$$\frac{N_1 C_1}{2} + \frac{N_1 N_2 C_2}{2}$$

This is not the product of two complete searches, but its second term grows with the product of the two spaces. Related or reused secrets lose most of this gain.

Theorem 5 bounds the chance of obtaining the BIP39 seed in the random-oracle model. The bound is the total probability of the most likely password and passphrase pairs that the budgets allow, plus explicit error terms. For the comparison with the cost above, each password counts as twelve Argon2id calls and each pair as one seed computation. For uniform secrets, when those error terms are negligible, success probability one half requires approximately $12 \cdot N_1 \,/\, 2$ Argon2id calls and $N_1 \cdot N_2 \,/\, 2$ seed computations. With C_1 for twelve calls and C_2 for one seed computation, the resulting budget threshold is approximately the cost above; the precise bound retains the error terms (A.6, "One cost budget"). The theorem's bound concerns a budget that limits every execution, whereas the expression above estimates the expected cost of a sequential search.

The same probability bound holds with or without the wallet check. The theorem assumes independent secrets, no data that identifies the phrase without the passphrase, and no funded wallet that another MHFE password opens on the same container. All funds must stay under the passphrase: a funded wallet of the phrase under another BIP39 passphrase is excluded, even if that passphrase remains unknown to the adversary. With the wallet check, the theorem also assumes a generation that meets its selection rule and reveals nothing about its trials.

Example with independent 30-bit secrets, in the model above:

Source	Search	Expected time on one card
12 words	MHFE password, then passphrase	~17 years plus ~6 minutes
24 words	about 2^{30} MHFE passwords, each with a search over passphrases	~12,200 years

Without a passphrase, a 24-word source offers no such gain: wallet history provides an external check of a candidate recovered with an MHFE password. A short source instead supplies an internal verifier, with the false-match probability described above.

This is a choice for the user, not a weakness of either option. A short source gives a recovery that reports whether a candidate passes its verifier, automatic length detection and simple password rehearsal. A 24-word source with an independent passphrase gives the strongest combination of the two secrets. The specification's Rationale section presents this choice.

Deniability

A person who is forced to disclose a password can disclose a different one, prepared in advance. This section states precisely what such a decoy disclosure achieves and proves it. The notion is modeled on receiver-deniable encryption in the sense of Canetti, Dwork, Naor and Ostrovsky [44]: an honest disclosure and a prepared one are compared in two experiments, and the adversary must tell them apart from the information it has. The notion is narrower than their definition. There, a faking algorithm can make a ciphertext look like an encryption of any chosen alternative message.

Here, the owner chooses a decoy password, and the decoy phrase is whatever that password recovers. The proofs hold for one container and one disclosure in the model below. Like Theorems 3 and 4 above, they are proofs within that model, and like them they have not been reviewed by an independent cryptographer.

The idea in words. For every password, encryption is a permutation: it shuffles all 2^{256} states. A shuffle of a uniformly random state gives a uniformly random state. So a container made from a 24-word phrase is a uniformly random state, whatever password made it. Reading such a container with any other password again gives a uniformly random state, which is exactly what an honest owner's phrase is. A prepared disclosure and an honest one therefore look the same. What differs is that behind a prepared disclosure a second wallet exists, the real one, and the adversary finds it only with about the probability of guessing the real password.

Notation. The suite and the settings are fixed. E_P is the permutation $\mathrm{Perm}(P, .)$ of the 256-bit states and D_P its inverse. A password is identified with its normalized encoding P_{enc}, so spellings that normalize to the same bytes are one password. Passwords are drawn from a distribution over normalized encodings with probabilities $p_1 \geq p_2 \geq \ldots$. A wallet is identified by its phrase, including its length: a packed 12-word state read as 24 words is a different phrase with a different wallet.

A usage scenario U is a randomized procedure that, given a phrase, produces the public record of its wallet's use, such as its transactions. Distinct wallets get independent records, and a wallet that is never used has an empty record. Independent records are an abstraction: a transfer between two wallets links their records. In these experiments U is an abstract generator of records that uses only the supplied phrase and independent randomness. It does not query the cryptographic random oracles or receive the hidden password or round keys. It models public wallet use, not the cryptographic computations that produce real transactions. An extension in which generating records makes such queries must account for them separately before claiming the bounds below.

Lemma 1 (consistency). For every password Q and container Y, the state $X' = D_Q(Y)$ read as 24 words is a valid BIP39 phrase, and $E_Q(X') = Y$.

Proof. Every 256-bit state is valid 24-word entropy, and E_Q and D_Q are inverse permutations.

A prepared disclosure is therefore a genuine opening of the container: recovering Y with Q gives X', and applying the permutation E_Q to X' gives Y. The full creation procedure with Q gives the same container unless $Y = X'$, a fixed point, which the creation procedure refuses.

Lemma 2 (uniformity). If a state Z is uniformly random and a permutation F is chosen independently of it, then $F(Z)$ is uniformly random and independent of F.

Proof. For every fixed F and state y, $\Pr[F(Z) = y] = \Pr[Z = F^{-1}(y)] = 2^{-256}$.

Lemma 3 (one input, two rounds). Let a Feistel network have at least two rounds with independent random round functions, and let its input (L_0, R_0) be fixed. Then its output is uniformly random.

Proof. The first mask is uniform, so $R_1 = L_0 \oplus M_0$ is uniform. The second mask is uniform and independent of R_1, so after round 1 the state $(L_2, R_2) = (R_1, R_0 \oplus M_1)$ is uniform. The later rounds form a permutation independent of the first two masks, so the output is uniform by Lemma 2.

The lemma concerns a single input. For two inputs with the same right half, two rounds keep the relation $L_2 \oplus L_2' = L_0 \oplus L_0'$. A uniform output for each input alone therefore does not make a Feistel network indistinguishable from a random permutation. One input suffices below because an adversary who is not given the real

password sees the real permutation applied to a second input only after a bad event charged in the proof of Theorem 2, such as an Argon2id query that happens to use the real password. Otherwise that permutation is applied once, to the real source.

Experiments. Both experiments give the adversary a container Y, a disclosed password and the public record of the disclosed phrase's wallet, and both let it look up the record of any phrase. A lookup of the disclosed phrase returns that record. In the prepared experiment, a lookup of the real phrase, when it differs from the disclosed one, returns the real wallet's record. Every other lookup returns an empty record. In both experiments, creation refuses a container equal to its source, as the specification requires, and the owner then draws another password.

Rejection and failure convention. In these ideal experiments, redrawing a password means sampling from its original distribution conditioned on the allowed set. For creation at fixed source and oracle tables, that set is the passwords whose container differs from the source; for the decoy draw it is the passwords different from the accepted P. If the allowed set has zero probability, the experiment instead gives the adversary a common failure symbol $\perp$, with no container, password or wallet record. The adversary's usual oracle-query budgets and final guess still apply. When the allowed probability is positive, this conditional sample has exactly the distribution of the stated redraw loop, which terminates almost surely. The experiment reveals no retry count or duration. This convention defines the outcome even for a singleton password distribution or for oracle tables on which every supported password is refused; it changes no operational algorithm.

The raw source entropy is sampled freshly and uniformly, independently of the shared oracle tables, the passwords and the adversary's prior information. The password distribution is fixed independently of those tables, and the initial draws of P and Q are independent; the stated rejection rules are applied afterward. For short sources this assumption concerns E before packing, not the packed state X, whose verifier is computed using SHA-256. Marginal uniformity alone is not enough: entropy computed from a public oracle answer can be uniform while already known to the adversary.

- **Honest disclosure.** A 24-word phrase with uniformly random entropy X is created and a password P is drawn. The container is $Y = E_P(X)$. The wallet of X is used according to U. The adversary receives Y, P and the record $U(X)$.
- **Prepared disclosure.** The owner's real source, with packed state X, and a password P give $Y = E_P(X)$. A decoy password Q is drawn from the same distribution and drawn again while it equals P; then $X' = D_Q(Y)$. The disclosed phrase is X' read as 24 words, and its wallet is used according to the same U. The real wallet, that of the owner's original seed phrase, is used in any way. The adversary receives Y, Q and the record $U(X')$.

The adversary knows the construction and how decoys are prepared. It may query the random oracles of the Security model adaptively, and it may look up the record of any phrase. Let k be the number of distinct passwords with which it queries Argon2id, q the number of its HMAC-SHA-256 queries and l the number of its lookups. Whenever the experiment itself calls a cryptographic primitive, it uses the same oracles. The scenario U makes no such calls, and the proof counts the twelve HMAC calls of the decoy recovery $D_Q(Y)$ separately. The adversary outputs a guess of the experiment. Its advantage is the difference between the probabilities that it answers "prepared" in the two experiments. The condition that both disclosed wallets follow the same scenario U is essential: a decoy created a minute ago cannot pass for a wallet with years of history.

Theorem 1 (24-word source, exact). Let the real source have 24 words and uniformly random entropy, with the independence and password-sampling conditions of the experiments, and ignore the refusal of fixed points and the redrawing of Q. Then the random oracles, the container, the disclosed password, the disclosed phrase and its wallet's record have exactly the same joint distribution in both experiments.

Proof. In the prepared experiment X is uniform and independent of P, of Q and of the oracles, so by Lemma 2 $Y = E_P(X)$ is uniform and independent of all of them. For each value of Q and of the oracles, Lemma 2 applied to D_Q shows that $X' = D_Q(Y)$ is uniform, and $Y = E_Q(X')$ by Lemma 1. So the oracles together with (Y, Q, X') have the distribution of the oracles together with $(E_P(X), P, X)$ in the honest experiment. Adding the records $U(X')$ and $U(X)$ keeps the two distributions equal, since the scenario is the same. The argument holds for any fixed functions in place of the oracles.

What Theorem 1 leaves open is the real wallet: a lookup of X shows its record in the prepared experiment only. Theorem 2 bounds the effect of that and of everything else.

Theorem 2 (bound, every source length). Let the real source have 12 to 24 words and uniformly random entropy E of ENT bits, with the independence and password-sampling conditions of the experiments. For a source shorter than 24 words, let the adversary have no information about the source length beyond what the experiment gives it. In the random-oracle model the advantage is at most

$$(k + 2) \cdot p_1 + l \cdot 2^{-\mathrm{ENT}} + (12 \cdot q + 146) \cdot 2^{-256}.$$

Proof. The proof moves from the prepared experiment to the honest one through a sequence of games in the sense of Shoup [49]. Each step either changes nothing in distribution, or leaves two games identical until a stated event occurs, and then changes the result by at most that event's probability. The oracles are sampled lazily.

- **Game 0** is the prepared experiment.
- **Game 1** draws Q once, independently of P, and keeps it even if it equals P. The owner also keeps the first password even if it gives a fixed point. Games 0 and 1 use the same random choices. They differ only if the first Q equals P, which has probability at most p_1, or if the first password gives a fixed point, which by Lemma 3 has probability 2^{-256} in the random-oracle model. Couple each conditional draw to its first unconditioned draw: if that draw is allowed, keep it; otherwise draw from the conditional distribution, or return $\perp$ if its mass is zero. Thus a failure under the convention above is already inside one of these same bad events; it is not discarded by conditioning the experiment on success.
- **Game 2** encrypts the real source with private copies of the random functions: Argon2id with the password P, and HMAC-SHA-256 with the twelve resulting keys K_i, are answered by random functions independent of the shared oracles. A lookup of the real phrase returns an empty record unless the real phrase is the disclosed one. Games 1 and 2 proceed identically until one of these bad events occurs: the adversary queries Argon2id with P; the decoy recovery uses P, that is $Q = P$; the adversary or the decoy recovery calls HMAC-SHA-256 with one of the keys K_i; or the adversary looks up the real phrase while it differs from the disclosed one. BLAKE2b and SHA-256 remain shared in both games and answer every query alike. An adversary may compute salts, but without P those salts give only independent Argon2id outputs, and a query with P is already a bad event.
- **Game 3** is the honest experiment without the refusal of fixed points. It differs from the honest experiment by at most 2^{-256}, again by Lemma 3. The same coupling includes its failure outcome when no password is allowed.

Games 2 and 3 have the same distribution. In Game 2 the twelve round functions that produced Y are independent random functions, independent of the shared oracles: they come from the private copies, and their mask input messages contain the round index and the half, so no two rounds share an input even where two salts coincide. By Lemma 3, Y is uniform and independent of the source, of Q and of the shared oracles. As in Theorem 1, the adversary then faces the same random oracles, a container that is uniform and independent of them, the disclosed password, the phrase that this password recovers, that phrase's record and empty records for every other phrase. This is exactly what it faces in Game 3, including every answer to its adaptive queries.

In Game 2 everything the adversary sees is independent of P, of the keys K_i and of the real source, so the bad events have these probabilities. Each password has probability at most p_1, so the adversary's k passwords include P with probability at most $k \cdot p_1$, and $Q = P$ has probability at most p_1. The keys K_i are uniform 256-bit values, independent of the keys of the decoy recovery, so the q HMAC queries of the adversary and the twelve of the decoy recovery hit one of them with probability at most $12 \cdot (q + 12) \cdot 2^{-256}$. The real phrase is determined by E, so each lookup hits it with probability at most $2^{-\mathrm{ENT}}$.

Adding the steps gives the bound: p_1 and 2^{-256} for Game 1, $(k + 1) \cdot p_1 + 12 \cdot (q + 12) \cdot 2^{-256} + l \cdot 2^{-\mathrm{ENT}}$ for Game 2 and 2^{-256} for Game 3.

Remark on cost. An ordinary search computes $D_p(Y)$ for candidate passwords p, twelve Argon2id evaluations each, and looks up the results. Theorem 2 bounds the probability of success by the number of passwords tried and lookups made. It does not show that this search is the cheapest way to try a password. For one container of a uniform source in the random-oracle model, Theorem 3 and Corollary G10 give success bounds matching this strategy up to their error terms and false-match losses. They cover the verifier of a short source and checks against wallet data at every source length with the empty or a known BIP39 passphrase; a passphrase unknown to the adversary is not covered.

Corollary (simulating a prepared disclosure with a programmable record oracle). In the synthetic experiments above, a simulator receives the container and access to the same cryptographic functions and real-wallet record oracle as an adversary without a disclosure. This base record oracle returns the real wallet's record for its phrase and an empty record for every other phrase. The simulator need not know the real phrase or its password. It draws a password from the distribution and recovers the container with it, which costs twelve Argon2id evaluations. It then samples a record for the recovered phrase from U. For subsequent lookups it wraps the base oracle: a lookup of the recovered phrase returns this sampled record, and every other lookup is forwarded unchanged to the base oracle. In particular, the real wallet's record remains available when its phrase differs from the disclosed one.

The simulation produces the prepared experiment's disclosure and subsequent oracle answers, except that the drawn password is not rejected when it equals P, an event of probability at most p_1. An adversary's success probability therefore differs by at most p_1. The simulation applies to every source length and needs no idealized cryptographic function. It requires fresh, programmable synthetic records at the time of disclosure and the stated access to the real-wallet oracle; it does not construct that oracle from the container. It does not simulate disclosure against a fixed public ledger or change answers to queries made before disclosure. The bounds of Theorems 1 and 2 stand on their own. The real wallet's protection still rests on the password and on MHFE.

The owner must not select the decoy password by the phrase it produces. For example, a decoy phrase that also passes a 21-word verifier, which happens with probability about 2^{-32}, must be kept, because an honest 24-word phrase can do the same. The same holds for the 16-bit check of `MHFE-WALLET-CHECK-SEED-1`: a decoy phrase that passes it, with probability about 2^{-16}, must be kept too.

Honey encryption. For a 24-word source the construction has the structure of honey encryption in the sense of Juels and Ristenpart [43], with a uniform message model and the identity as its encoder. Every password recovers an equally likely phrase, so only external information, above all the use of the real wallet, identifies the right one. This supplement does not claim that MHFE meets the formal security definition of honey encryption; whether it does also depends on what external information is available. Part II's remark that MHFE has no such model refers to wallet entropy in general. A short source is different: its verifier deliberately rejects wrong passwords, which is why Theorem 2 needs an adversary who does not know the source length.

Suite 4: the same analysis at every length

The experiments, Lemmas 1 to 3 and the proofs apply to the same-length suite 4 with ENT-bit states in place of 256-bit ones. Every ENT-bit state is valid entropy for a phrase of the source's length, so Lemma 1 holds at every length, and Lemmas 2 and 3 do not depend on the width. Both disclosures read the container at the source's own length, so the honestly disclosed phrase has that length instead of 24 words. With these changes:

- **Theorem 1 for suite 4 (exact, every length).** For a suite 4 source of 12, 15, 18 or 21 words with uniformly random entropy and the independence and password-sampling conditions of the experiments, and ignoring the refusal of fixed points and the redrawing of Q, the random oracles, the container, the disclosed password, the disclosed phrase and its wallet's record have exactly the same joint distribution in both experiments. The proof is that of Theorem 1.
- **Theorem 2 for suite 4.** Under the same source and password-sampling conditions, in the random-oracle model the advantage is at most $(k+2)\cdot p_1 + (l+2)\cdot 2^{-\mathrm{ENT}} + 12\cdot(q+12)\cdot 2^{-256}$. The proof is that of Theorem 2. The two fixed-point terms become $2^{-\mathrm{ENT}}$, because the permutation acts on ENT bits, while the keys K_i remain 256-bit Argon2id outputs. The condition on the source length is not needed: the container shows the length in both experiments alike.

For a 12-word source the fixed-point term and each lookup term are 2^{-128}, a small absolute probability. How they compare with p_1 depends on the password distribution, and the full bound above applies. Both theorems need freshly uniform source entropy that is independent of the oracle tables, the passwords and the adversary's prior information. A phrase generated with chosen words or known public oracle answers, for example, is not covered.

Like a 24-word suite 3 source, a suite 4 source has the structure of honey encryption with a uniform message model at every length. As for suite 3, these results hold for one container and one disclosure in the stated model. They do not cover related containers of both suites for one phrase (see Suite 4: what the narrower state changes), and they have not been independently reviewed.

Limits of the guarantee.

- In suite 3, both theorems compare a prepared disclosure with an honest owner of a 24-word source, and in suite 4 with an honest owner of a phrase of the same length. An adversary who knows that the original seed phrase of a suite 3 container has fewer than 24 words, for example from the device or software that created it, sees that a 24-word reading cannot be the original seed phrase.
- The decoy password must be drawn like a real one. A noticeably weaker or differently formed password is evidence outside the experiments.
- The decoy wallet must follow the same usage scenario as an honest wallet. Its phrase cannot be chosen, so it is a new wallet, and its use has to begin as early and look as ordinary as an honest owner's would. A transfer between the real and the decoy wallet links their records.
- The bounds include the probability of guessing the real password, but not of learning it in another way. If it leaks, the decoy does not help.
- A phrase created with the wallet check `MHFE-WALLET-CHECK-SEED-1` depends on the seed and check functions, and the theorems do not automatically cover it. Every 24-word reading is checked against it. A decoy phrase passes it only in about one case in 65,536, so an adversary who knows that the owner used the check sees the difference. The analysis of that check discusses decoys for such phrases.
- Several containers of one phrase, two different passwords named for one container, a BIP39 passphrase and side channels are not covered. The paragraphs below discuss the first two.

- Cryptography cannot promise that a particular person will believe a disclosure; the theorems only bound what the adversary can learn from the information in the experiments. Where the two disclosures have exactly the same distribution, as in Theorem 1, a disclosure leaves the adversary's belief that a decoy is in use where it was before. It removes evidence of deception, but whether the adversary stops depends on what it expects to gain and what continuing costs it, not on the cryptography. The bound of Theorem 2 limits the average advantage, not the change of belief after every rare observation.

Repeated disclosures and challenges to the disclosed wallet. The theorems cover one disclosure. Naming the same decoy password again adds no cryptographic information: an honest owner repeats the real password, and the second answer equals the first. The same holds for answers computed only from the disclosed phrase and a public challenge, such as an address or an extended public key at a given derivation path, or a message signed with a key of the disclosed wallet. A decoy phrase is a genuine BIP39 phrase whose keys the owner controls. The adversary, who can recover the phrase from the container and the disclosed password, could compute such answers itself, so they cannot increase its advantage. Where the computation calls a function whose queries the bounds count, for example HMAC-SHA-256 when a signature nonce is derived with it, those calls count among the adversary's queries.

A challenge tied to a known address of the real wallet, such as a request to sign with it, is different: the decoy wallet cannot answer it, and knowing that address, like any other external record, is evidence outside the model. What can add information is what changes between demands: the decoy wallet's later history, unless it continues as an honest wallet's would, and evidence that appears in the meantime.

Naming two different passwords for one container, both opening funded wallets, falls outside the model of an honest owner with one password per container. That does not make several decoys for one container useless in every situation, for example when different adversaries are involved, but the present analysis says nothing about it.

Several containers of one phrase. Copies of one container made with the same password and settings are identical. One decoy password therefore opens all of them in the same way, and the copies give no new cryptographic distinguisher, although each copy is another place where the container can be found.

Two containers Y_1 and Y_2 of one phrase made with different passwords or settings behave differently. An honest disclosure opens both to the same wallet, but decoy passwords Q_1 and Q_2 normally open them to two different phrases. For two containers of the same suite and word count, let b be the state width: 256 bits for suite 3 and ENT bits for suite 4. If each fresh password trial gives an independent uniform candidate state, the probability of hitting a fixed decoy phrase after t distinct normalized password trials is $1 - (1 - 2^{-b})^t$, approximately $t \cdot 2^{-b}$ when t is much smaller than 2^b. Thus the scale is $t \cdot 2^{-256}$ for suite 3 and, for example, $t \cdot 2^{-128}$ for a 12-word suite 4 container. This is a heuristic under the stated independence assumption, not a proven bound for related MHFE containers.

With two different decoy phrases, an adversary who knows that the containers hold one phrase sees two wallets where an honest owner shows one. Theorems 1 and 2 concern a single container and do not cover this case: they do not compose for containers whose sources are related. Where deniability matters, further copies of a backup should therefore be made by copying one verified container word for word, with the same password and settings, not by encrypting the phrase again. This also avoids copies that are only as strong as the weaker of two passwords or the cheaper of two settings.

Evidence outside the model. The theorems bound only what the experiments show. The hidden volumes of a deniable file system were exposed by traces that the operating system and applications left, not by their encryption [50], and other records can likewise expose a decoy:

- an unencrypted record of the phrase, such as a paper copy, or a hardware wallet that the owner can be made to unlock, once the adversary finds it and links it to the container, which is immediate if it is kept next to the container phrase;

- extended public keys, output descriptors, saved addresses or watch-only wallets that identify the real wallet;
- notes, photographs, password-manager entries or labels about the backup, including a note that marks a password as a decoy;
- exchange, tax and payment records that link the real wallet to the owner.

Deniability therefore requires every record that the adversary can obtain and link to the container to be consistent with the disclosure. This limit is not specific to MHFE. Blockchain analysis of the two wallets is likewise outside the cryptographic guarantee.

Parameter rationale

Cold storage. MHFE is meant for long-term offline backups that are created once and recovered rarely. Waiting a couple of minutes for such an operation is acceptable, while the attacker repeats the work for each password guess, at a speed that depends on the hardware. The parameters are chosen for that use and not for everyday unlocking.

Memory and passes follow RFC 9106. The first option that RFC 9106 recommends is Argon2id with 2 GiB of memory, four lanes and one pass [4]. Its general procedure is to choose the largest amount of memory the application can afford and then to increase the number of passes until the available time is used. MHFE applies that procedure: 2 GiB and four lanes as recommended, and twelve passes because a cold-storage operation can afford several seconds per round. The twelve passes are this project's choice under that procedure, not a value the RFC prescribes.

On the author's mid-range laptop from 2022 (Intel Core i7-1260P, 16 GB) one Argon2id call with these settings took about 4.6 to 5.0 seconds with the reference C code and about 10 seconds with OpenSSL, using four threads (see Performance measurements). A recovery therefore takes about one to two minutes, and a creation, which ends with a full recovery as its check, takes about twice as long. Newer computers are faster and commonly have far more memory.

More memory is worth more than more passes. For the owner, doubling either doubles the waiting time. In an area-time cost model the attacker's cost grows roughly with memory multiplied by time, so doubling the memory quadruples it while doubling the passes only doubles it. That holds only under assumptions: the attacker is limited by memory capacity, such as a custom chip or a graphics card running many guesses side by side; the time of one guess grows in proportion to its memory; the hardware is fixed; and no favourable memory-time trade-off is used. Under them, compared with suite 2's 512 MiB, one suite 3 guess costs about four times as much for a card limited by memory bandwidth and roughly sixteen times as much for an attacker limited by memory capacity. It is a model, not a guarantee against every attacker.

2 GiB is the right default: most current laptops and desktops can spare it, although a machine with only 4 GB may not once the system and other programs are counted. It is also the most that the reference C implementation of Argon2 accepts when pointers are 32 bits wide, so 32-bit WebAssembly builds support exactly this default and no higher memory level. Users with larger computers can choose more through the memory level in the native program.

Two work-factor settings. The PIM multiplies the pass count; the memory level raises the memory in steps of 1.5 and 4/3, doubling it every two levels. They are separate because memory must fit the computer that will be used for recovery, possibly years later, while waiting time is a matter of patience. A single number that raised memory first would stop a user with a 16 GB laptop at the point where memory runs out, even if that user were willing to wait longer. Argon2id itself, and tools such as KeePass, also expose memory and passes separately.

Both settings serve as headroom. As hardware becomes cheaper, new containers can keep or raise their cost per guess without a new suite.

Why PIM stops at 1023. PIM k multiplies the owner's time roughly by $k + 1$, so 1023 gives a factor of 1,024: ten bits, and roughly 17 to 34 hours per recovery at 2 GiB on the reference laptop. Beyond that, every further doubling adds one bit while doubling a wait that is already measured in days, and a single extra dice word adds almost thirteen bits for

free. The limit is a judgement of what is still sensible, not a technical boundary; Argon2id itself would allow far more passes.

Why the memory level uses half steps. Doubling at every level would leave large gaps: a laptop with 16 GB could safely use 8 GiB but nothing between 8 and 16. Alternating factors of 1.5 and 4/3 give the series 2, 3, 4, 6, 8, 12, 16, 24 GiB and so on. This series fits real machines more closely, keeps every value a whole number of GiB and can be computed exactly with integers: $m(\mathrm{MEM}) = (2 + \mathrm{MEM} \bmod 2) \cdot 2^{20+\mathrm{floor}(\mathrm{MEM}/2)}$ KiB.

Why the memory level stops at 21. Level 21 means $3 \cdot 2^{30}$ KiB, 3 TiB. Level 22 would need 2^{32} KiB, one more than the largest memory Argon2id accepts, so level 21 is a real technical limit. Such amounts are far beyond ordinary personal recovery computers; the upper levels provide headroom for unusually large systems rather than a practical default.

Keeping the settings secret. Recovery needs a non-default setting exactly, so the owner must remember it; nothing requires writing it next to the container. An owner may also keep it secret, as VeraCrypt allows for its PIM. That helps only to the extent that the value is unpredictable to the attacker, and it gives no fixed additional margin.

An example makes the effect concrete under explicit assumptions: N equally likely passwords, a PIM drawn independently and uniformly from $0 \ldots Q-1$, a fixed memory level, a test of one password at PIM j costing $(j+1) \cdot C_0$, a reliable way to recognise the right pair, and no work shared between tests. An attacker who tries all passwords at PIM 0, then all at PIM 1 and so on, which is the best order under these assumptions, pays for a true PIM k

$$\text{Known PIM: } C_{\text{known}}(k) = C_0 \frac{(N+1)(k+1)}{2},$$
$$\text{Hidden PIM: } C_{\text{hidden}}(k) = C_0 \frac{N(k+1)^2 + (k+1)}{2}.$$

For large N the ratio approaches $k+1$: about 1,024 for a true PIM of 1023 in this model, which is a conditional result for that value, not a general gain from the range. Averaged over the uniform PIM the ratio of the mean costs is $(N(2Q+1)+3) \,/\, (3(N+1))$, approximately 683 for $Q = 1024$ and large N. Popular values, the owner's habits or any other guidance reduce the effect, and a guessable choice gains much less.

A forgotten secret setting has to be searched by the owner as well. An owner who knows the password and the memory level, tries PIM 0, 1 and so on, and can recognise a successful recovery pays for a true PIM k about $(k+1)(k+2) \,/\, 2$ times the cost of a default recovery, or $(k+2) \,/\, 2$ times the cost with the PIM known. Recognising success needs the verifier of a short source, which can also give a false match with small probability, or, for a 24-word source, external information such as a known address. For a phrase created with `MHFE-WALLET-CHECK-SEED-1`, every 24-word reading also gets its 16-bit check, which a wrong setting passes in about one case in 65,536. A strong password remains the better investment; a secret setting is an optional extra with its own risk of being forgotten.

Twelve rounds. Total cost could in principle be reached with any round count by adjusting Argon2id, so the round count is chosen for structural reasons:

- The known-pair filter saves one round, one twelfth of a recovery. Theorem 4 gives a success bound that matches this eleven-call strategy up to the fractional credit, error terms and false-match losses stated under Known pairs.
- Twelve sequential steps give natural progress reporting and cancellation points.
- Twelve is above the round counts for which Patarin proves strong bounds for ideal random round functions (seven against chosen plaintexts, ten against chosen plaintexts and ciphertexts) and well above SLIP-0039's four rounds. This is a margin, not a proof for MHFE, because the proofs assume independent random round functions.

Parallel lanes. In the author's measurements the same browser recovered a container in 90 to 95 seconds with the threaded build and in 234 to 239 seconds with the single-threaded one, and one Argon2id call took 4.6 to 5.0 seconds natively with four threads; the measurements give the details. These figures illustrate the effect of threading on one computer rather than establish a general speed ratio. The specification requires implementations to compute the four lanes in parallel wherever threads are available, so the owner can use that parallelism too. The actual benefit depends on the implementation and hardware.

Unicode 18.0.0. Suite 2 pinned Unicode 18.0.0 before Rust normalization tables were available, so its reference implementation could only accept ASCII passwords. Suites 3 and 4 initially pinned Unicode 17.0.0 to support full Unicode passwords with the available libraries. Rust's `unicode-normalization` 0.1.26 now supplies Unicode 18.0.0 tables, and version 0.7.0 of this specification explicitly extends both suites to that repertoire.

Under UAX #15's stability rules, every previously valid password remains valid with the same NFKD UTF-8 bytes; the character refusals and byte limits remain unchanged. Implementations with only the Unicode 17.0.0 admission check reject characters newly assigned in Unicode 18.0.0, so passwords containing them need updated recovery software. This compatibility statement is about the MHFE password. BIP39 passphrases use ordinary NFKD, without MHFE's unassigned-character rejection, so a previously unassigned code point in a passphrase can acquire a decomposition or combining class and change a seed when the normalization tables are upgraded.

The extracted `mhfe-core` 0.1.0 implements the shared password encoding. The separate Unicode 18 corpus records eight fast cases, identifies the checked implementation artifact, and keeps that artifact's results apart from the historical Unicode 17 corpus. It supplements that corpus without changing its bytes. Its normalization checks do not establish the cryptographic security or memory-cleanup properties of an implementation.

Final-word-preserving cycle walking (research idea)

This idea is not part of suite 3; it is recorded because it answers a real wish. The container would end with the same word as the original 24-word seed phrase, so that an owner who remembers the last word of each wallet could tell which container phrase belongs to which wallet: an 11-bit check of the container phrase, not of the password. It would work by cycle walking, applying the permutation again and again until the last word matches, during creation and, in the inverse direction, during recovery. An unreleased suite 2 draft specified it as an optional profile.

It is not used because of what it costs and reveals. A match is expected after about 2,048 complete permutations, with a long tail. With the suite 3 parameters and the measured 70 seconds per recovery, that is about 1.7 days on average on the reference laptop and about 8 days at the 99th percentile (see the detailed analysis), and about twice as long for a creation with its check. These times also make trying variants of a half-remembered password impractical. The last word reveals about 11 bits of information about the source, three of them directly, but does not confirm the password. The walk length gives a timing signal, and the owner must remember that the profile was used. The cheaper variants that come to mind do not avoid these costs. Part II gives the rules, the cost distribution, the cheaper variants, a variant on 253 bits and the known-pair analysis under Final-word-preserving cycle walking for 24-word sources.

Open questions for review

Theorems 1 to 5 have proofs under their stated conditions; Theorem 1's equality also holds for fixed functions. The work below concerns additional models, the applicability of the assumptions and independent validation of the existing results.

Additional mathematical results.

1. Prove a bound like Theorem 3 for 12- to 21-word sources with wallet data that depends on an unknown BIP39 passphrase, and for non-uniform or partly known sources. Theorem 5 covers 24-word sources with an unknown passphrase, with or without the wallet check `MHFE-WALLET-CHECK-SEED-1`, when all funds stay under that passphrase and no data identifies the phrase without it. A bound like Corollary G10 for a phrase created with the wallet check, when such data is known, remains open, and so does a bound with a funded wallet of another MHFE password on the same container.

2. Prove bounds for several containers and several known pairs, and for preprocessing with retained advice. The multi-container game has to be stated first: in a composable model, per-session confidentiality of password-based encryption is unachievable when salting bounds guesses only globally [48].

3. Prove suite 4 analogues of Theorem 4 and Corollary G10; without wallet data a suite 4 container in the core permutation experiment is uniform and independent of the password by Lemma 2. Operational creation additionally refuses fixed points, which occur with probability $2^{-\mathrm{ENT}}$ per first draw in the ideal model; the suite 4 deniability extension accounts for this refusal. In suite 4 the terms that charge a guessed half would have 2^h in place of 2^{128}, which says nothing for $h = 64$ because the salt function can be tabulated; a proof needs an order lemma that charges a guessed half only through the Argon2id evaluation at its salt.

4. Extend the deniability experiments to containers whose sources are related; to adaptive demands repeated while the wallet history develops; to challenges and evidence from outside the disclosure, such as records that link a wallet to its owner; to a BIP39 passphrase; and to phrases created with the wallet check `MHFE-WALLET-CHECK-SEED-1`.

5. Tighten Theorem 4's fractional credit and the gap between Theorem 5 and its matching attacks when narrow tests admit many false matches.

6. Analyse attackers that query the functions in superposition; the games allow classical queries only.

7. If the final-word idea is ever revived, analyse it for shortcuts across repeated applications and for its timing signal.

Model assumptions and concrete resource costs.

- Justify or replace A1 for Argon2id, the independence of the functions, HMAC as a random function, wallet-data derivation as a random function (Corollary G10 and Theorem 5), and the BIP39 seed and wallet-check digest as random functions (Theorem 5). Analyse side channels separately.
- Translate Argon2id call counts into memory and time through a resource model and a proven bound for Argon2id. Check the practical cost estimates against real graphics cards, published Argon2id cracking figures and the memory-time trade-offs discussed in RFC 9106; measurements alone do not establish a lower bound.

Independent validation.

- Review Theorems 1 to 5 and the corollaries and propositions, including the deniability extension to same-length decoys in suite 4.
- Maintain the conformance coverage and reproducible generator/verifier provenance of the suite 3 corpus as the specification evolves, retain independent full-cost replay evidence, and obtain an implementation by another author. The current corpus records OpenSSL 3.5.5 replay of all 17 positive transcripts and seven negative recovery cases, one of them under the current length rules. This is independent primitive-level reproduction, not an independently authored MHFE implementation.
- Review the Unicode 18.0.0 assignment check in a real implementation, including newly assigned characters and byte-for-byte agreement for passwords accepted under Unicode 17.0.0.

Behavioural research. Determine when a decoy changes the decision of a rational adversary, as a Bayesian signalling game in which the decoy wallet's balance and history are costly signals and the adversary has a prior belief about the owner's wealth, comparing pooling, separating and semi-separating equilibria. This question concerns the adversary's decisions beyond the disclosure experiments proved here.

Suite 4: what the narrower state changes

Suite 4 keeps the source's length. Its state is the source entropy, ENT = 128, 160, 192 or 224 bits, split into halves of $h = \mathrm{ENT} \,/\, 2$ = 64, 80, 96 or 112 bits, and it has no verifier. This section examines what the narrower halves change compared with suite 3.

The deniability theorems of Part I extend to suite 4, as stated in the suite 4 extension. The guessing bounds with wallet data or a known pair, Corollary G10 and Theorem 4, do not extend. In suite 4 the terms that charge a guessed half would have 2^{h} in place of 2^{128}, which says nothing for $h = 64$ because the salt function can be tabulated, and no suite 4 proof has been written for any width (appendix). Without wallet data, exact password independence holds for the core permutation (Lemma 2). The operational creation refusal adds the $2^{-\mathrm{ENT}}$ exception per first draw, and the deniability extension accounts for this exception. The procedure costs below are therefore not proven lower bounds for suite 4.

Salt diversity. For fixed settings and round index a salt is a function of one h-bit half, so it takes at most 2^{h} values, 2^{64} for a 12-word source. This is an upper bound on its diversity, not a guarantee of h bits of randomness. RFC 9106 permits a 64-bit salt length under space constraints [4], but that is context, not a security justification for suite 4. Suite 4 salts remain 128 bits long but are derived from an h-bit state half, and their distribution, correlations and reuse require construction-specific analysis.

Containers that share a salt. In a model where only containers are known, the salt of the last round comes from the container's left half L_{12}, which anyone holding the container can read. Take containers with the same ENT and settings whose halves are independent and uniformly random. The expected number of pairs with equal left halves among N such containers is $N(N-1) \,/\, 2^{h+1}$. For $N = 2^{32}$ containers of 12 words, this is about one half, with a probability of about 39 % that at least one such pair exists, and correspondingly fewer for longer sources.

An ideal random permutation maps any fixed source to a uniform container, but the actual construction's bijectivity alone does not establish this distribution. A fixed permutation maps uniform source entropy to uniform container entropy, while nonuniform or related sources require an additional argument.

Testing one password guess by full recovery on both containers of a pair with equal left halves takes 23 Argon2id calls instead of 24. A group of g containers with the same left half saves $g - 1$ calls for the last round. Coincidences of the salts of other rounds depend on states that change with every password. Such coincidences cannot be found in advance, but they can be detected during a search, and with a known source the salt of the first round is known as well. Whether such coincidences allow a larger saving across many containers has not been shown either way.

Precomputation. A table that covers every possible input for likely passwords is not feasible. For a 12-word container, covering every possible half-state requires evaluating 2^{64} inputs for each password at fixed settings and round index. Each Argon2id call uses the selected memory and pass settings, with 2 GiB at the default memory level. Partial tables for chosen passwords and observed salts are possible. Their value depends on how often salts repeat, as in the previous paragraphs.

Known pairs. The filter of Observation 2 works for any width. With an original seed phrase and its container, it tests a guess with 11 Argon2id calls. Its comparison now covers h bits. In the model of independent uniform round functions, a wrong password passes it with probability 2^{-h}. For an exhaustive search over five EFF words, 7776^{5} candidates, the

expected number of false matches is then about 1.5 for 12 words. A candidate that passes the filter is checked by computing the skipped round, which is one more Argon2id call if the states were kept, and by comparing the other half.

Generic Feistel bounds. The results for ideal Feistel networks, such as Patarin's [29], bound an adversary that obtains values of the permutation through an encryption or decryption oracle without knowing the key. They hold under that theorem's assumptions and for q much smaller than 2^h queries. In the scenario of one stolen container no oracle is assumed, and known pairs exist only where a password was reused or a source leaked. These query bounds therefore cannot be read directly as a number of password guesses. The random-function assumptions behind any such bound remain as open for suite 4 as for suite 3.

Deniability. Every ENT-bit state is valid entropy for a phrase of the source's length, so the reasoning of the consistency and uniformity lemmas of Part I applies in the smaller space. Every password opens a suite 4 container to a valid phrase of the same length, and the permutation for that password maps it back. A decoy therefore keeps the source's length. This removes the suite 3 objection that a known short original seed phrase exposes a 24-word decoy. The experiments and both theorems of Part I extend to suite 4 at every length, as stated in the suite 4 extension.

These results do not carry over to related containers of different suites. If one phrase is encrypted under both, an adversary who finds the short suite 4 container learns the length of the original seed phrase. That length can expose a decoy disclosure of the suite 3 container if the disclosure relies on a 24-word reading. Where deniability matters, further backups should be exact copies of one container.

Copying errors. A short container has a BIP39 checksum of $\mathrm{ENT} \,/\, 32$ = 4, 5, 6 or 7 bits. A word replaced at random during copying still passes it in about one case in 16, 32, 64 or 128. The container then recovers a different valid wallet without any error. Checking the finished backup against a known address matters more than in suite 3.

What follows. The narrower state reduces the number of possible salts. A full recovery uses twelve Argon2id calls, and the known-pair filter above uses eleven. These are costs of the stated procedures, not lower bounds on all attacks against suite 4. Provided that no cheaper attack on the construction exists, the protection against password guessing is set by the password and the cost of Argon2id, as Theorems 3 and 4 prove for suite 3 in the random-oracle model. The analysis has not been independently reviewed.

Part II. Detailed design discussion

The sections below develop the design in more detail; the introduction gives their history. Unless a statement names suite 4, it concerns suite 3. Suite 4 is covered by Suite 4: what the narrower state changes. Cross-references such as "**Reference Implementation**" refer to sections of this Part. Citation numbers refer to the single reference list in `README.md`.

For the requirements, use the specification; Part I gives the main analysis.

Part I and the appendix give the proved results and their conditions. The cited Feistel results assume independent random round functions and do not transfer to MHFE automatically. Estimates labelled as heuristics retain their stated assumptions. The consolidated security claim is under Status and security claim.

Motivation

The specification's Motivation gives the practical purpose, and its Suite parameters and Application requirements say what recovery needs. This section adds the background.

Practical purpose: protecting a physical backup without expanding it

The practical goal is to keep a recovery phrase on a familiar, capacity-limited physical backup while making possession or a photograph of that backup insufficient, by itself, to recover the original seed phrase.

Physical seed-backup products such as Cryptosteel, and comparable metal backups or capsules, provide a finite number of character or word positions. Cryptosteel's own instructions describe storing longer BIP39 phrases using abbreviated words [5]. Such media cannot accommodate arbitrary expansion without changing the storage arrangement. A password-encrypted container that remains a valid 24-word BIP39 phrase could use the same word-oriented recording method, subject to the medium's capacity and supported wordlist. This is the practical reason for investigating a fixed-size format rather than simply adding more fields to a backup.

The 24-word container also has the ordinary syntax of a valid BIP39 mnemonic and carries no in-band marker identifying it as encrypted. A casual observer may therefore take it for an ordinary recovery phrase and may not realize that another mnemonic is concealed behind it. This ambiguity can reduce opportunistic attention and avoid revealing the encryption workflow itself.

The container's ordinary appearance provides practical concealment. By itself, that appearance is not the formal plausible-deniability claim analysed in Part I. Storage context, accompanying instructions, known wallet addresses, repeated containers, or prior knowledge of MHFE may reveal the record's purpose and permit password candidates to be tested at the construction's KDF cost. Part I analyses decoy disclosures, which remain possible when the use of MHFE is known.

The password, and a BIP39 passphrase if the wallet uses one, are kept apart from the container: memorized or recorded in a separate place, possibly in a discreet form meaningful to the owner. This preserves both confidentiality and the container's ordinary appearance. A disguised written password, however, is no substitute for resistance to guessing.

Author's motivation and hypothesis: the author considers managing one sufficiently strong, memorable password potentially easier than memorizing 12 or 24 mnemonic words in their exact order. A separate, discreet password record may also be easier to manage than another full mnemonic record. On that basis, the author considers an encrypted physical backup with a separately managed password potentially safer against accidental viewing or photography than keeping the complete recovery phrase openly readable. This is the author's usability and security hypothesis that motivates the research. It is not a measured user-study result or a claim that the current MHFE construction has proven security.

Prior problem statement and community context

BIP39 provides a compact human-readable representation of wallet entropy [3], but it does not define an in-place encryption format for an existing mnemonic. As the specification's Motivation notes, a Bitcoin Stack Exchange discussion in May 2021 posed almost the same goal and linked an experimental AES-CTR prototype [8], [10]. That prototype is analysed in Earlier BIP39 backup-encryption and obfuscation proposals.

The discussion and its linked prototype stop short of a complete, reviewed, interoperable proposal. They do not provide a stable format, a memory-hard KDF profile, comprehensive test vectors, or a construction-specific security analysis. This unresolved discussion is the closest historical anchor for presenting MHFE to technical forums such as Delving Bitcoin. MHFE treats it as evidence that the use case predates this specification, not as validation of the present construction.

Baseline design goals

Conventional password-encryption formats normally store additional information such as a random salt, nonce, version, and authentication tag. The baseline goal here is narrower: emit one exact 24-word BIP39 container while making recovery depend on a separate password.

The baseline construction targets the following constraints; research alternatives explicitly identify which constraints they retain or relax:

- the plaintext is a valid 12-, 15-, 18-, 21-, or 24-word BIP39 mnemonic and the encrypted container is a valid 24-word BIP39 mnemonic;

- the underlying transform is a permutation over all 256-bit BIP39 entropy values;
- no per-container bits are available outside that 256-bit space;
- the password is the only user-held secret required by the transform itself;
- password guessing should require memory-hard work;
- short-source recovery verification must be clearly distinguished from AEAD authentication.

Practical recovery requirements

The specification states what recovery needs. With the default settings, that is only the container and the password. A non-default PIM or memory level must also be remembered, and in a rare case so must the word count of the original seed phrase (see Suite parameters and Recovering a mnemonic). A BIP39 passphrase, if the wallet uses one, is not part of MHFE and cannot be reconstructed from the container.

Because the container does not identify itself as MHFE, recovery also needs compatible software and the knowledge that the record is a container. Losing access discusses what this means for heirs. Before the backup of the original seed phrase is retired, the specification asks for a rehearsal from the finished backup and a comparison with known wallet data (see Application requirements).

Notes on the construction

These explanatory paragraphs accompany the normative text of the specification.

Salt and mask lengths

The 128-bit lengths of S_i and M_i do not imply 128-bit variants of the underlying cryptographic primitives. S_i is truncated from a 256-bit BLAKE2b digest, K_i is the full 256-bit Argon2id output, and M_i is truncated from a 256-bit HMAC-SHA-256 output. The mask must be 128 bits because it is XORed with one 128-bit half of the fixed 256-bit Feistel state. Expanding that mask to 256 bits would require a 512-bit state and would no longer fit a 24-word BIP39 container. Expanding the salt to 256 bits would not add independent entropy, because its variable Feistel-branch input is only 128 bits.

Argon2id lanes and default cost

RFC 9106's first and second recommended Argon2id options both use $p = 4$, and its general parameter-selection procedure likewise selects four lanes [4]. Suite 3 therefore selects $p = 4$. Reducing p merely to lengthen wall-clock time is not presumed to improve password-guessing resistance, because an attacker can parallelize independent password candidates and because changing p changes the Argon2 function itself.

Suite 3's default tuple of $m(0)$ = 2 GiB, $t(0)$ = 12 passes and four lanes is not one of RFC 9106's two recommended tuples. It keeps the memory and the four lanes of the RFC's first recommended option and deliberately raises that option's single pass to twelve for an infrequent high-cost backup operation. With four lanes, 2 GiB is the total Argon2 memory, nominally 512 MiB per lane; it is not 2 GiB per lane.

One MHFE permutation or inverse performs twelve sequential Argon2id calls; an encryption with its required check performs twenty-four. If one working buffer is reused, its peak Argon2 allocation is nominally $m(\mathrm{MEM})$, 2 GiB at the default memory level, regardless of PIM. At the default settings, the nominal full-memory-pass volume of one permutation is $12 \cdot 12 \cdot 2\mathrm{GiB} = 288\mathrm{GiB}$. In general it is $288\mathrm{GiB} \cdot (\mathrm{PIM} + 1) \cdot m(\mathrm{MEM}) \,/\, 2\mathrm{GiB}$. These values are not runtime predictions or attack-cost proofs. The mapping and its permitted ranges are exact and fixed for suite 3. Their safety and usability, including the suitability of the upper limits, remain provisional until the settings are measured across representative target systems and reviewed in the full construction.

Domain separation

The explicit purpose labels, $\mathrm{BE32}(\mathrm{MEM})$, $\mathrm{BE32}(\mathrm{PIM})$, and $\mathrm{BE32}(i)$ separate salt derivation from mask derivation, different permitted work factors, other protocols, and other rounds. This does **not** establish security against a named class of Feistel attack. It prevents accidental reuse of one byte-level domain for different protocol roles.

`SUITE_ID` identifies the complete experimental cryptographic suite, not merely an editorial revision of this document. Any incompatible change to the Feistel geometry or the round count N, which is 12; state packing or encoding; password normalization or limits; salt derivation; the Argon2 variant, version, fixed parameters, the permitted ranges or cost mappings of PIM and the memory level; or $\mathrm{RoundMask}$ requires a new `SUITE_ID`, `DS_SALT`, and `DS_MASK`. Selecting a different permitted PIM or memory level under the unchanged mappings does not define a new suite. An identifier must not be reused for a changed mapping.

The specified Unicode 17-to-18 repertoire extension is compatible because it preserves both admission and the exact P_{enc} of every previously valid password, with the other suite rules unchanged. It therefore keeps the suite identifiers and domain strings. It does not permit an implicit upgrade to later Unicode versions.

Mnemonic and seed

A **BIP39 mnemonic** is not itself the BIP39 seed. This construction packs the entropy encoded by a valid BIP39 mnemonic into a 256-bit state. After decryption, normal BIP39 seed derivation may still use the separate optional BIP39 passphrase. Version 0.7.0 also uses Unicode 18.0.0 for the NFKD in that seed derivation. Earlier implementations could accept unassigned code points in the passphrase, unlike in the MHFE password, so newly assigned characters can change the passphrase's normalized bytes and the derived seed. The recovered mnemonic is unchanged, but such an existing wallet needs its original normalization behavior for recovery, as the specification's wallet-check section explains.

Only the end salts are public

S_i is an actual Argon2 salt input, but it is deterministically derived from the Feistel state. It does not add entropy and is not guaranteed to be globally unique. Only the salts at the two ends of the chain can be computed without the password. For a known plaintext, R_0 and therefore S_0 are known. When inversion begins from a ciphertext, $R_{N-1} = L_N$ is likewise available from the ciphertext state. Every other salt depends on the password and is as sensitive as the state it comes from.

A leaked intermediate salt S_i allows a filter that works backwards from the container and needs $11 - i$ Argon2 calls with twelve rounds: one for S_{10}, eleven for S_0. With a known source the filter can also start from the other end, so that S_1 is checked after one forward round. Leaked intermediate values lie outside the model of Theorems 3 and 4, so these are the costs of the described filters, not proven bounds. The term **state-derived salt** is used throughout this document instead of "pseudo-salt".

Choice of HMAC

HMAC-SHA-256 is a conventional keyed function. The complete effective MHFE round function, however, derives its HMAC key from its own input through Argon2id, so the choice of HMAC alone does not make the effective round functions independent random functions.

Rationale

Why Feistel

The core requirement is a reversible mapping from exactly 256 bits to exactly 256 bits. Feistel networks provide a convenient way to construct a permutation from a round function that does not itself need to be invertible. For one round:

$$\begin{aligned}\text{Input: } & L_i \| R_i, \\ \text{Mask: } & M_i = F_i(R_i), \\ \text{Output: } & R_i \| (L_i \oplus M_i).\end{aligned}$$

The right half remains available after the round as the next left half. This property is what allows a salt derived from R_i to be recomputed during decryption.

Why twelve rounds

The round count is informed by results for ideal balanced random Feistel schemes, but those results must be stated with their attack models. Patarin [28] proves near-full-branch security for seven or more rounds against adaptive chosen-plaintext attacks. The same paper states ten or more rounds for adaptive chosen-plaintext-and-ciphertext attacks. A later result [29] establishes its stated chosen-plaintext-and-ciphertext bound for six or more rounds under that paper's conditions.

$N = 12$ is therefore the suite 3 choice: it is six rounds above the six-round threshold in [29], five rounds above the seven-round CPA threshold in [28], and two rounds above the distinct ten-round CPCA threshold stated in [28]. These differences are engineering margins, because MHFE uses password-, settings- and state-dependent effective functions $G_{i,P,\mathrm{MEM},\mathrm{PIM}}$ rather than independent random ones. For password guessing against one container or one known pair in the random-oracle model, Theorems 3 and 4 and Corollary G10 bound every attack by the strategies that spend all twelve evaluations per password, or eleven with a known pair, up to the terms stated there. As a margin for other properties, such as the pseudorandomness of the permutation, twelve rounds are an engineering candidate rather than a theorem-backed security level.

Why state-derived salts

A conventional password-encryption format stores a random salt. The zero-metadata requirement removes that option for the full 24-word domain. MHFE instead derives the Argon2 salt from state that is available in both encryption and decryption.

This does not create new entropy. It diversifies the KDF per state without storing an additional field. Take two independently sampled states compared at the same settings and round index. Their salts can be equal for two reasons: their 128-bit R_i values are equal, or unequal inputs collide after BLAKE2b-256 is truncated to 128 bits. If the branches are modeled as independent uniform 128-bit values and BLAKE2b as a random function, the combined probability is close to 2^{-127} for one pair. The corresponding birthday scale is approximately $2^{63.5}$ comparable invocations. Different settings or round indices make the hash inputs distinct, so in that model only the truncated-hash collision probability of approximately 2^{-128} remains. Accidental collisions are therefore expected to be negligible at any realistic number of containers.

MHFE does not claim conformance to NIST SP 800-132, which requires the randomly generated part of a PBKDF2 salt to have at least 128 bits [20]. For Argon2id the more direct guide is RFC 9106, which recommends a 128-bit salt for password hashing and a salt unique for each password [4]. MHFE salts have 128 bits but are derived from the state, not generated at random: for a fixed password, the permutation must map every 256-bit source to exactly one 256-bit container, which leaves no room for fresh randomness. Uniqueness is therefore a property to be argued, not one given by construction.

The estimate above, about 2^{-127} for one pair of salts at the same settings and round index under the stated model, covers accidental repetition between containers of independently generated sources. It does not by itself make derived salts equivalent to independently generated ones. A reviewer would also ask whether an adversary can force salts to repeat, whether related states let expensive evaluations be reused, and what changes for identical sources, short sources and chosen inputs. The threat models below and the open questions of Part I address parts of this.

SLIP-0039 uses the same kind of salt, the right half of the state, for its extendable backups [19]. This shows that the technique is not unusual, but it does not establish the security of MHFE.

Suite 4 keeps a 12-word source at its own length, with 64-bit halves (see Shorter BIP39 mnemonics), and has at most 2^{64} possible salt inputs per round. The hashed salt remains 128 bits long, but its diversity is limited to at most 2^{64} values, not guaranteed 64 bits of randomness. This requires a separate analysis; see Suite 4: what the narrower state changes.

These estimates treat intermediate Feistel branches as independent and uniform. Reprocessing the same state with the same password, settings and round index repeats the same salt by design; that is not an accidental collision.

Research note: public pre-mixing of the KDF schedule

Public reversible full-state pre-mixing was considered as a way to make the first state-derived KDF input depend syntactically on the complete source state. No current profile selects it. For a uniformly random 256-bit source it has little apparent practical benefit: the original 128-bit right branch already gives a distinct-source pair-collision probability of 2^{-128} and a birthday collision scale of approximately 2^{64} independently sampled sources.

Pre-mixing also cannot prevent adversarially constructed branch collisions, because the transform would be public and invertible. Nor would it add entropy or remove the deterministic relation between the two halves of a packed short source. The idea is retained only as a record of a considered alternative. It should not add another cryptographic operation to a suite without a demonstrated benefit and separate analysis.

BIP39 checksum semantics

A 24-word BIP39 mnemonic contains 256 entropy bits and an 8-bit checksum, but only 2^{256} 24-word sequences are valid. The checksum does not provide 8 extra payload bits.

The baseline transform therefore operates only on the 256-bit entropy and recomputes the standard checksum afterward. This preserves the BIP39 word count but leaves no independent room for an authentication tag or version field.

For shorter sources, the universal packing of suite 3 places V_r in the otherwise unoccupied part of that same 256-bit state. This does not alter the outer BIP39 rule: every suite 3 container still carries the ordinary 8-bit checksum of Y outside the 256-bit encrypted payload.

Baseline and research alternatives

This document keeps one precisely identified baseline, suite 3, while examining competing directions. The research section compares balanced and source-heavy unbalanced Feistel, as well as different payload-verification choices. Keeping alternatives in one document is intentional, but an interoperable implementation would still need to select and identify a complete profile.

Backward Compatibility

This construction changes no Bitcoin consensus, peer-to-peer, or RPC rules. Compatibility concerns are entirely at the wallet/application layer.

An MHFE-encrypted 24-word mnemonic is intentionally syntactically valid BIP39. A legacy wallet that supports 24-word English BIP39 mnemonics will generally accept the encrypted mnemonic as an ordinary mnemonic and generally derive a different wallet from it; the construction does not rule out fixed points. The encrypted mnemonic contains no in-band flag that says "decrypt me first".

Fixed points are an expected property of the idealized permutation model, not by themselves a structural failure of the design. A uniformly random permutation on a finite set has exactly one fixed point in expectation. As the set grows, its fixed-point count approaches a Poisson distribution with parameter 1. For one particular 256-bit state X, however, the probability that $\mathrm{Perm}_{P,\mathrm{MEM},\mathrm{PIM}}(X) = X$ is only 2^{-256} in that model. A ciphertext-only observer cannot determine from Y alone whether it is such a fixed point of the unknown permutation parameterized by the password and the settings. The existence of fixed points therefore does not supply a generic password test or key-recovery shortcut.

The rare equality still matters operationally, because it defeats concealment for that concrete state. For a 24-word source, $Y = X$ makes the encrypted mnemonic identical to the source mnemonic. For a shorter source it would expose the complete packed state $X = E \| V_r$, even though the word counts differ. The specification therefore requires creation to compare the input and output states, to refuse to present an unchanged state as an encrypted backup, and to ask for a different password, PIM or memory level. Reapplying the same suite with the same password, settings, and input cannot escape the same fixed point. These operational checks do not replace analysis of whether the concrete MHFE construction behaves sufficiently like the idealized permutation.

Applications implementing MHFE must therefore present encrypted mnemonics as a distinct workflow and must not silently pass them to normal BIP39 seed derivation. Users must know that a backup is MHFE-encrypted; compatible software supplies the exact suite definition rather than requiring the user to memorize its literal identifier.

With the correct password and a compatible suite implementation, MHFE decryption recovers the original seed phrase for use by unmodified BIP39-compatible software. For a short source, the recovery-verifier check supplies a failure signal for most incorrect candidates. For a 24-word source, merely completing decryption does not verify that the inputs were correct. If that wallet also uses the optional BIP39 passphrase, the BIP39 passphrase remains an independent second input and is applied only after MHFE decryption.

Security Considerations

Status and security claim

Part I proves deniability results for suites 3 and 4 in the stated disclosure experiments (Theorems 1 and 2 and their extensions). The appendix proves guessing-success bounds for suite 3: one container of a uniform short source (Theorem 3), a known pair (Theorem 4), a uniform source checked against wallet data with an empty or known BIP39 passphrase (Corollary G10), and a 24-word source with an unknown BIP39 passphrase, with or without the wallet check (Theorem 5). The scope tables give the conditions and separate the recovery goals.

These results establish the specified guarantees under their hypotheses. Their principal limits are:

- the guessing bounds use ideal functions and count calls; they do not prove the concrete primitives' behaviour or physical memory and time costs;
- pseudorandomness of the full MHFE permutation (PRP/SPRP) is a separate property. The classical Feistel results assume independent random round functions, which has not been established for MHFE's Argon2id-derived functions;
- no AEAD authentication and no built-in wrong-password detection for a 24-word source; the optional source profile adds only a statistical check, with the limits analysed below;
- suite 4 has no built-in verifier at any length and narrower halves with less salt diversity; its additional limits are discussed in the narrower-state analysis;

- the deniability results concern the stated disclosure experiments; they do not establish a general honey-encryption [43] guarantee or predict whether an adversary believes a disclosure.

The open questions and A.7 list additional models and validation work. MHFE remains experimental and has not been reviewed by an independent cryptographer. No real funds should depend on it.

Resource exhaustion and untrusted containers

A checksum-valid input of a word count admitted by the selected suite can force a decoder to perform every expensive Argon2id operation required by that suite. The outer BIP39 checksum filters transcription errors, but it is not authorization to consume unbounded resources. Both suites fix their KDF parameters for each setting and bound the length of the normalized password. Implementations must additionally enforce a local resource ceiling and fail rather than substitute cheaper parameters.

Applications should start recovery only after an explicit user action, should keep the interface responsive during long operations, and should offer cancellation where the execution environment permits it. A worker or background thread is a responsiveness boundary, not a cryptographic vault. Before allocating large amounts of memory, implementations must reject unknown suite identifiers, a PIM outside $0 \ldots 1023$, a memory level outside $0 \ldots 21$ or above what they support, and any other caller-supplied parameter override. These measures limit denial-of-service and accidental resource use. They do not reduce the attacker's offline password-guessing cost.

Faults during computation

A hardware error, or an attacker able to cause one, may corrupt a round during creation. The container would then not decrypt to the source, and the backup could be lost. The specification therefore requires creation to decode the new container again and invert it, with twelve further Argon2id calls computed afresh, and to report a mismatch. An application may show the container while this check runs, marked as not yet verified.

The check detects many accidental errors, including a transient fault in either half, since the same fault would have to recur at the matching place of the inverse. It does not detect a fault that disables the comparison. Nor does it detect a persistent fault, such as failing memory or a miscompiled build, that affects the permutation and its inverse in the same way. Decrypting the container on a second computer detects a fault of the first computer's hardware. Decrypting it with an independent implementation detects an error in the software, including a miscompiled build, which the same build would repeat on any computer. Resistance to deliberately induced faults, which an attacker controlling the computer could repeat to collect several faulty results, is not established.

Threat models

The following models must be kept distinct. Unless a model states otherwise, the attacker is assumed to know the suite and the public settings. The original source length may be known, tried explicitly, or inferred with the same recovery-verifier rules available to the owner. Known wallet identity data can provide an external password test, especially for a 24-word source.

T1 — single-container offline guessing. The attacker has one encrypted entropy Y. A baseline attack evaluates a candidate inverse permutation $\mathrm{Perm}^{-1}_{P',\mathrm{MEM},\mathrm{PIM}}(Y)$ for each password guess P' and then checks a short-source V_r relation or public wallet data of the original phrase, such as a known address. For suite 3, a uniform source and the empty or a known BIP39 passphrase, Theorem 3 and Corollary G10 bound success in the random-oracle model. The baseline attack matches those bounds up to the stated error terms and false-match losses. Evidence about the source itself belongs to T8.

T2 — same-password multi-container, ciphertext-only. Many ciphertexts were created under the same normalized password, settings, and suite, but their plaintext mnemonics are unknown. This is not automatically equivalent to a chosen-plaintext or known-pair oracle. Determinism reveals ciphertext equality when the packed plaintext is repeated,

and any attack may also consider the structured short-source subsets. But unknown plaintexts do not become known pairs merely because many containers are available.

T3 — disclosed known pairs. The attacker knows one or more specific mappings (X_i, Y_i) made under the same password, settings, and suite, for example because the plaintext mnemonic was later disclosed or compromised by another channel. Such pairs may be used to filter password guesses or to attack other containers governed by the same permutation.

T4 — chosen-input oracle access. The attacker can obtain forward transformations of chosen plaintexts, inverse transformations of chosen ciphertexts, or both, under one fixed password, settings, and suite. Forward-only access is closest to classical PRP analysis, while access to both complete directions is closest to SPRP analysis. An interface that reveals only verifier acceptance is a weaker oracle, but it may still provide a useful password filter.

T5 — cross-container aggregation under a shared weak password. Independently leaked known pairs from different users or wallets form one permutation only when the same normalized password, settings, and suite are shared. If the same weak password is reused with different settings or suites, those samples belong to different permutations and do not form one classical multi-query transcript. They may still be combined as evidence in a dictionary attack on the shared password, by evaluating each applicable parameter set. A large corpus created under unrelated passwords provides no such shared-password test. Related but different passwords, such as a common root with different endings or a small edit of an old password, select independent permutations in the random-oracle model. Learning one of them therefore gives no cryptographic shortcut to the others. It does narrow the attacker's dictionary for the others, often drastically; independent passwords for different containers avoid this.

T6 — active container modification or substitution. The attacker can replace or alter a recorded container and recompute its ordinary BIP39 checksum. Suite 3 provides no cryptographic authentication. A short-source verifier rejects most incorrect recovered states but does not authenticate the container or its origin; a 24-word source has no internal verifier at all. This integrity and availability threat must be analyzed separately from offline password guessing and confidentiality. The checks that a recovery can use trust different things, and none of them alone protects against substitution:

Check	What it confirms	What it does not confirm
The container's BIP39 checksum	The words were recorded and read without most accidental errors	That the container is genuine: anyone can recompute it
The recovery verifier of a short source	The recovered state satisfies the verifier relation for the detected length	Authenticity: whoever knows the password can build another passing container
The creation check, decoding the words and inverting	This implementation and computer produced a consistent container	Anything about a copy that is replaced later
A known receiving address	The recovered phrase belongs to the expected wallet	Anything, if the address itself came from an untrusted source
The BIP32 master key fingerprint	A 32-bit match: a wrong phrase passes with probability about 2^{-32}	That the match is not deliberate: about 2^{32} trials find a phrase with a given fingerprint
The 16-bit check of `MHFE-WALLET-CHECK-SEED-1` on a 24-word reading	A 16-bit match, if the phrase was created with that profile: a wrong phrase or BIP39 passphrase passes with probability about 2^{-16}	Anything, if the phrase was created without the profile; that the match is not deliberate: about 2^{16} trials find a passing phrase

T7 — old containers after a change. A container stays useful to an attacker for as long as its original seed phrase controls funds. Encrypting the same source again under a new password, higher settings or a new suite does not neutralize an old container that was copied or stolen. The old container can still be attacked with its own password and

settings, and it recovers the same wallet. This differs from T5, because the old and the new passwords may be unrelated. The practical protection is that of the weakest copy an attacker can obtain. A new password protects the funds only after they are moved to a new original seed phrase, or after every old copy is destroyed.

T8 — partly known or weak source. The analysis assumes a uniformly random source that the attacker does not know. If part of the source leaks, the remaining uncertainty can be small. For example, when the first 11 words of a 12-word phrase are known, the last word carries 7 entropy bits and the 4-bit checksum, so 128 completions remain. With a known address and no BIP39 passphrase, those completions can be tested without MHFE at all. A strong container password does not undo a leak of the source itself, and a source generated with little entropy is weak in the same way. The bounds of Part I describe a uniform source; extending them to partial leaks would require including that information in the model explicitly.

T9 — leaked intermediate values. The attacker obtains part of a computation: a salt, a state between rounds, a round key or mask, or a dump of the Argon2id work area, for example from a log, an error report, a file kept to resume a long operation, or swap. Leaks of intermediate values gives the cost of a password test in each case.

T10 — dishonest implementation. The software used for creation or recovery was modified to leak secrets or to write containers that differ from the specification. Determinism as a check on implementations explains what a second implementation can detect.

T11 — attacker in the future. The container is photographed now and attacked years later, with cheaper hardware or better cryptanalysis. Attacks in the future estimates the effect.

T12 — observer on the same computer. Another process, or a script in another browser tab, shares the computer during creation or recovery and measures cache timing. Observers on the same computer estimates what it gains.

T13 — password known from elsewhere. The owner also used the password for another service, or it appears in a published list of breached passwords. Such lists are tried first, and the attack then costs only as many recoveries as the list is long. In the model of Part I such a password simply has a very high probability; the practical rule is a password used nowhere else.

T14 — faults during computation. A hardware error, or an attacker able to cause one, corrupts a round during creation or recovery. Faults during computation describes what the creation check detects.

T15 — many owners at once. The attacker collects the containers of many independent owners and needs to open any one of them. Many containers of many owners explains why salts derived from the state limit what the population gives the attacker.

Where known pairs can come from

A known pair is not free information. Realistic sources include later voluntary disclosure, inheritance, audit, migration, independent compromise of a plaintext backup, malware observing a later recovery, or deliberately published test material. The attacker must be able to link a specific plaintext X_i to a specific encrypted container Y_i.

Many containers of many owners

An attacker who collects many containers may be satisfied with opening any one of them. Consider the direct password-guessing procedure for independently generated suite 3 containers, under the stated random-function model. Unless password candidates and salts coincide, that procedure evaluates Argon2id separately for each container. Accidental salt collisions are expected to be negligible at realistic scales in this model. This is not a proof excluding other ways to share work, nor a lower bound on every multi-container attack. Suite 4 has narrower salt diversity and its own analysis.

Suppose that the passwords are drawn independently and uniformly from 2^n values, and that the attacker fixes in advance b_i distinct guesses for container i, B guesses in total. Container i is then opened with probability $b_i / 2^n$, so the expected number of containers opened is exactly $B / 2^n$, however the budget is split, and the probability of opening at least one is at most the same. An adaptive attacker does better, because after a success it wastes no guesses on an opened container. For example, take two containers whose passwords have two possible values each, and a budget of two guesses. An attacker who guesses on the first container until it opens, and only then on the second, opens 1.25 containers on average, against 1 for any split fixed in advance. The gain is small while each container receives only a small fraction of its password space.

The real leverage comes from non-uniform passwords. With passwords that people choose, the attacker spends the budget on the most likely ones across all containers and opens the weakest, as in the cracking of stored password hashes. It can also prefer containers that it expects to use cheaper settings or to hold more funds. The protection of a population is therefore that of its weakest passwords, and the default settings that most owners keep set its price.

Known-plaintext shortcut and KDF cost

N Feistel rounds do not by themselves prove a lower bound of N expensive Argon2 evaluations per wrong password guess. For MHFE's suite 3 with $N = 12$, Theorem 4 bounds guessing success for an attacker with a known pair, and Theorem 3 with Corollary G10 bounds it for an otherwise unknown uniform source. In the random-oracle model these bounds match strategies that use $N - 1$ and N calls per tested password, respectively, up to their stated error terms, fractional credit and false-match losses.

Given a known plaintext/ciphertext pair, an attacker can compute forward from X through some rounds and backward from Y through the remaining rounds. At a skipped round, the Feistel relation

$$L_{i+1} = R_i$$

can be checked without evaluating that round's mask. When the compared halves depend on the password guess, this check is a password filter that omits a selected expensive round. For example, omitting the only round of a one-round network gives no password-dependent filter. The rejection rate and the best multi-round shortcut depend on the full construction; for MHFE they are given by Observation 2 and Theorem 4.

In a multiround design whose earlier rounds are inexpensive and depend on the password, making only the last round memory-hard lets an attacker first apply a known-pair password filter that omits the expensive round. How well that filter works still depends on the preceding rounds.

Leaks of intermediate values

The attacker always has the container Y. A leaked intermediate value gives the attacker an explicit way to test a password guess P' more cheaply than a full recovery, which costs twelve Argon2id calls. Rounds are numbered from 0 to 11, and W_k is the state after k rounds, so $W_0 = X$ and $W_{12} = Y$. Two salts are public even without a leak: S_{11}, which comes from the container's left half, and S_0, when the source is known. The table gives the cost of each explicit test it describes. These tests are filters that reject wrong guesses with overwhelming probability; they are not lower bounds for all attacks.

What leaked	Explicit test of a guess	Argon2id calls per guess
The initial hash H_0 of an Argon2id call, or the first blocks of its work area, when the call's salt is known (S_{11}, or S_0 with a known source)	Compute H_0 for the guess with BLAKE2b, and the first blocks from it if needed, and compare (RFC 9106, section 3.2 [4])	none, only BLAKE2b

What leaked	Explicit test of a guess	Argon2id calls per guess
A salt S_i, $1 \le i \le 10$, or the half it comes from	Invert the container down to R_i and compare the salt	$11 - i$; 1 for S_{10}
The state W_k, $1 \le k \le 10$	Compute between W_k and the container, skipping one round as in the known-pair filter; or, if the recovered source can be checked by its verifier or by wallet data, invert W_k to the source	$11 - k$; with a checkable source $\min(11 - k, k)$: 1 for $k = 1$ or $k = 10$, 5 for $k = 5$ or $k = 6$
The state W_{11}	Invert the container's last round and compare	1
The key K_{11} or the mask M_{11} of the last round	Compute $\mathrm{Argon2id}(P', S_{11})$, derive the mask if needed, and compare	1
The key K_0 or the mask M_0, with a known source	Compute $\mathrm{Argon2id}(P', S_0)$, derive the mask if needed, and compare	1
The key or mask of another round i	Invert the container down to R_i, then one call for round i	$12 - i$
The Argon2id work area of a round, dumped at the end of the call	The dump determines the round's key; test as for that key	as for the key
The source mnemonic	The known-pair filter of Part I	11
The normalized password	None needed	0
Round numbers and an ordinary progress indicator	Nothing about the password: at fixed settings every permutation performs the same twelve rounds; timing observations belong to T12	12, unchanged

The moment of a leak matters as much as its content. Material from the start of an Argon2id call with a known salt reduces a guess to hashing. The state W_{10} or W_{11}, the last round's key, or, when the source can be checked, the state W_1 reduces a guess to one Argon2id call instead of twelve. A state in the middle costs no more than five calls when the source can be checked, and $11 - k$ otherwise.

At a high PIM one call can take hours, so a state written to disk to resume an interrupted operation is exactly such a leak. Resuming safely means recomputing from the start, or keeping the state only in the memory of the running process, and that memory is itself exposed if it is swapped to disk. This is why the specification forbids logging, displaying, exporting or persisting intermediate values, including in error reports.

Birthday bounds and password guessing

For a balanced Feistel network with a $2n$-bit state and n-bit branches, classical small-round multi-query analyses contain birthday-scale terms around:

$$q \sim 2^{n/2}$$

where q counts queries or known/chosen pairs under one fixed permutation.

For suite 4's 128-bit same-length state, $n = 64$, which gives a birthday scale around 2^{32} in those classical games. That number must **not** be reinterpreted as "the password breaks after 2^{32} guesses". In T1, different password guesses select different password-indexed permutations at the selected settings, so the guesses do not accumulate as q queries to one

fixed $\mathrm{Perm}_{P,\mathrm{MEM},\mathrm{PIM}}$. For the 256-bit state of suite 3, $n = 128$ and the corresponding birthday scale is around 2^{64} queries under one fixed permutation.

The multi-query viewpoint becomes relevant only when many samples genuinely belong to the same permutation, parameterized by the password and the settings, and the attack has the information model that the particular proof or distinguisher requires.

Patarin and beyond-birthday security

The birthday scale is not a universal ceiling for balanced Feistel networks. Patarin's positive security results show that balanced Feistel constructions with enough rounds and independent random round functions can achieve security far beyond the basic birthday scale and approach the information-theoretic scale associated with the branch size [28], [29], [51]. Reference [52] instead develops generic attacks on Feistel schemes and supplies adversarial limits.

These results show that a 64-bit branch does not by itself imply a hard 2^{32} security ceiling. The relevant question for MHFE is whether its effective round functions satisfy assumptions strong enough to justify any Patarin-style reduction.

Effective round function

For fixed password P, settings, and round index i, define the effective round function:

$$
\begin{aligned}
D_i(R) &= \mathrm{DS_SALT} \| \,\mathrm{BE32}(\mathrm{MEM}) \| \,\mathrm{BE32}(\mathrm{PIM}) \| \,\mathrm{BE32}(i) \| R,\\
S_i(R) &= \mathrm{Trunc}_{128}(\text{BLAKE2b-256}(D_i(R))),\\
K_i(R) &= \mathrm{Argon2id}(\mathrm{password} = P_{\mathrm{enc}},\ \mathrm{salt} = S_i(R),\\
&\qquad \mathrm{memory} = m(\mathrm{MEM})\ \mathrm{KiB},\ \mathrm{passes} = t(\mathrm{PIM}),\ \mathrm{lanes} = 4,\\
&\qquad \mathrm{version} = \texttt{0x13},\ \mathrm{type} = \mathrm{Argon2id},\\
&\qquad \mathrm{secret} = \mathrm{empty},\ \mathrm{associated\ data} = \mathrm{empty},\ \mathrm{output} = 256\ \mathrm{bits}),\\
D'_i(R) &= \mathrm{DS_MASK} \| \,\mathrm{BE32}(\mathrm{MEM}) \| \,\mathrm{BE32}(\mathrm{PIM}) \| \,\mathrm{BE32}(i) \| R,\\
G_{i,P,\mathrm{MEM},\mathrm{PIM}}(R) &= \mathrm{Trunc}_{128}(\text{HMAC-SHA-256}(K_i(R), D'_i(R))).
\end{aligned}
$$

Encryption and decryption use exactly this 128-bit salt, 256-bit derived key, Argon2id profile, and round-mask function. The expanded definition leaves no Argon2 parameter or optional input implicit. For fixed settings, $m(\mathrm{MEM})$ and $t(\mathrm{PIM}) = 12 \cdot (\mathrm{PIM} + 1)$ are as specified for suite 3. Here $K_i(R)$ is the functional form of the round key; evaluating it at the actual branch R_i gives the normative round key K_i.

Although the internal derived subkey depends on R, $G_{i,P,\mathrm{MEM},\mathrm{PIM}}$ is still one deterministic function from 128-bit inputs to 128-bit outputs. Therefore generic Feistel analysis cannot be dismissed merely because the internal subkey is input-dependent.

At the same time, attacks that specifically rely on reusing one fixed internal subkey over many R values do not automatically transfer unchanged to this construction.

Argon2 salt-separation hypothesis

A potentially favorable heuristic concerns diversification across distinct (i, R) inputs. Different inputs are not guaranteed to produce different 128-bit salts, and distinct salts do not mathematically guarantee distinct 256-bit Argon2id outputs. In an ideal random-function model, two distinct salts produce the same 256-bit output with probability 2^{-256} per pair, so accidental output collision is not the principal concern here. Repeated $(\mathrm{MEM}, \mathrm{PIM}, i, R)$ inputs under the same password and suite necessarily reuse the same salt and key.

If, for fixed unknown P, fixed settings (and therefore fixed memory and passes), and all other suite parameters, the mapping

$$S \longmapsto \mathrm{Argon2id}(P_{\mathrm{enc}}, S;\ \text{fixed suite 3 parameters})$$

behaves with sufficient pseudorandomness and decorrelation over distinct public salts, this might support a random-function model for the effective functions $G_{i,P,\mathrm{MEM},\mathrm{PIM}}$. A useful assumption would need to cover their joint behavior across rounds and inputs, and to account for salt collisions and repeated inputs. It would also need to incorporate both the guessability of the password and behavior across attacker-chosen password candidates. Input-dependent keys alone do not establish an advantage over a conventional keyed round function.

Argon2id was designed and analyzed primarily as a memory-hard password hash/KDF and for resistance to time-memory trade-offs. RFC 9106 discusses Argon2 security as a hash function and KDF. Neither cited source, however, establishes the construction-specific joint-pseudorandomness property that MHFE would require across many salts at one fixed low-entropy human password. A1 is therefore a separate assumption that must be justified or removed by a future proof or construction [4], [25].

Call this open assumption **A1 — Argon2 salt-separation hypothesis**. Theorems 3 and 4 assume A1 in its random-oracle form; they do not prove it.

Argon2id and side channels

Argon2id combines data-independent and data-dependent memory addressing. Slices 0 and 1, the first half of the first pass, follow the Argon2i-style data-independent strategy, while the remaining computation uses Argon2d-style data-dependent addressing. It is therefore inaccurate to characterize Argon2id's memory-addressing pattern as uniformly data-independent or uniformly data-dependent [4].

Implementations should use a well-reviewed Argon2id library and follow its memory-wiping facilities where available. They should also analyze timing/cache exposure on the target platform as a separate task. Native desktop and browser/WASM implementations may have different side-channel constraints even on otherwise supported high-memory systems.

Observers on the same computer

Argon2id computes the first half of its first pass with memory addresses that do not depend on the data, and the rest with addresses that depend on the memory contents [4]. Suppose that an observer sharing the computer, such as another process or a script in a browser that can measure cache timing, recovers the block addresses of the data-dependent phase of one Argon2id call with a public salt. The observer can then reject most password guesses by computing only up to that phase, about half a pass of one call, and comparing the addresses that the guess would produce with the observed ones. Against twelve calls of twelve passes each, that is roughly 1/288 of the computation of a full guess in this idealized setting.

This figure is not a measured speed-up. It depends on how precisely real measurements resolve 1 KiB blocks, and false matches must be removed by further checks. The password's entropy is unchanged; only the cost of testing a guess falls.

The first Argon2id call of a recovery, and of the creation check, uses the public salt S_{11}, so a computation on a shared computer exposes exactly such a call. A separate browser window does not prevent observation through the shared processor caches; recovery belongs on a trusted offline computer with nothing else running. Argon2i on the rounds with a public salt records how a future suite could reduce this shortcut for the public-salt calls, to about 3.5 bits.

Recovery verification is not authentication

A reversible mapping over all 2^{256} entropy values consumes the entire 256-bit output domain. A separate authentication tag cannot be embedded without reserving some outputs, adding external bits, or giving up full-domain bijectivity.

The ordinary 8-bit BIP39 checksum on the encrypted mnemonic detects many accidental transcription errors. A uniformly random 264-bit candidate passes that relation with probability 2^{-8}. The checksum is not a cryptographic authentication tag. An attacker can modify the 256-bit entropy and recompute a valid checksum. Likewise, the checksum recomputed after decryption cannot validate the MHFE password, because every 256-bit candidate entropy has one corresponding valid BIP39 checksum.

Short-source profiles deliberately reserve a strict subset of the 256-bit plaintext domain by requiring $X = E \| \mathrm{Trunc}_r(\text{SHA-256}(E))$. A uniformly random recovered state satisfies that relation with probability 2^{-r}, so the verifier detects most wrong-password candidates and untargeted corruption under the corresponding uniform-candidate model. The verifier nevertheless remains an unkeyed relation inside the encrypted plaintext. It does not provide AEAD authenticity, establish the container's origin, or protect against every maliciously constructed replacement. Nor does it conceal from an offline attacker whether a fully decrypted password candidate passed the same relation.

Determinism and equality leakage

For one fixed password, settings, and plaintext entropy, the ciphertext is deterministic:

$$\mathrm{Perm}_{P,\mathrm{MEM},\mathrm{PIM}}(X) = Y$$

with no nonce. Re-encrypting the same X under the same normalized P and the same settings yields the same Y. Because $\mathrm{Perm}_{P,\mathrm{MEM},\mathrm{PIM}}$ is a bijection, the converse also holds within one fixed suite, password, and settings: equal ciphertexts imply equal packed plaintexts. An observer who knows that two containers use that same permutation can therefore recognize reuse of one packed state, although the state itself remains unknown. This normally indicates reuse of one source. The specification's Security Considerations say what equal containers do and do not show.

Ciphertext equality across different passwords, settings, or suites does not establish plaintext equality, because each of them selects a different permutation. For two independently and uniformly sampled BIP39 sources of the same ENT-bit length, the probability that the two sources are equal is $2^{-\mathrm{ENT}}$. Even the shortest supported 128-bit source therefore has probability 2^{-128} for one pair and a birthday scale near 2^{64} sources. Accidental equality is negligible at realistic scales, but deliberate reuse remains visible under one fixed permutation.

Determinism as a check on implementations

A modified implementation could leak or keep the password or the phrase, write a container that only it can decrypt, or encrypt under a key that its author knows. For a fixed source, normalized password, suite and settings there is exactly one correct container. An implementation that writes anything else is therefore detected by an independent computation of the same container, or by decrypting it with an independent implementation. A correct implementation has no free choice that could carry hidden information. Hidden data would make a container differ from the correct one, so such a check would detect it. Formats with a random salt or nonce can be recomputed with the recorded values too, but there the implementation chooses those values, and they can carry data undetected.

Such a check cannot reveal secrets that malicious software kept or sent elsewhere while producing a correct container. A check run on the same, possibly compromised computer is also not an independent environment. Offline use, reproducible builds, published checksums and, for significant funds, a second implementation from an independent

source limit these risks. Running that second implementation on a separate offline computer gives an independent environment, at the price of exposing the secrets to that computer as well.

The OpenSSL-based script that verified the suite 3 vectors is meant only for public test inputs. It takes the phrase and the password as command-line arguments and is not a tool for real wallets.

Password quality

Memory-hard KDF evaluations are intended to raise offline-guessing cost; they do not create entropy in a weak human password. Effective guessing cost depends on the unavoidable KDF evaluations, password quality, available verification, and any structural shortcut that reduces the required Argon2 work per guess.

Choosing one of at most K complete, independent uniform password draws permits a limited preference for memorable words. For n words from a list of size L, any selected result has probability at most $K \,/\, L^n$, so its min-entropy is at least $\max(0, n \cdot \log_2(L) - \log_2(K))$ bits. Five EFF words selected from at most eight draws therefore retain at least about 61.6 bits of min-entropy. This is a bound, not an exact entropy value or an average guessing-time estimate.

The limit covers the entire selection process, including restarts, and does not permit replacing, reordering or shortening words. A generator cannot certify choices made outside it. Decoys need the same selection distribution. A private story can be made after selection without changing the password.

Attacks in the future

A container kept for decades can be photographed now and attacked later, when each guess costs less. If the cost of a guess halves every H years, a container loses $T \,/\, H$ bits of protection after T years for an attacker with a fixed budget. These rates are assumptions describing a scenario, not predictions or a guaranteed period of protection. Memory bandwidth, which dominates Argon2id, has improved more slowly than arithmetic.

Years	Bits lost, halving every 2 years	Bits lost, halving every 3 years	Dice words to compensate
10	5	3.3	0.3 to 0.4
20	10	6.7	0.5 to 0.8
30	15	10	0.8 to 1.2

One extra dice word, about 12.9 bits, offsets roughly 26 to 39 years of such a decline under these assumptions. The protection of an existing container never grows: higher settings and longer passwords help only containers created with them, and T7 applies to the old ones.

A weakness found in Argon2id or in the construction would call for re-encryption under a new suite, as the specification explains. A second memory-hard function records a hedge that was considered and not adopted.

MHFE uses only symmetric primitives. A generic quantum search in the manner of Grover's algorithm reduces the number of evaluations of an unstructured search from $O(N)$ to $O(\mathrm{sqrt}(N))$ [53]. Each evaluation, however, would have to carry out the chosen MHFE password test reversibly, including its Argon2id work. Theorem 3 and Corollary G10 bound the success of classical attackers in the random-oracle model; a strategy using twelve calls per tested password matches those bounds up to the stated error terms and false-match losses. This document does not analyse quantum query algorithms or estimate their resources. MHFE does not protect the wallet's elliptic-curve signature keys against Shor's algorithm [54].

Sensitive-memory handling

Implementations should minimize copies of plaintext entropy, encoded password, Argon2 outputs, and intermediate Feistel states. They should erase such buffers when the language/runtime provides a reliable mechanism. This is an implementation hygiene requirement, not a substitute for analysis of the cryptographic construction.

Losing access

Losing access is a risk of its own, independent of any attacker. Recovery needs the container, the password, any non-default settings and software that implements the suite, possibly decades later. In the rare case that the packed state also passes the verifier of another length, recovery also needs the word count of the original seed phrase or a receiving address of the wallet.

The BIP39 checksum detects most copying errors but corrects none. The optional MHFE-REPAIR-1 card can repair unreadable or wrong words within the code's bound. The card is not needed for an intact container and does not replace a second complete copy. The rehearsal check of the specification, a password recalled from time to time, a copy of the container in a second place and an offline copy of a compatible release reduce these risks.

A recovery plan. Record what recovery needs and where to find it: which backup is an MHFE container (and, for a suite 2 container, its suite), compatible offline software, the password format, any PIM or memory level other than the default, the word count of the original seed phrase only in the rare case of the specification's creation step 2, and how to retrieve a trusted wallet-identity reference and any separate BIP39 passphrase. A note that shows a suite 3 source has fewer than 24 words removes the decoy option, so keep it apart from the container, like the password. Check scenarios in which a location is destroyed, the main container copy is lost, the password is forgotten, the owner cannot help an heir, or the original computer is unavailable. Instructions, password backups, references and software must be accessible without first opening this wallet or using the secret whose loss they are meant to remedy. Rehearse the plan from the finished backup; instructions do not replace missing secrets.

A damaged backup. If one word of a container cannot be read and its position is known, about $1 + 2047 / 256$, roughly nine, of the 2,048 possible words give a valid BIP39 checksum, counting the right one, and exactly eight do if it is the last word. Each candidate needs a full recovery. In total that takes about 9 to 18 minutes at the default settings on the reference laptop, and about 6 to 13 days at PIM 1023. Two unreadable words give about 16,400 candidates, roughly 11 to 23 days of continuous computation at the default settings.

A short source's verifier will normally reject every wrong candidate, subject to the false-match probabilities given under Source-length identification. For a 24-word source, each recovered phrase has to be compared with a known address or other trusted wallet data. If the phrase was created with `MHFE-WALLET-CHECK-SEED-1`, the 16-bit check of every 24-word reading rejects almost every wrong candidate first. For an ordinary phrase with the same damage, the candidates need no Argon2id work and can be checked against the wallet directly. A second copy of the container is therefore worth more with MHFE than without it. It should be an exact copy of the same verified container, not a second encryption of the phrase, for the reasons given under Deniability.

Heirs. A backup that holds a container looks like an ordinary phrase. An heir who does not know that it is a container would enter it in a wallet, find an empty wallet or a decoy, and might conclude that nothing is there. An heir has to learn separately that the backup holds an MHFE container, which suite and settings it uses, the password, and where to find compatible software. Marking the backup as an MHFE container helps the heir. The marking does not weaken the deniability proofs, which already assume that the adversary knows that MHFE is used. It does, however, reveal that use, can raise an adversary's suspicion and removes the option of presenting the container as an ordinary phrase. How these facts are passed on involves the same trade-off.

An independent emergency route. Two kinds of backup could survive loss of all usable MHFE-password knowledge and all container copies: a separate threshold backup of the original entropy, retaining its source length and wordlist, or an encrypted source backup with an independently random, threshold-shared key. A sufficient coalition obtains the source without the MHFE password or any container copy; individual holders below the threshold do not. Any separate BIP39 passphrase still needs a recovery arrangement. Such a route adds custody and software dependencies, and protection is limited by the weakest complete recovery route.

A forgotten password and a lost main container copy can also be remedied through the ordinary route, by a secure full-password backup and another exact container copy. An independent route is needed when every means of recovering the ordinary route's required material has been lost. The current deniability experiments do not cover these additional recovery artifacts.

Related Work and Alternatives

SLIP-0039

SLIP-0039 is especially relevant prior art [19]. Its master-secret encryption already uses a four-round Feistel network. PBKDF2 acts as the round function, and the current right half R is included in the PBKDF2 salt. In its extendable-backup mode the additional salt prefix is empty; the right half remains part of the PBKDF2 salt.

The broad idea "Feistel + password KDF + current Feistel half in the KDF salt" is therefore **not novel to MHFE**. Any novelty claim must be narrower. If justified, it should focus on the exact combination of a full 256-bit BIP39 entropy permutation, no in-band per-container metadata, Argon2id memory hardness, and the resulting security/compatibility analysis.

SLIP-0039 differs materially in purpose and format. It is a Shamir mnemonic-sharing standard with its own identifier, extendable-backup flag, iteration exponent, wordlist, and share structure. It does not define an in-place 24-word BIP39-to-BIP39 encryption format.

BIP38

BIP38 specifies passphrase-protected private keys and uses scrypt plus AES [32]. It stores format information and a 32-bit address hash inside an expanded encoded record. It is useful prior art for password normalization, KDF parameterization, test vectors, and wrong-password verification, but it does not satisfy the zero-expansion 24-word requirement.

BIP39 optional passphrase

BIP39 itself already gives every mnemonic/passphrase pair a valid derived seed and has no built-in passphrase-error signal. This can support deniability in some contexts, but known wallet information can still verify a guess. The passphrase mechanism affects seed derivation, not encryption of the mnemonic backup [3]. MHFE must not be presented as a replacement for the BIP39 passphrase; the two can coexist.

Honey encryption

Juels and Ristenpart formalize honey encryption for low-min-entropy keys. They use a distribution-transforming encoder so that decryption under wrong keys yields plausible messages [43]. This is the relevant source for the term, but it does not describe MHFE's present security claim. MHFE does not define a general encoder for arbitrary, non-uniform distributions of wallet entropy; known wallet addresses can verify a candidate; and the short-source recovery verifier intentionally rejects almost all wrong-password candidates. Syntactically valid BIP39 output is therefore not sufficient to claim honey-encryption security in general. Part I analyses, together with decoy disclosures, the special case of a uniformly generated 24-word source with a uniform model and the identity as its encoder.

Earlier BIP39 backup-encryption and obfuscation proposals

A 2021 Bitcoin Stack Exchange discussion asked directly how an existing BIP39 mnemonic could be encrypted into another mnemonic without changing the recovered wallet seed. It linked a small AES-CTR prototype [8], [10], [55]. This is direct community history for the problem statement.

The initial prototype derived its AES key as $\mathrm{SHA\text{-}256}(\mathrm{password})$. A later revision changed that step to PBKDF2-HMAC-SHA512 with 2,048 iterations and the fixed salt `mnemonic-encryption`. Both revisions use AES-CTR with an all-zero IV. Reusing one password therefore repeats the CTR keystream, so for two source entropies E_1 and E_2 and their ciphertext entropies C_1 and C_2:

$$C_1 \oplus C_2 = E_1 \oplus E_2$$

The relation alone does not recover either of two independently random and otherwise unknown entropies. If either entropy is known or attacker-controlled, however, it reveals the other one directly. The keystream reuse is separate from the cost of deriving the password key, and it violates CTR's cross-message counter-block uniqueness requirement [11]. The prototype's author also proposed the scheme on the bitcoin-dev mailing list in May 2021 (see the update below). MHFE avoids this particular relation through a password-indexed permutation and state-derived round inputs.

`Seedshift`, `bip39_obfuscator`, and `BIP39Colors` are community projects that shift BIP39 word indices or re-encode them as Traditional Chinese wordlist code points or RGB colors [37], [38], [39]. They illustrate demand for backups that do not visibly expose the English words of the original seed phrase. However, they provide forms of concealment rather than comparable modern encryption.

`bip39_obfuscator` applies a public, deterministic index-for-index mapping with no secret key; anyone who recognizes the representation can reverse it. `BIP39Colors` likewise uses a public deterministic encoding. It packs the positions of 12 or 24 BIP39 words into 8 or 16 hexadecimal RGB colors and includes enough position information to recover the words even when the colors are reordered [39]. It has no secret key and therefore provides visual obfuscation rather than confidentiality against an informed observer. `Seedshift` applies manually computable modular shifts derived from dates. Its own documentation warns that the result is not cryptographically secure and can be brute-forced [37].

None of these constructions uses a memory-hard KDF or provides a security argument for a password-indexed pseudorandom permutation. They should therefore be treated as public re-encodings, obfuscation, or a simple shift cipher, not as substitutes for reviewed mnemonic encryption. This comparison does not itself establish the security of MHFE.

`MnemonicCrypt` is a closer implemented comparison. It removes the source BIP39 checksum, applies configurable Argon2id and AES-CBC with a separate random 128-bit salt, and renders the ciphertext and salt as word sequences [33]. For a 12-word source it produces a 24-word encrypted mnemonic plus a 12-word salt mnemonic, 36 words of recovery material in total. For a 24-word source it produces a 36-word encrypted mnemonic plus a 12-word salt mnemonic, 48 words in total. The KDF parameters must also be preserved separately. Its padded representation is intentionally an extension of BIP39 rather than an ordinary wallet-generated mnemonic. It therefore addresses password hardening and word-oriented storage, but not MHFE's design constraint of a fixed 24-word format with no in-band metadata.

Other mnemonic encryption systems reserve capacity or change the mnemonic domain. `Mnemonikey` encodes a 128-bit OpenPGP seed in a custom 4,096-word list. Its encrypted 16-word form carries a version, creation time, random salt, encrypted seed, checksum, and a 5-bit password verifier, and it derives its AES-128 key with Argon2id [34]. `pktseed` defines a custom 15-word PKT seed containing version, encryption, checksum, birthday, and seed fields [35]. Its current encryption code derives a 19-byte mask from Argon2id with a fixed salt and XORs that mask with the birthday-

and-seed payload. Reusing the same passphrase therefore creates a cross-record XOR relation analogous to the 2021 prototype [10]. These are useful comparisons for compact mnemonic metadata and recovery verification, but neither transforms an existing BIP39 mnemonic into another BIP39 mnemonic.

`seed-otp` takes a different approach: it adds a separately stored per-word pad modulo 2,048 [36]. This can preserve the source word count and BIP39 wordlist membership. It can also provide one-time-pad security when the pad is uniformly random, secret, and never reused. Its ciphertext normally fails the BIP39 checksum, and the scheme moves the backup burden to another secret of comparable size rather than deriving protection from a memorable password.

A non-exhaustive search of public GitHub repositories and the cited community discussions, repeated on 2026-09-22 and extended on 2026-10-03, found many encrypted wallet files, mnemonic obfuscators, secret-sharing formats, and custom word encodings. It found no implementation combining all of the following properties:

- an existing 12-, 15-, 18-, 21-, or 24-word BIP39 source;
- one ordinary checksum-valid 24-word BIP39 ciphertext container;
- exact recovery of the original entropy under a password;
- for every shorter source, use of all otherwise unused state capacity as one hash-based recovery verifier $V_r = \mathrm{Trunc}_r(\text{SHA-256}(E))$, whose first $\mathrm{ENT} \,/\, 32$ bits are exactly the source mnemonic's BIP39 checksum and whose 128-, 96-, 64-, or 32-bit width gives uniform-candidate false-acceptance probability 2^{-128}, 2^{-96}, 2^{-64}, or 2^{-32}, respectively, without expanding the container;
- no mandatory separately stored salt, nonce, authentication tag, or expansion words; the default settings are implicit, while deliberately selected non-default settings must be remembered; and
- a memory-hard, state-derived KDF schedule inside a format-preserving permutation.

This search result narrows the known comparison set. It is not an exhaustive prior-art search, a novelty claim, or evidence that MHFE is secure.

Update, 2026-10-03: further prior-art search. A further search of the bitcoin-dev archive, public GitHub repositories, Monero and Lightning wallet formats, the IACR ePrint archive, arXiv and Google Patents found the related works below. They show that two ideas predate MHFE: encrypting a mnemonic with a password into a mnemonic of the same length, and opening a valid decoy wallet under a wrong password.

- **The 2021 prototype on bitcoin-dev.** Tobias Kaupat, the author of the prototype above, also proposed it on the bitcoin-dev mailing list in May 2021 under the subject "Encryption of an existing BIP39 mnemonic without changing the seed" [9]. Replies recommended stretching the password with PBKDF2, scrypt or Argon2. The construction is the one analysed above: a password-only key, AES-CTR with a zero IV and a repeated keystream.
- **seed-encrypt** [42], since September 2024, encrypts the entropy of a 24-word phrase into another 24-word phrase. Its key sequence uses Argon2id with 2 GiB; each result feeds the next call, and the pass count increases. Creation uses the latest key after reaching a user-selected time limit; the author recommends an hour or more. Balloon hashing appears in comments but is not called by the cited implementation. Unlike MHFE, it uses one built-in salt for every user, so the key sequence for one password and thread count applies to every container. It encrypts the two 128-bit halves independently with AES-256 and accepts only 24-word sources. Recovery enumerates candidate phrases along the key sequence: the time limit is a stopping rule, not a fixed number of KDF calls that is portable across hardware. Three optional extra words encode a revision prefix, thread count and time-limit setting.
- **Monero's seed offset passphrase** [40] has been in the Monero wallet since at least 2019. It adds `cn_slow_hash(passphrase)` to the spend key and shows the result as another 25-word seed. A wrong passphrase gives another valid wallet, and a guest tutorial in Monero's documentation suggests decoy wallets [56]. It is not a BIP39 format. It has no salt, so one hash per passphrase applies to every seed, and it has no recovery check.

- **Polyseed** [41], a 16-word Monero seed format since 2021, can encrypt its secret with a password. It XORs a mask from PBKDF2-HMAC-SHA256 with 10,000 iterations and a fixed salt, and it sets a flag bit. It is a format of its own, its KDF is not memory-hard, and one password gives the same mask for every seed.
- **aezeed** [57], in LND since 2018, enciphers a new wallet seed with the AEZ wide-block cipher and scrypt into 24 words of the BIP39 list. It stores a version, a 5-byte salt and a checksum among those words. It defines a new seed rather than encrypting an existing BIP39 mnemonic, and its words are not a BIP39 phrase.
- **Seed XOR** [58], 2021, and **BIP39-XOR** [59], 2023, split a phrase into several valid phrases of the same length whose XOR is the original seed phrase. Each part is a working wallet and can serve as a decoy. The other parts, however, are secrets as large as the phrase itself, and there is no password.
- **PhraseCrypt** [60], 2026, includes a honey-encryption mode that XORs BIP39 entropy with a mask derived by PBKDF2-HMAC-SHA256 with 200,000 iterations and a random 16-byte salt. A wrong password yields a valid phrase of the same length instead of a password error. Its stored form is an expanded Base64 container containing a version byte, salt and ciphertext, not a BIP39 phrase. A dedicated duress password is listed as an idea for future work, not an implemented feature.

This non-exhaustive search identified no paper or patent in the IACR ePrint archive, arXiv or Google Patents that describes the encryption of an existing BIP39 mnemonic into another mnemonic. The closest patents identified concern turning an encrypted secret into words, or splitting and storing seed phrases. This does not establish that no closer prior art exists.

These findings narrow MHFE's proposed contribution to a combination of properties rather than any single element. Among the BIP39-to-BIP39 password-encryption tools examined here, none derives its salts from the state and so avoids both a separately stored salt and one salt shared by all users. None fills the free bits of a shorter source with a verifier that begins with its own BIP39 checksum, inside a universal 24-word container that hides the source length. None runs a memory-hard KDF in every round of a format-preserving permutation. None states a model with proofs for its plausible deniability.

This does not make state-derived Feistel salts or a KDF in each round new: SLIP-0039 already uses both [19], as discussed above. The proposed contribution is the combination in suite 3, together with the specification, independently reproduced test vectors and reference implementation. The search remains non-exhaustive and does not establish novelty or security.

Luby-Rackoff and Patarin

Luby-Rackoff supplies the foundational theory for constructing pseudorandom permutations from round functions. Patarin's later work is relevant to generic attacks and beyond-birthday security for multi-round balanced and unbalanced Feistel schemes. These papers define the PRP/SPRP models and Feistel bounds against which MHFE should be evaluated. Their proofs assume independent idealized round functions. MHFE instead derives every effective round function from one password and state-dependent Argon2id inputs. The bounds of these papers therefore cannot be claimed for MHFE without a separate construction-specific reduction [28], [29], [30], [51], [52].

Format-preserving encryption

General FPE constructions demonstrate how to build permutations on constrained domains [61]. By themselves, however, they do not provide memory-hard password guessing or solve the no-metadata salt problem. Morris, Oberschelp, and Santhakumar construct a no-expansion pseudorandom permutation in the bounded retrieval model using a large key, random-oracle assumptions, and the Thorp shuffle [62]. Its hybrid analysis and explicit treatment of uniform distinct messages are relevant methodology, but its leakage model, key structure, round function, and security game differ materially from MHFE.

Thorp and maximally unbalanced Feistel

Thorp-style constructions are important prior art for very small domains. They show that the birthday behavior of a small balanced branch is not a universal limitation of all Feistel architectures. Published Thorp bounds [63] use many cheap micro-rounds. Substituting a full Argon2id invocation for every micro-round can require hundreds or more expensive calls at these state sizes, depending on the selected bound and target security. This suggests a substantial latency problem. Neither a practical latency figure nor a universal minimum round count follows, however, without choosing and analyzing a specific construction. Whether a memory-hard key can safely control a whole pass of cheap unbalanced rounds without reintroducing a state/salt circular dependency remains an open research question.

Theoretical Investigation and Alternative Designs

The sections below explain the packing and the geometry that suite 3 adopted, and they record the alternatives that were studied but not adopted. Suite 3 uses the balanced 128|128-bit Feistel network and the universal packing described below. Suite 4 defines the balanced same-length shorter-state construction; its separate analysis appears above. Other shorter-state variants, the final-word profile and the source-heavy 1:3 family remain research alternatives. They are not production recommendations or finalized encodings. Unless explicitly labeled otherwise, the construction formulas and tables below give sizes in bits.

Shorter BIP39 mnemonics

The BIP39 entropy sizes are:

Words	ENT	BIP39 checksum	Full mnemonic bits	Spare bits after storing ENT only
12	128	4	132	128
15	160	5	165	96
18	192	6	198	64
21	224	7	231	32
24	256	8	264	0

The last column assumes that the original checksum is discarded and later recomputed. It does not describe the capacity left after storing the complete original word bitstring.

A balanced Feistel transform that operated directly on each ENT-bit source entropy would use branches of 64, 80, 96, 112, and 128 bits respectively. The 64-bit branch of a 12-word mode does not automatically imply a 2^{32} ceiling on password security. Classical birthday bounds and password guessing are different attack models, and Patarin-style results show that multi-round Feistel can exceed the basic birthday regime in ideal models. Nevertheless, every shorter state size requires separate analysis. Suite 4 defines this mode, and its section examines what the narrower state changes.

Universal 24-word containers for shorter sources

Suite 3 packs every standard BIP39 source length with one formula:

$$
\begin{aligned}
\mathrm{ENT} &= \text{bit length of source entropy } E,\\
r &= 256 - \mathrm{ENT},\\
V_r &= \mathrm{Trunc}_r(\text{SHA-256}(E)),\\
X &= E \| V_r.
\end{aligned}
$$

When $\mathrm{ENT} = 256$, $r = 0$, V_r is the empty bitstring, and $X = E$. For every shorter source, the packing keeps the source entropy exactly and fills every remaining position of the 256-bit state with deterministic recovery-verifier bits. It adds no independent checksum field, second custom check, tag, padding rule, or random filler.

Source words	ENT	r	Packed state X	Uniform-candidate verifier acceptance
12	128	128	$E_{128} \| \mathrm{Trunc}_{128}(\text{SHA-256}(E_{128}))$	2^{-128}
15	160	96	$E_{160} \| \mathrm{Trunc}_{96}(\text{SHA-256}(E_{160}))$	2^{-96}
18	192	64	$E_{192} \| \mathrm{Trunc}_{64}(\text{SHA-256}(E_{192}))$	2^{-64}
21	224	32	$E_{224} \| \mathrm{Trunc}_{32}(\text{SHA-256}(E_{224}))$	2^{-32}
24	256	0	E_{256}	No internal verifier

This construction exploits a direct relationship with BIP39. For a source entropy of length ENT, the ordinary BIP39 checksum is:

$$
\mathrm{CS} = \mathrm{Trunc}_{\mathrm{ENT}/32}(\text{SHA-256}(E))
$$

Because V_r is a longer prefix of that same digest for every short source, its first $\mathrm{ENT} \,/\, 32$ bits are exactly the original BIP39 checksum. The remaining bits extend the same check:

Source words	Original BIP39 checksum inside V_r	Additional verifier bits	Total V_r
12	4	124	128
15	5	91	96
18	6	58	64
21	7	25	32
24	Not present in V_r; recomputed on output	0	0

The saving is conceptual and structural: the source checksum is not stored twice, and all remaining capacity becomes one continuous verifier. For example, the 21-word profile does not store a 7-bit checksum plus a separate 25- or 32-bit field. It stores one 32-bit SHA-256 prefix. The first 7 bits of that prefix already are the source checksum, and its remaining 25 bits strengthen recovery screening.

The full data path is:

$$\text{Source entropy } E \| \operatorname{Trunc}_{256-\mathrm{ENT}}(\mathrm{SHA256}(E))$$
$$\downarrow$$
$$\text{256-bit packed plaintext } X$$
$$\text{MHFE } \mathrm{Perm}_{P,\mathrm{MEM},\mathrm{PIM}}$$
$$\downarrow$$
$$\text{256-bit encrypted entropy } Y$$
$$\text{BIP39 checksum } \mathrm{CS8}(Y)$$
$$\downarrow$$
$$\text{24-word encrypted container}$$

The inner V_r and the outer $\mathrm{CS8}(Y)$ serve different purposes. V_r is encrypted with the source and checks a candidate recovery after MHFE decryption. $\mathrm{CS8}(Y)$ is the ordinary visible BIP39 checksum of the encrypted container and detects many recording or transcription errors before decryption. The outer checksum contributes no additional wrong-password rejection, because it is computed from Y and anyone can recompute it.

The measurable reasons for this short-source packing are recovery verification, probabilistic source-length detection, complete use of otherwise unoccupied payload capacity, and the hypothesized near-full-width marginal distribution of the first-round KDF context. Its explicit costs are known redundancy, an offline password oracle after candidate decryption, and a structured and correlated initial Feistel state. The last cost has to be handled by analysis: adding another public hashing or mixing layer would not remove the deterministic relation.

Recovery-verifier properties

For each short source length, recovery parses X into $E \| V_r$ and accepts that reading only if:

$$V_r = \operatorname{Trunc}_r(\text{SHA-256}(E))$$

Recovery compares all r bits. For a uniformly distributed candidate X, the set of states that satisfy this relation has exactly 2^{ENT} members among 2^{256}, because each possible E determines one and only one V_r. The acceptance fraction is therefore exactly 2^{-r}. This counting statement does not require treating SHA-256 as a random oracle. Applying that fraction to actual wrong-password decryptions does require an appropriate assumption about the candidate distribution of the MHFE permutation.

V_r is an unkeyed **recovery verifier**, not AEAD. It does not authenticate the container or its origin. Anyone can alter Y and recompute the visible outer BIP39 checksum. Without the password, however, that party cannot in general choose the resulting decrypted X or recompute a valid verifier inside the encrypted plaintext. Under the uniform-candidate model, an altered candidate passes with probability 2^{-r}. A party that knows the password can construct a different valid container.

Both the owner and an offline attacker testing password candidates can evaluate V_r after candidate decryption. The intended cost control is that obtaining the candidate requires the MHFE inverse and its memory-hard Argon2id evaluations. In the random-oracle model, Theorem 3 shows that for one container no shortcut using fewer KDF evaluations gains more than its stated error terms. Whether the real primitives, several containers or other models permit such a shortcut remains open.

The verifier does not add source entropy. A 12-word source still has 128 bits of source entropy, not 256, even though its packed state is 256 bits wide. The remaining 128 bits of that state are completely determined by E.

Fixed 128|128 state structure

The packing gives every source length the same 256-bit balanced Feistel geometry:

$$L_0 = E[0:128]$$
$$R_0 = E[128:\mathrm{ENT}] \| V_r$$

Source words	L_0	Raw source bits in R_0	Hash-derived bits in R_0	Conditional source entropy in R_0 given L_0
12	128	0	128	0
15	128	32	96	32
18	128	64	64	64
21	128	96	32	96
24	128	128	0	128

For every short profile, R_0 includes hash-derived information computed from all of E, and it also feeds the first state-derived salt. The packing makes one fixed implementation possible and avoids separate 64-, 80-, 96-, and 112-bit balanced cores. It does not create independent entropy or by itself prove stronger Feistel security. Dependencies among round inputs, repeated salts, related structured states, and shortened password-testing paths still require analysis.

For a uniformly random source, the final table column is exact rather than approximate. Let $\mathrm{Ent}(A|B)$ denote the Shannon entropy of A given B, in bits. Write $E = L_0 \| T_t$, where $t = \mathrm{ENT} - 128$. The packing copies T_t verbatim into R_0 and appends a deterministic hash prefix. For each fixed L_0, different T_t values therefore produce different R_0 values, so:

$$\mathrm{Ent}(R_0|L_0) = \mathrm{Ent}(T_t|L_0) = t$$

Under the stated uniform-source assumption, the resulting conditional source entropy is exactly 0, 32, 64, 96, or 128 bits for 12-, 15-, 18-, 21-, or 24-word sources respectively. This conditional property is distinct from the marginal distribution of R_0. Observed alone, a branch can look close to a full-width 128-bit value while still being correlated with L_0, or in the 12-word case completely determined by it. Well-distributed does not mean independent.

Early whole-source sensitivity and the structured plaintext domain

The verifier provides one concrete early whole-source-sensitivity property. A conventional balanced Feistel round function sees only R_0, and MHFE likewise forms its first state-derived KDF input from R_0. In each short-source profile here, however, the hash-derived suffix of R_0 is computed from all source entropy E. Under the hash-prefix heuristic stated below, the first state-derived salt is therefore sensitive to changes in either original half.

When a change is confined to L_0, the raw entropy tail in R_0 stays fixed, and the sensitivity comes from V_r. Under the heuristic that the relevant SHA-256 prefix behaves like a uniform r-bit value, the probability that this verifier suffix stays unchanged is 2^{-r}: 2^{-128}, 2^{-96}, 2^{-64}, or 2^{-32} for 12-, 15-, 18-, or 21-word sources respectively. If R_0 changes, the subsequent 128-bit salt hash is also expected to change, except for its own collision probability.

This property comes with a structured plaintext domain. For each short source length, the valid packed states form the set:

$$\mathrm{Packed}_{\mathrm{ENT}} = \{E \| \mathrm{Trunc}_{256-\mathrm{ENT}}(\text{SHA-256}(E)) : E \in \{0,1\}^{\mathrm{ENT}}\}$$

$\mathrm{Packed}_{\mathrm{ENT}}$ contains exactly 2^{ENT} states and occupies the fraction $2^{\mathrm{ENT}-256}$ of the complete 256-bit domain. It is a structured subset, specifically the graph of a deterministic truncated-hash function. It is not generally a linear subspace.

That restricted size is not a special price paid for early whole-source sensitivity. It is mathematically unavoidable for any deterministic lossless encoding of an ENT-bit source into a 256-bit container: there are only 2^{ENT} distinct sources to place in 2^{256} possible states. The hash-based construction chooses how those states are distributed and also supplies recovery verification and early whole-source sensitivity.

A permutation satisfying full-domain PRP security remains indistinguishable when an adversary restricts its queries to a structured subset. The existence of $\mathrm{Packed}_{\mathrm{ENT}}$ is therefore not by itself evidence of a weakness. Because MHFE's KDF schedule depends on the evolving state, the public relation defining $\mathrm{Packed}_{\mathrm{ENT}}$ needs its own analysis. Theorems 3 and 4 bound guessing success in the random-oracle model for one container and one known pair, including partial tests, with the stated error terms. Related inputs remain open.

Collision-diversity hypothesis for the first state-derived salt

Across distinct, independently sampled source entropies, the mixed raw-and-hash construction may give R_0 close to 128 bits of collision diversity, even though the hash suffix adds no independent entropy. This statement excludes reuse of the exact same source, because the construction is deterministic and necessarily reproduces the same R_0.

For the 21-word profile:

$$R_0 = \underbrace{E[128:224]}_{96\text{ bits}} \,\|\, \underbrace{\mathrm{Trunc}_{32}(\text{SHA-256}(E))}_{32\text{ bits}}$$

To produce the same R_0, two distinct independent sources must first have the same 96-bit entropy tail and then the same 32-bit hash prefix. Under the heuristic that the SHA-256 prefix behaves independently for distinct full inputs sharing that tail:

$$\Pr[R_0^{(1)} = R_0^{(2)}] \sim 2^{-96} \cdot 2^{-32} = 2^{-128}$$

The same heuristic pattern holds for every supported source length:

Source words	R_0 construction	Heuristic distinct-source pair-collision probability
12	$\mathrm{Trunc}_{128}(\text{SHA-256}(E_{128}))$	approximately 2^{-128}
15	$E_{\mathrm{tail},32} \| \mathrm{Trunc}_{96}(\text{SHA-256}(E_{160}))$	approximately $2^{-32} \cdot 2^{-96} = 2^{-128}$
18	$E_{\mathrm{tail},64} \| \mathrm{Trunc}_{64}(\text{SHA-256}(E_{192}))$	approximately $2^{-64} \cdot 2^{-64} = 2^{-128}$
21	$E_{\mathrm{tail},96} \| \mathrm{Trunc}_{32}(\text{SHA-256}(E_{224}))$	approximately $2^{-96} \cdot 2^{-32} = 2^{-128}$
24	$E_{\mathrm{tail},128}$	2^{-128} for independent uniform sources

The 2^{-128} entries describe the probability that one distinct, independently sampled pair has the same R_0. They do not mean that collisions require 2^{128} samples. The corresponding birthday scale is approximately 2^{64} independent sources.

The first state-derived salt is:

$$S_0 = \mathrm{Trunc}_{128}(\text{BLAKE2b-256}(\text{DS_SALT} \| \text{BE32}(\text{MEM}) \| \text{BE32}(\text{PIM}) \| \text{BE32}(0) \| R_0))$$

The salt's dependence on R_0 may provide close to full-width diversification across containers. For two distinct containers using the same suite and settings, an idealized random-function calculation gives two ways for their S_0 values to coincide: equal R_0 values, or a hash collision between unequal values. Using the preceding 2^{-128} heuristic for equal R_0, the combined probability is $2^{-128} + (1 - 2^{-128}) \cdot 2^{-128} = 2^{-127} - 2^{-256}$, which is close to 2^{-127} rather than exactly 2^{-128}. Conditional on L_0, the source entropy remaining in R_0 is still only 0, 32, 64, 96 or 128 bits, as shown above. The salt diversification may reduce accidental salt reuse, but it adds no entropy.

Source-length identification

A decoder infers a short source length without stored metadata. Every recovery checks all four short relations, even when the user states a length, and the detected length takes precedence over a stated one. After decrypting the container to one 256-bit candidate $X_{\text{candidate}}$, the decoder tests:

Candidate words	E	V	Accept if
12	$X_{\text{candidate}}[0:128]$	$X_{\text{candidate}}[128:256]$	$V = \text{Trunc}_{128}(\text{SHA256}(E))$
15	$X_{\text{candidate}}[0:160]$	$X_{\text{candidate}}[160:256]$	$V = \text{Trunc}_{96}(\text{SHA256}(E))$
18	$X_{\text{candidate}}[0:192]$	$X_{\text{candidate}}[192:256]$	$V = \text{Trunc}_{64}(\text{SHA256}(E))$
21	$X_{\text{candidate}}[0:224]$	$X_{\text{candidate}}[224:256]$	$V = \text{Trunc}_{32}(\text{SHA256}(E))$

A single passing relation gives a strong probabilistic indication of that source length. The recovery rules are defined only in Recovering a mnemonic: which readings are given, in which order and with which labels, how a stated length is used, and the 16-bit wallet check of every 24-word reading. This section gives the probabilities behind those rules.

Perfect self-description is impossible while the 24-word source mode covers all 2^{256} plaintext states. Every packed short state is also a possible 256-bit entropy value for some 24-word source. Therefore failure of all short checks means only "no short profile was verified"; the 24-word reading of the state remains unverified and is not proof of a correct password. For a uniformly distributed 256-bit candidate, including an independently generated 24-word source under the model above, the accidental match rates of the individual verifier relations are:

Short relation tested	Probability	Approximate frequency
21 words	2^{-32}	1 in 4,294,967,296
18 words	2^{-64}	1 in 18,446,744,073,709,551,616
15 words	2^{-96}	1 in 79,228,162,514,264,337,593,543,950,336
12 words	2^{-128}	1 in 340,282,366,920,938,463,463,374,607,431,768,211,456

The probability that at least one short relation passes is bounded by:

$$2^{-32} \leq \Pr[\text{at least one short relation passes}] \leq 2^{-32} + 2^{-64} + 2^{-96} + 2^{-128}$$

The 2^{-32} term from the 21-word profile dominates, so the probability of at least one accidental short-relation match is approximately 2^{-32}, or about one in 4.29 billion. This is strong probabilistic screening for accidental uniform candidates, not exact type information. A deliberately constructed 24-word entropy can equal a valid packed short state with certainty.

Under the uniform-candidate heuristic, a wrong-password candidate passes no short relation unless it produces one or more accidental short matches. Because every recovery checks all four relations, this bound of about 2^{-32} applies even when the user states a length. A stated length still helps. A wrong password matches a stated 12-word length only with

probability 2^{-128}, and a match at another length is reported as differing from the stated one.

Final-word-preserving cycle walking for 24-word sources

An unreleased suite 2 draft defined and implemented the optional profile `MHFE-BIP39-256-EXPERIMENTAL-2-CYCLE-WALK-FINAL-WORD`. The profile preserves the complete final word of a 24-word source and leaves the permutation unchanged. It is not part of suite 3. This subsection describes the same construction over the suite 3 permutation, the research idea of Part I.

Such a profile would be separate from standard encryption, and it would have to be selected explicitly during both creation and recovery. The 24-word container does not encode which profile was used, so the owner would have to remember that it was used: standard recovery would silently produce a different 24-word wallet. Its purpose would be to let an owner who remembers the last word of each wallet tell which container phrase belongs to which wallet without decrypting anything, and to catch a container phrase whose last word was copied wrongly. That catch is an 11-bit check of the container phrase, not of the password or of the other 23 words.

Let $\mathrm{FW}(E)$ be the 11-bit index of the final BIP39 word obtained from 256-bit entropy E. This index consists of the final three entropy bits followed by the eight-bit BIP39 checksum. Let $\mathrm{Perm}_{P,\mathrm{MEM},\mathrm{PIM}}$ be the suite 3 permutation. Creation applies the permutation at least once and continues until the complete final-word index matches:

$$\begin{aligned} \mathrm{target} &= \mathrm{FW}(X), \\ Y &= \mathrm{Perm}_{P,\mathrm{MEM},\mathrm{PIM}}(X), \\ \text{while } \mathrm{FW}(Y) &\neq \mathrm{target}: \\ & \quad Y \leftarrow \mathrm{Perm}_{P,\mathrm{MEM},\mathrm{PIM}}(Y). \end{aligned}$$

Recovery likewise applies the inverse at least once and continues until the candidate matches the container's final-word index:

$$\begin{aligned} \mathrm{target} &= \mathrm{FW}(Y), \\ X_{\mathrm{candidate}} &= \mathrm{Perm}^{-1}_{P,\mathrm{MEM},\mathrm{PIM}}(Y), \\ \text{while } \mathrm{FW}(X_{\mathrm{candidate}}) &\neq \mathrm{target}: \\ & \quad X_{\mathrm{candidate}} \leftarrow \mathrm{Perm}^{-1}_{P,\mathrm{MEM},\mathrm{PIM}}(X_{\mathrm{candidate}}). \end{aligned}$$

The first application is mandatory because the starting state already belongs to the selected class. The result is the next distinct member of that class on the same permutation cycle, so no iteration counter is stored. If the walk returns to its starting state before it finds a distinct member, creation or recovery must fail. Implementations should expose progress after each complete permutation and should permit cancellation before the next permutation begins.

The encrypted container reveals the source's final BIP39 word. That word exposes three bits of the source entropy directly and, through the eight-bit checksum, about 11 bits of information about the source in total. It therefore confines the source to a class of about 2^{245} values. This preserved word is not an authentication tag or password verifier: a successful recovery under a wrong password returns a different 24-word candidate with that same final word. The correct password and settings still recover the exact source.

Under an ideal-permutation heuristic, a final-word class has density 2^{-11}, giving a match probability of approximately 1 in 2,048 per complete permutation. The iteration count is therefore modeled geometrically, but this is an estimate rather than a strict latency bound. The table below uses the measured command-line recovery time of about 70 seconds per permutation under Performance measurements only as an illustration:

Statistic	Complete permutations	Approximate sequential time
Median	1,420	27.6 hours
Expected value	2,048	39.8 hours
95th percentile	6,134	119.3 hours
99th percentile	9,430	183.4 hours

There is no small deterministic bound. The distance to the next member of a class can be vastly larger than 2,048. The runtime depends on the input and the password, and this creates availability and timing side-channel concerns. Cancellation and progress reporting keep an application responsive but do not reduce the cryptographic work. Under this model each password guess has a similarly large expected cost, although individual walk lengths vary, so trying many variants of a half-remembered password is generally impractical.

Why the proposed cheaper variants do not solve the problem. Three cheaper variants come up naturally. The first is a walk with a fast permutation and only one expensive Argon2id call to derive its key. The difficulty is the salt of that call. The salt must be computable from the original seed phrase during creation and from the container during recovery, and the only thing the two share is the last word itself: 11 bits. There would therefore be only 2,048 possible salts in the world. An attacker could compute Argon2id for a password dictionary once per salt and then test every such container almost for free. That is exactly the shared dictionary that state-derived salts exist to prevent.

The second variant, doing the expensive part first and a cheap walk afterwards, fails differently: during recovery the backward walk has no way to recognise where the expensive part ended.

The third variant is to find a matching container cheaply with light parameters and then look for a way to reach it with the real ones. This does not help either. For a given password and settings, the expensive permutation sends the source to exactly one container, which is fixed but unpredictable until it is computed. Any search for a password, setting or tweak that makes that container land on the right last word succeeds with probability about 1 in 2,048 per attempt, so it costs as many expensive evaluations as cycle walking itself. The variant on 253 bits below would need about 256 expensive permutations instead of 2,048, but it requires a new permutation and its own analysis.

Preserving the complete final word by excluding three bits

The final word of a 24-word BIP39 mnemonic contains the last three source-entropy bits followed by the eight-bit BIP39 checksum. A hypothetical new profile could keep the three entropy bits unchanged and apply a newly defined permutation only to the other 253 bits. Cycle walking would then need to match only the eight-bit checksum. Under the same ideal-permutation heuristic, each iteration would succeed with probability approximately $1 / 256$. The expected work would therefore fall from 2,048 to 256 applications of the new permutation, still with the complete final word preserved. Using the measured command-line time of about 70 seconds per permutation under Performance measurements only as an illustration gives:

$$256 \cdot 70\ \mathrm{s} = 17{,}920\ \mathrm{s} \approx 4\ \mathrm{h}\ 59\ \mathrm{min}$$

This is not a shortcut that can be applied after the current 256-bit permutation. The BIP39 checksum depends on all 256 entropy bits, including the final three, so changing those bits after a checksum match generally invalidates the checksum. The existing 256-bit permutation could be restricted to one fixed three-bit suffix by an inner cycle walk, but that walk would itself cost approximately eight permutation applications and would restore the overall $8 \cdot 256 = 2,048$ expected-work factor. A direct construction would instead require a new, separately analyzed 253-bit permutation, new domain separation, new test vectors, and an explicit profile identifier.

The faster profile would deliberately disclose three bits of the source entropy. It would partition the domain into preserved suffix classes and replace the frozen balanced 256-bit construction with a different geometry. No security reduction for that construction is provided here. The author does not consider preserving a recognizable final word sufficient justification for sacrificing the full-state confidentiality objective or changing the cryptographic construction in this way. This direction is recorded for completeness and is not recommended for inclusion in any suite.

Known-pair composition across cycle-walking iterations

Cycle walking changes the known-pair problem from one exposed application of $\mathrm{Perm}_{P,\mathrm{MEM},\mathrm{PIM}}$ into a stopped iteration of the same permutation, parameterized by the password and the settings. Write:

$$\begin{aligned} W_0 &= X, \\ W_j &= \mathrm{Perm}_{P,\mathrm{MEM},\mathrm{PIM}}(W_{j-1}), \\ \tau &= \min\{j \geq 1 : \mathrm{FW}(W_j) = \mathrm{FW}(W_0)\}, \\ Y &= W_\tau. \end{aligned}$$

Under threat model T3, an attacker may know the cycle-walking endpoints (X, Y). Except when $\tau = 1$, those endpoints are not an adjacent pair (W_{j-1}, W_j) for one application of $\mathrm{Perm}_{P,\mathrm{MEM},\mathrm{PIM}}$. The intermediate states and τ remain hidden unless timing or instrumentation exposes them. The single-permutation Feistel shortcut described above therefore cannot simply be applied independently τ times: each application would require an adjacent internal pair that the attacker does not initially possess.

The opposite assumption is also unjustified. All iterations reuse the same $\mathrm{Perm}_{P,\mathrm{MEM},\mathrm{PIM}}$, password, settings, and suite; they are not independently keyed layers. The stopping rule also reveals a structured transcript condition:

$$\begin{aligned} &\mathrm{FW}(W_j) \neq \mathrm{FW}(X) \quad \text{for } 1 \leq j < \tau, \\ &\mathrm{FW}(W_\tau) = \mathrm{FW}(X). \end{aligned}$$

A construction-specific analysis must determine whether a meet-in-the-middle computation, a Feistel invariant spanning several applications, repeated state-derived salts, or the final-word non-membership conditions can test a password while omitting expensive rounds in more than one application. Any such saving must be analyzed jointly with the number of cycle-walking iterations. Neither multiplying the one-permutation shortcut by 2,048 nor charging 2,048 independent full attacks is a justified cost model.

Timing can expose an additional filter even before such a shortcut is found. In the idealized geometric model with final-word-match probability $p_{\mathrm{match}} = 1 \,/\, 2048$, let τ be the observed correct stopping time and τ' the stopping time under an independent wrong-password trajectory. If an exact iteration count can be associated with Y, this timing-only filter does not require knowledge of X. An attacker can inverse-walk from Y under each password guess and compare its first-return count with the observed value. For one concrete observation $\tau = t$, with Ex the expected value:

$$\begin{aligned} \Pr[\tau' = t] &= p_{\mathrm{match}}(1 - p_{\mathrm{match}})^{t-1}, \\ \mathrm{Ex}[\min(t, \tau')] &= \frac{1 - (1 - p_{\mathrm{match}})^t}{p_{\mathrm{match}}}. \end{aligned}$$

When both stopping times are independently sampled from that geometric model and the result is averaged over the correct τ:

$$\Pr[\tau' = \tau] = \frac{p_{\text{match}}}{2 - p_{\text{match}}} = \frac{1}{4095},$$
$$\operatorname{Ex}[\min(\tau, \tau')] = \frac{1}{1 - (1 - p_{\text{match}})^2} \approx 1024.25.$$

Thus an exact stopping-time observation would reject about 4,094 of 4,095 idealized wrong-password trajectories by the stopping-time condition alone. Reaching that decision would take approximately 1,024.25 complete permutations on average. This does **not** mean that password entropy is reduced by a fixed number of bits or that these idealized values carry over unchanged to MHFE. It shows that variable runtime is part of the password-guessing model, not only a user-interface problem. The timing-only, endpoint-only, endpoint-plus-stopping-time, leaked-intermediate-state, and multiple known-pair cases require separate lower bounds on the unavoidable number of Argon2id evaluations.

The resulting outer BIP39 mnemonic has the same complete final word as the source mnemonic. Its 11-bit index is exposed as a class label. It is not an independent verifier: every password defines its own inverse permutation, and a successful inverse cycle walk under a wrong password returns a candidate in the requested final-word class. The method therefore does **not** verify the password, authenticate the recovered phrase, or contradict the information-capacity argument.

The method changes the mathematical cycle-walk core into one permutation that preserves each of 2,048 final-word classes, equivalently 2,048 restricted class permutations. No independence between those restrictions is claimed. Because the operational rule above rejects fixed points of this induced permutation, that wrapper is a partial mapping on each full class, invertible on its accepted states. Separate security and worst-case-runtime analysis is required.

The unreleased suite 2 draft implemented this application of cycle walking as an optional profile. Black and Rogaway prove that cycle walking induces a uniform permutation on the target subset when the underlying block cipher is ideal [31]. Applying it to MHFE assumes that MHFE behaves like an ideal permutation. That theorem applies to the mathematical core before the operational rejection rule is applied; the core still permits returning the starting point.

Direction A and Direction B: geometry comparison

Two research directions are retained:

- **Direction A — balanced Feistel, the geometry of suite 3.** Use 128|128 bits for the 256-bit container. The formulas of suite 3 describe this direction.
- **Direction B — source-heavy 1:3 Feistel, alternative candidate.** Update one quarter using the remaining three quarters, then rotate the state. A 128-bit state splits as 32|96 bits; a 256-bit container needs 64|192 bits instead.

The table below keeps the earlier comparison of hypothetical original-length states. It does not show the geometry of the universal 256-bit packing that suite 3 uses:

Source words	Entropy state (bits)	Direction A split (bits)	Direction B split (bits)
12	128	64 / 64	32 / 96
15	160	80 / 80	40 / 120
18	192	96 / 96	48 / 144
21	224	112 / 112	56 / 168
24	256	128 / 128	64 / 192

For a universal 256-bit outer payload, every source length instead uses 128|128 bits in Direction A or 64|192 bits in Direction B. Input entropy and permutation state size are different quantities. Packing a short source together with deterministic verifier redundancy into a longer state creates no additional source entropy.

Evidence and trade-offs

Hoang and Rogaway analyze their unbalanced $\mathrm{Feistel}^r[m, n]$ construction using independently and uniformly random round functions [21]. The figure and appendix numbers below refer to the full version, the ePrint revision of November 29, 2018. Figure 4 explicitly compares proven CCA-security bounds on a 128-bit string for $m = 32$, $n = 96$ (bold curves) and the balanced $m = n = 64$ (dashed curves), at 18, 36, 72, and 144 rounds. Their Appendix E comparison, particularly Figure 7 and its surrounding discussion, states that imbalance improves the bounds when enough rounds are available, while the balanced construction has the stronger bound when rounds are scarce. Their round counts apply to that paper's idealized construction.

- **State updated by one round:** Direction A updates one half; Direction B updates one quarter.
- **Rounds until each original chunk has been targeted once:** Direction A requires 2; Direction B requires 4.
- **State supplied to a round function at fixed total width:** Direction A supplies one half; Direction B supplies three quarters.
- **Main 256-bit suite and round-mask input:** Direction A uses the 128-bit right half as the specified HMAC message suffix. Direction B's 192-bit source requires a different round-function definition.
- **Potential benefit:** Direction A fits the baseline more simply and needs fewer updates per traversal. Direction B offers a larger source branch and favorable idealized multi-query bounds when enough rounds are available.
- **Main cost concern:** Direction A has guessing bounds in the random-oracle model (Theorems 3 to 5), but no PRP or SPRP proof and no justified secure round count for other properties. Direction B needs more expensive updates per traversal, and KDF cost may dominate its theoretical benefit.

Targeting each chunk once is not a security criterion. Comparisons must use equal measured latency and peak memory, not merely the same number of rounds. More sequential Argon2id calls increase work, but they do not automatically multiply peak memory by the same factor. Reducing each call's parameters to meet a time budget changes attack cost.

With the current 128-bit salt truncation, the table below gives upper bounds on the source width retained as distinct salt values, not measured entropy or security levels:

Total state (bits)	A: source width (bits)	B: source width (bits)	A: source/salt-width cap (bits)	B: source/salt-width cap (bits)
128	64	96	64	96
192	96	144	96	128
256	128	192	128	128

Hash collisions, structured plaintext, and knowledge available to the attacker matter in addition to these widths. A 128-bit salt cap is not automatically a 128-bit security bound on the entire cipher, because the effective round function also processes the state.

For an original-length 12-word experiment, the larger source branch makes Direction B particularly interesting. For 18 words, its benefit must be assessed alongside the salt truncation and the changed round function. For the universal 256-bit container, Direction A already has a 128-bit source branch, so Direction B needs a concrete demonstrated advantage to justify its additional complexity. These are research priorities, not security findings.

Known-pair filtering cost

Write one Direction B round on four equal chunks as:

$$(A, B, C, D) \to (B, C, D, A \oplus G_i(B \| C \| D))$$

After three consecutive rounds, the initial D becomes the first output chunk without modification. Given a known plaintext/ciphertext pair, an attacker can evaluate the rounds outside a three-round gap and test that equality without evaluating the gap. In an adaptation with one KDF evaluation per round, a four-round design therefore allows an initial filter using only one KDF call. Equivalently, the first output chunk after four rounds is $A \oplus G_0(B \| C \| D)$.

In an idealized random-function model the four-round filter has a false-acceptance rate of 2^{-w}, where w is the chunk width: 32, 48, or 64 bits for states of 128, 192, or 256 bits. Surviving guesses require further checks. For balanced Feistel, a related known-pair filter can omit one round. Part I describes this filter. For MHFE's suite 3, Theorem 4 gives a success bound that matches the filter's eleven-call strategy in the random-oracle model up to the stated fractional credit, error terms and false-match losses.

Direction B therefore requires a separate round count, round-function definition, and password-attack analysis. A larger KDF input alone does not justify choosing it.

Reference Implementation

The implementation description and performance measurements in this section concern the suite 3 snapshot used for its public corpus and version 0.4.0. The later suite 4 and optional profiles are part of specification version 0.5.0. Their current vector locations are linked under Test Vectors below.

The current repositories are mhfe_core and mhfe. The specification's Reference Implementation section distinguishes their roles and identifies the historical revisions used for the suite 3 vectors. Version 0.4.0 implements only suite 3. Its Rust library, command-line tool and WebAssembly build use the reference C implementation of Argon2 as their single Argon2 engine. Browsers get two builds of that code: one with threads for cross-origin isolated pages and one without threads for all other pages. Version 0.3.0 of the implementation remains available for suite 2.

The library, the command-line tool and the browser API detect the source length automatically after one inverse permutation. They return a unique short-source match, fall back to an unverified 24-word reading when none matches, and report every matching length when detection is ambiguous. They also accept an explicit source length that replaces detection. The current rules forbid this: a stated length never replaces detection. In this respect versions 0.4.0 and 0.5.0 do not conform to the current specification.

Creation refuses a container equal to its source. When the implementation finds too little free memory, or the operating system refuses to reserve the Argon2 memory, it stops with a typed error, `NOT_ENOUGH_MEMORY` or `MEMORY_ALLOCATION_FAILED`, instead of reducing the memory. The operating system or the browser can still end the process during the work.

The operational result exposes only the recovered phrase, its word count and whether it passed a verifier. Source entropy, packed states, Argon2 outputs, masks and round traces appear only in the separate vector code, which works only on the fixed public test inputs. Buffers that the implementation owns and that hold passwords, phrases, entropy and round material are zeroized when they are dropped or reused, and the C code wipes the Argon2 work area at the end of every round. Short-lived working buffers inside dependencies are not wiped, and in the browser a failed Argon2 round leaves its work area to be discarded with its worker. The implementation's security notes list these limits. This zeroization is a memory-hygiene measure rather than a guarantee that compilers, allocators, browser strings, operating systems or earlier reallocations leave no residual copies.

No cryptography specialist has independently reviewed the implementation, and it is not independent evidence for its own vectors. The independent evidence is the OpenSSL-based verifier recorded with the suite 3 corpus, which replays every transcript in both directions and every recovery case through its recovery, including the current recovery expectations. A toy reduced-width model must not be used as evidence of production security.

Performance measurements

The table below gives whole-operation timings at the default settings, each measured from start to end on the author's laptop (Intel Core i7-1260P). The command-line figures come from the release builds of commit `ac195e8373df24fb11a1b0b241a324575ea09b03`. The browser figures come from the MHFE 0.4.0 browser package, driven in headless Chromium through the MHFE panel of the Wallet Key Derivation Tool. The measurement record, published in commit `b1d83504ba2458681111c14cade52fa0a178ddb4`, keeps the details:

Environment	Recovery	Encryption with its check
Command line	67 to 72 s	2 to 2.3 minutes
Browser, fast mode (served, cross-origin isolated, threads)	90 to 95 s	2.7 to 2.8 minutes
Browser, standard mode (page opened as a file, one thread)	about 4 minutes	about 7.7 to 8 minutes

In paired runs on the command line, the build with SSSE3 took about 6 to 10 percent less time than the default build. One Argon2id call at 2 GiB and 12 passes took 4.6 to 5.0 seconds natively with four threads, 5.7 seconds in the browser's fast mode and about 19 seconds in its single-threaded standard mode.

These are observations on one computer, not a normative performance target, a minimum attacker cost or a guarantee for similar machines. CPU power policy, thermal state, memory bandwidth, compiler, Argon2 implementation, native versus WebAssembly execution and concurrent load can change the result materially. Broader measurements on representative x86-64 and ARM64 systems would be needed before these figures could support a portability or deployment claim.

Test Vectors

The suite 3 corpus is published in `vectors/suite3/`: 17 positive transcripts, seven recovery cases and 54 fast validation cases. The reference implementation generated them. An independent OpenSSL-based verifier replayed every transcript at full cost in both directions and every recovery case through its recovery, including the current recovery expectations. Each transcript records the source mnemonic and entropy, the normalized password bytes, the packed state, every round's salt and mask input messages, salt, Argon2id output, mask and state, the container and the recovered result. The corpus notes record provenance and checks, and the specification's Test Vectors section states the requirements.

The current suite 4 corpus is in `vectors/suite4/`, and the optional wallet-check, repair-word and password-check-word examples are in the profile vectors. Each corpus records its own coverage and verification evidence. The historical suite 3 replay described above does not establish verification of these later sets. The suite 2 vectors are archived in `vectors/archive/suite-2/` and must not be replayed under suite 3.

Following BIP 3's recommendation that test vectors be available under CC0-1.0 or FSFAP in addition to any other license, the vectors are released under CC0-1.0 so that implementations can copy them without license friction [1].

Known Limitations

This section records the present boundaries of both experimental suites and the optional profiles. It is not a roadmap and does not commit the author to further research or implementation work.

- The selected Argon2id and Feistel parameters are supported by limited measurements on the author's laptop.
- Suite 3's short-source recovery verifiers intentionally provide an offline password-checking signal. Its ordinary 24-word source mode, by contrast, has no internal wrong-password test, and automatic source-length detection remains probabilistic.
- Versions 0.4.0 and 0.5.0 of the reference implementation accept a stated source length that replaces detection, which the current length rules forbid.
- Suite 4 has no built-in verifier at any supported length. Its narrower states limit salt diversity, reveal the source's word count and require separate analysis of password filters and shared work; see the narrower-state analysis.
- The optional wallet check for new 24-word wallets is a 16-bit statistical filter, not confirmation of wallet identity. It conditions the source distribution, and the deniability theorems for uniformly generated sources do not automatically apply. With a nonempty passphrase it checks the recovered mnemonic and passphrase together. With the empty passphrase it filters MHFE password guesses alone.
- Repair words and the password check word add redundancy, not authentication. They repair only within their stated bounds. Disclosure of the repair words reveals information about the container, and disclosure of the password check word reveals information about the password. Passing either check does not establish the intended wallet.
- Password-guessing bounds are proved only in the random-oracle model, for suite 3 with one container, one known pair or a 24-word source with an unknown BIP39 passphrase or generated with the wallet check (Theorems 3 to 5, Corollary G10). What remains open is listed under Open questions for review, including several containers, suite 4 and the salt-separation hypothesis A1.
- Final-word-preserving cycle walking is not part of suite 3. It was implemented as an optional 24-word profile of an unreleased suite 2 draft and is analysed here as research. Its variable runtime and public final-word class would require separate analysis. The 253-bit shortcut and source-heavy unbalanced Feistel remain non-normative research alternatives and are not part of the reference implementation.

The implemented utility may be used for public experiments and interoperability testing under the warnings in this document. Nothing in this section should be read as a promise of a future version or as a claim that the current construction is suitable for protecting real funds.

Research directions

These directions were first recorded on 2026-10-03, after suite 4 was added, and extended on 2026-10-05. None changes the algorithms of suite 3 or suite 4. The optional wallet-check, repair-word and password-check-word profiles are now defined in the specification.

This section keeps the analysis of the wallet check and the password check word, and it sets the repair words apart from the planned container check words. It also distinguishes all three profiles from the remaining proposals, which are not part of the specification. Each direction is written down with its costs, so that a later suite or application can take it up or leave it. The directions cover application aids, possible changes to the round function and the source, arrangements of the existing suites, and alternative formats that would use a different cryptographic construction. Container check words are planned application work; the other proposals remain research directions with the limits stated below.

Container check words: a hash of the container (planned)

The specification now defines repair words, MHFE-REPAIR-1: linear Reed-Solomon parity that corrects unreadable and wrong words. Container check words are a different, planned record: a hash that recognises a copy and filters repair candidates, but corrects nothing by itself.

A short public digest of the container lets an owner or an heir check a copy and identify candidates for repairing it without the password and without any Argon2id work. The digest is computed from the container alone. Here Y is the decoded container entropy, serialized as its raw bytes without the BIP39 checksum, and n is its word count:

$$C = \text{SHA-256}(\text{domain} \| \text{BE32}(n) \| Y)$$

Its first 33 bits, written as three English BIP39 words, are the container check words. They must be labelled as a check: three words are not a valid BIP39 mnemonic length. All 256 bits, written as 64 hexadecimal digits, are the seal. The domain string, the exact layout and test vectors from two independent implementations are still to be fixed.

The digests have four uses:

- **Checking a copy.** Every copy and every new engraving is compared with the check words before a password is typed. This matters most in suite 4, whose checksum has only 4 to 7 bits.
- **Repair.** For e unreadable words at known positions, the application enumerates candidates, keeps those with a valid BIP39 checksum and compares their check words. In an ideal-hash model, about $2^{11e-\text{CS}-33}$ wrong candidates survive on average, where CS is the BIP39 checksum length. This count excludes the correct candidate and is a probabilistic estimate, not a guarantee of unique repair. With a 24-word container ($\text{CS} = 8$), the estimates are 2^{-30}, 2^{-19}, 2^{-8} and about 8 for one through four unreadable words. For suite 4, three unreadable words leave about $1 / 16$, $1 / 32$, $1 / 64$ or $1 / 128$ false candidates for 12, 15, 18 or 21 words; four leave about 128, 64, 32 or 16. Naive enumeration of three missing words in a 24-word container needs about 2^{33} BIP39 checksum evaluations and 2^{25} container-digest evaluations. When the last word is missing, its checksum bits can instead be recomputed, reducing the entropy assignments to 2^{25}. A substituted word at an unknown position requires trying the positions too: up to $n \cdot 2047$ replacements for one substitution, and approximately $\text{binomial}(n, e) \cdot 2047^e$ for exactly e substitutions. Every proposed correction is shown to the user before recovery, and unresolved candidates remain explicitly ambiguous.
- **Telling container phrases apart.** Copies of different containers, for example an old one kept after a re-encryption, are recognized before a recovery fails for an unclear reason.
- **Detecting a substitution.** The 33-bit check words provide no authentication: a search for another valid container matching a fixed set of check words takes about 2^{33} digest evaluations on average in the ideal-hash model. The full seal can detect a deliberate swap when its original value is obtained from an independently trusted record that the attacker cannot also replace. Neither digest authenticates a container phrase if the attacker can replace both it and the reference.

For an adversary who already has the whole container, the digests add no information about the password: both can be computed from that container, which the deniability experiments already provide. This argument applies only to a digest of the container. A digest of the password supplies a cheap password test. A digest of the source or a recovered phrase supplies a check after recovery, with the composition losses discussed in A check for new 24-word and suite 4 sources by choosing the entropy. A digest of settings alone tests the settings, not the password, but can reveal settings that were intended to remain secret. The container-check format therefore takes only the container as input.

The digests also help an adversary who holds a damaged or partial copy, by identifying candidate containers. The practical number of words that can be repaired depends on their positions, the checksum length, measured search throughput and the time budget. Five known missing positions can require 2^{55} checksum evaluations when the last word is not among them. Even at a hypothetical billion evaluations per second this is about 1.14 years. A full seal does not make that enumeration cheap.

Once the container is reconstructed, its password protection is the same as for a complete copy, subject to the existing attack assumptions. Both digests are therefore kept like a copy of the container. Written next to a container phrase, they also show that the phrase is a special backup rather than an ordinary one, so by default they belong on a separate card, kept apart from the container phrase like the repair card, not on the backup itself. The 256-bit seal is as long as the entropy of a suite 3 container, and longer than that of a suite 4 container: against complete loss, another exact container copy preserves more useful recovery information.

Argon2i on the rounds with a public salt

Every round of suites 3 and 4 uses Argon2id. In two calls the salt can be computed without the password: S_{11}, which every recovery derives first from the container's left half, and S_0 when the source is known. As Observers on the same computer explains, an observer who records the data-dependent memory addresses of such a call can reject a guess after about half a pass of that call. In an idealized model this is roughly 1/288 of a full guess, or about 8.2 bits.

A future suite could use Argon2i in these two calls and Argon2id in all others. Argon2i chooses its memory addresses independently of the password and the salt [4], so the recorded addresses of the round-11 call would be the same for every guess. The next call with data-dependent addresses is round 10, whose salt depends on the output of round 11. In this trace-comparison procedure, each guess computes the whole round-11 call before it compares addresses, so each guess costs about $1 / 12 + 1 / 288$ of a full recovery's modeled work, or 3.5 bits instead of 8.2. This is a cost estimate for the described procedure, not a lower bound for every side-channel attack.

The price is memory hardness: data-independent addressing is more exposed to time-memory trade-off attacks than data-dependent addressing [4]. If both Argon2i calls of a full recovery became free, ten Argon2id calls would remain. With calls counted as equal-cost units, the reduction for that procedure is $\log_2(12 / 10)$, about 0.26 bits of work. This does not bound the loss under cheaper filters, structural attacks or combined time-memory and side-channel attacks. The margin in the number of Feistel rounds concerns the permutation; it does not prove a minimum guessing cost for the proposed mixed construction.

Suites 3 and 4 keep Argon2id in every round. The gain applies only when an observer shares the computer, which lies outside the assumed trusted recovery environment. The specification strongly recommends a trusted offline computer, but that recommendation does not technically prevent shared-cache observation. The precision that the observer model assumes is speculative, and one function type throughout keeps implementations and test vectors simple. The rule is recorded for a suite created for other reasons: calls whose salt can be computed without the password use Argon2i, and the other calls use Argon2id.

A second memory-hard function

A future suite could alternate two independent memory-hard functions, for example Argon2id in odd rounds and scrypt in even rounds. Suppose a shortcut eliminated the evaluation cost of one function while the other function and the composition stayed sound. A full recovery would then retain six costly calls. With equally costly calls this halves its modeled work, a loss of about one bit. This is not a proof that the mixed construction survives an arbitrary break of either primitive: cheaper password filters, shared weaknesses and the composition need separate analysis. Such a hedge could help copies taken before a weakness became known, whereas later re-encryption cannot change the protection of an already copied container.

It is not adopted. It needs two engines and two sets of test vectors. At 2 GiB to 3 TiB, scrypt runs into the memory limits of common implementations and of 32-bit builds, and it is slow in WebAssembly. Its memory accesses also depend on the password, so scrypt rounds would add the side channel described above. Argon2id is a standardized and widely reviewed function [4], [26]. Against a partial weakness, one more dice word is the simpler hedge: it adds about 12.9 bits, as Attacks in the future shows.

A hidden wallet behind an honest disclosure

This arrangement uses suites 3 and 4 unchanged. Deriving a second wallet from a container with another password is already possible, as the Deniability section explains for decoys. This section records the reverse assignment of roles: the disclosed password opens the owner's genuine wallet, and the hidden wallet is the derived one. It is a way of using the construction, not a new cryptographic primitive.

1. The owner has a wallet W in genuine use: its history, exchange withdrawals and tax records belong to it.
2. The owner encrypts it with a password Q that they are prepared to disclose: $Y = E_Q(W)$, an ordinary creation.
3. With a second, strong password P, drawn independently, the owner computes $H = D_P(Y)$, read at the state width: 24 words in suite 3, the length of W in suite 4. H is not written down; the container and P give it again.
4. H is funded only from sources linked neither to the owner nor to W.
5. Under coercion the owner discloses Q. The container and Q open W. Its history, the records tied to it and, for a short suite 3 source, the verifier all agree, because they are genuine.

Compared with the decoy of the Deniability section, four of the limits of the guarantee fall away:

- the disclosed password need not be drawn like the hidden one;
- the disclosed wallet's history may begin long before the container;
- records tied to the disclosed wallet are consistent with it;
- a short suite 3 source passes its verifier.

The argument is shorter than that of Theorem 2. Y, Q and W are produced exactly as by an honest owner and do not depend on P. Everything the adversary sees, except the record of H, therefore has the distribution of an honest disclosure. Only a lookup of H or a fixed point can tell the two apart; the lookup is reached by finding P or by directly guessing the hidden phrase's entropy. The expected bound has the form $k \cdot p_H + l \cdot 2^{-b} + 12 \cdot (q + 24) \cdot 2^{-256} + 2^{-b}$, where p_H is the largest probability in the distribution of P given $P \neq Q$, and b is 256 in suite 3 and ENT in suite 4. The proof has still to be written and reviewed.

The arrangement resembles the hidden volume of a deniable file system whose outer volume holds genuine data. The leaks through the operating system and applications reported for those [50] apply to it as well.

Its limits:

- H protects only money that never touches W or anything linked to the owner. Moving the existing balance of W to H links the two records.
- H depends on the exact container, suite and settings. A new container for W, made for example to change Q or to move to another suite, does not carry H over. The specification's application requirements give every user the warning to keep the old container, P and the settings until the funds of H have been moved. Forgetting P or losing every copy of the old container loses H unless its funds were moved first.
- W and Q give the container again, and with P they give the hidden wallet: $H = D_P(E_Q(W))$. They must be guarded like the container.
- Q still protects W against theft and must be strong if W holds funds.
- An application that offers this shows the receiving addresses of H once, for funding, and keeps nothing. A watch-only wallet of H on an everyday device is evidence outside the model.

- Publishing the arrangement raises the probability an adversary assigns to any owner having a hidden wallet. The bound does not depend on that probability, but an adversary's behaviour may.

Check words for derived wallets in the current suites

A full-state wallet derived from a container under another password, $H = D_P(Y)$ in suites 3 and 4, has no verifier for that opening.

Check words are entirely optional. The owner need not generate, remember, record or enter them. Recovery still uses the container, password, suite and settings, with the BIP39 passphrase applied afterwards if used. Losing the check words does not prevent recovery, and declining this option does not weaken MHFE encryption. They provide only an additional way to compare the recovered seed with a previously trusted value. Only the usual independent wallet-identity reference confirms the intended wallet in the specification's sense. A future interface would leave this option off by default, place it in additional settings and always allow recovery without the check words.

An application could show two English BIP39 words computed from the final BIP39 seed, including the wallet's passphrase if one is used, under a fixed tag that also binds the suite and word count. The owner learns them with the password but types them into a separate field. The words never enter the password or Argon2id. They differ both from the public container check words, which hash only the container, and from the private password check word, which repairs one forgotten password word. An application profile has still to define this proposal's tag, serialization, mapping to words and independent vectors.

- In an ideal-hash model, a distinct wrong seed matches the two words with probability 2^{-22}, about 1 in 4.2 million. One word gives 1 in 2,048 and suits only a few manual attempts. These are consistency checks, not authentication or bounds on every attack.
- With two words, an application could try bounded typing variants, each ordinarily costing one recovery. For 100 wrong candidates the union bound is $100 \cdot 2^{-22}$, about 1 in 41,900. Dependence between candidates does not invalidate that bound when each candidate has the stated marginal estimate.
- The straightforward test recovers a mnemonic once per MHFE password and derives its BIP39 seed for each passphrase candidate. Hashing that seed with an independent passphrase included does not itself supply a separate mnemonic check. The pair-search term remains as in Composition with the BIP39 passphrase. This describes a procedure, not a proof that every attack requires full recovery or an optimal Cartesian-product search. Hashing the mnemonic entropy instead would provide a separate mnemonic test.
- With or without the words, the result stays labelled not verified in the specification's sense: these words are not one of the rehearsal check's wallet-identity references. A match would be reported as a 22-bit check-word match. It does not confirm an address, network or derivation path. Whether it may support an additional confirmation workflow is a decision for a future profile.
- The words would be remembered or guarded with the password, such as in an heir envelope, never with the container. Leaked check words recognize a candidate seed even for an empty wallet. Several records linked to one container may also be evidence of several wallets.
- A password invented without computing its recovery does not normally come with matching check words. A prepared decoy needs one recovery and seed derivation to learn them. If recovery is available, however, any chosen password produces a seed and its own words. What confirms the intended seed is a match to a previously trusted reference, not a newly computed pair of words. These operational limits do not extend the existing deniability theorems.

Derived wallets of a chosen length

A separate operation could instead take the first ENT bits of the raw recovered state $D_P(Y)$ and encode them with their own BIP39 checksum, without interpreting the rest as a verifier. The specification does not define that operation: ordinary recovery gives a short reading only where its verifier matches, whatever length the user states. What follows is a candidate application profile, not a change to either suite.

- Suite 3's 256-bit state supports all five lengths. For suite 4, a prefix reading could only have as many entropy bits as the recovered state contains: a 12-word container permits only 12 words, a 15-word container permits 12 or 15, and similarly for 18 and 21. Longer readings require another construction, rather than truncation. The default full-state reading remains the container's width.
- A candidate length signal is a password of the form `12:<secret>`, `15:<secret>`, `18:<secret>`, `21:<secret>` or `24:<secret>`. In the explicitly selected derived-wallet operation, the program would first apply existing password validation and NFKD normalization, then parse the exact ASCII prefix from the normalized result. The entire normalized password, including the prefix, would enter Argon2id unchanged. The profile would fix one syntax and require a nonempty strong secret. It would reject malformed or unsupported prefixes before Argon2id and retain the normalized length limit. Ordinary MHFE recovery would treat the prefix as literal password text. Codes such as `1:` to `5:` are another possible design, not interchangeable spellings of these passwords.
- The prefix signals the length but adds no secret entropy. Each independent wallet needs an independently generated secret core. Its complete normalized prefixed password must differ from the normalized main password, regardless of settings. If one secret core is disclosed, changing only its public prefix or settings does not protect another wallet.
- Prefix readings of one complete password share the first 128 entropy bits: their first eleven words are identical and their twelfth words share seven bits. A disclosed 12-word reading reveals all 128 bits, leaving only $160 - 128 = 32$ unknown bits in its 15-word reading. Searching these bits against wallet data needs no further Argon2id work. One complete password must therefore be assigned one reading length only.
- A chosen-length reading would be unverified by construction. Confirmation would use an independently known wallet-identity reference; the proposed check words could report a weaker seed match. A separately generated BIP39 passphrase remains additional protection and must be supplied when checking the actual wallet. The main wallet retains its normal source-verifier rules.
- For a hidden suite 3 opening, the candidate profile would refuse a password whose full recovered state accidentally passes any short-source verifier. In the uniform-state model this event has probability about 2^{-32}. The refusal prevents ordinary detection from presenting the hidden opening as a consistent short source. The rule does not apply to suite 4, which has no such verifier. Nor does it apply to ordinary decoy preparation, whose password-selection rules must remain those of the stated deniability experiment. The refusal conditions the hidden opening's distribution and needs its own analysis; the existing theorems do not cover it automatically.
- Such a short hidden reading cannot claim to be the source of the original suite 3 container. In an idealized lookup model its entropy-width term would be $l \cdot 2^{-\mathrm{ENT}}$ instead of $l \cdot 2^{-256}$. This is a proposed analysis term, not an established extension of the hidden-wallet bound.
- When the complete password is known and only the reading length was forgotten, one recovery supplies all prefix readings that fit the state. Comparing all five in suite 3 gives a modeled union bound of $5 \cdot 2^{-22}$, about 1 in 839,000, for wrong check-word matches. If the password's length prefix itself was forgotten, each candidate prefix is a different password and requires a full recovery; the one-recovery saving does not apply.

- A new container does not carry these wallets over. The specification's application requirements give every user the warning to keep the old container until their funds have been moved. Re-encrypting such a wallet as a source falls under the specification's rule for a reading without a verifier: the ordinary short-source verifier cannot confirm these projections, and encrypting a short projection under its old password does not recreate the original container. The specification does not define the proposed reading operation.

Nested containers

A container is itself a valid phrase, so the complete creation procedure can encrypt it again. Creation with P_1 produces the inner container; creation with P_2, treating that container as its original seed phrase, produces the outer one. Recovery first uses P_2 to recover the inner container, then P_1 to recover the original seed phrase. The backup shows only the outer container. When both stages use 256-bit states, this is $Y_1 = E_{P_1}(X)$ followed by $Y_2 = E_{P_2}(Y_1)$. With a shorter suite 4 inner container and a suite 3 outer layer, the outer procedure first packs Y_1 with its short-source verifier. Its raw permutation input is therefore $Y_1 \| V_{256-\mathrm{ENT}}(Y_1)$, not Y_1 alone. Each stage retains its own suite, settings and fixed-point refusal.

One use is a separation of people. The owner keeps P_1 and a custodian keeps P_2, and neither learns the other's password. The custodian removes the outer layer on their own offline computer and returns Y_1; before that, the custodian can check the owner's identity or wait for an agreed period. The arrangement gives no lasting control over spending: once Y_1 is returned, the owner alone can open the wallet. Nor does it revoke anything: older copies of the container keep working with the passwords they were made with.

Two layers do not establish multiplicative password security. A full recovery performs two recoveries, while joining two independent password parts into one ordinary password needs one. With a known original seed phrase and outer container, a meet-in-the-middle attack enumerates inner passwords forward and outer passwords backward, matching the complete intermediate representation. Its work is roughly the sum of the full search spaces. Storing the intermediate records, however, needs memory proportional to one of those spaces, and smaller-memory strategies have different costs. This known-source attack does not establish the same cost for a container-only adversary.

For a suite 4 inner container of 12 to 21 words and a suite 3 outer layer, the outer layer has an $r = 256 - \mathrm{ENT}$ bit verifier. This separates the password searches only when false outer matches are rare. Assume independently uniform spaces of N_1 inner and N_2 outer passwords, equally costly recoveries C, and only a wallet-identity reference for the original seed phrase. A sequential search then has modeled expected work

$$C\left(\frac{N_2}{2} + \frac{N_1}{2} + \frac{N_1 N_2}{2^{r+1}}\right)$$

The last term searches the inner-password space for each false outer match. For two four-word EFF passwords, $N_1 = N_2 = 7776^4$, false matches are negligible for inner lengths 12, 15 and 18. The sum of the full spaces is then about $2^{52.7}$, with expected work about $2^{51.7}$ recoveries, instead of a full pair space of about $2^{103.4}$. With 21 words ($r = 32$), about 426,000 false outer matches precede the correct one on average, and the described sequential procedure instead needs about $2^{70.4}$ recoveries. These are estimates for specified attacks, not security lower bounds or a proof of optimality.

A failed outer verifier shows an inconsistency in the outer recovery before the inner password is used. Without such a verifier, a final wallet mismatch may leave the failing layer unclear. The layer order and both sets of settings must remain available, and the custodian must retain P_2. The deniability theorems do not cover layered containers. Threshold sharing of the finished container, or a multisignature wallet, may serve a custody goal more directly. Seed XOR [58] can provide an all-shares-required split, but it is not a general threshold arrangement.

A check for new 24-word and suite 4 sources by choosing the entropy

The specification defines the optional source profile for new 24-word phrases, including its exact wallet-check bytes, generation and recovery procedure, empty-passphrase option and public vectors. This section analyses that profile and compares it with entropy-only checks and with possible checks for shorter suite 4 sources. Those alternatives are not defined profiles.

Entropy-only alternative. A 24-word source and every suite 4 source have no built-in verifier: a wrong password gives another valid phrase. An application that generates a new wallet could offer an alternative creation mode that builds a check into the source itself. For a fixed public domain tag, the generator draws uniformly random entropy E again until $\mathrm{SHA\text{-}256}(\mathrm{tag} \| E)$ begins with k zero bits. In an ideal-hash model this takes 2^k hash trials on average and leaves approximately $\mathrm{ENT} - k$ bits of generation entropy. For $k = 16$, the expected count is 65,536; a 24-word source retains about 240 bits, while suite 4 sources retain about 112, 144, 176 or 208 bits. This is hash-only setup work, whose elapsed time needs an implementation-specific measurement. The trials can be derived deterministically from one random seed, so that a generator fed with dice stays verifiable. The source then has no more entropy than that seed. For this variant, the estimate above also assumes that the random seed has at least the retained entropy and that the derived trials behave like fresh uniform draws.

If wrong recoveries are uniform candidate entropies, each passes the check with probability about 2^{-k}; that is a model estimate, not authentication or a proof about every attack. A recovery of a wallet created in this mode then tells the owner or an heir that the password or a setting was wrong, without a wallet-identity reference. Suites 3 and 4 do not change: the container is made from the source as usual, and the check is a property of how the source was generated. The mode thus gives a 24-word source a recovery filter without stored check words, while keeping about 240 bits of entropy. Its 16-bit filter is weaker than suite 3's 32- to 128-bit built-in verifiers for short sources, and those verifiers do not require conditioning the source entropy.

For a new 24-word wallet, an application may offer the defined wallet-check profile as an explicit choice. An analogous choice for shorter suite 4 sources would require a separate profile and analysis. The two source-generation approaches compared here are:

1. **An ordinary random phrase**, as today: no meaningful check after recovery (the 16-bit check is still evaluated and passes only by chance), the strongest combination with a BIP39 passphrase, and the deniability theorems as stated.
2. **A phrase with a wallet check**: a check after recovery, as a short source has in suite 3, with the costs listed below.

The specification requires applications offering the defined profile to explain its trade-offs before the owner's choice. It also permits offering the profile only with a nonempty passphrase. The container carries no sign of the profile, so every implementation evaluates the check on every 24-word reading, whether or not it offers the profile, with the passphrase the user enters or the empty one. The defined profile uses a check over the seed with $k = 16$, with the wallet's BIP39 passphrase, or the empty string when the wallet has none. The specification explains the profile's recommended use with a strong, independent passphrase. The following comparison first considers the entropy-only alternative, then the check over the seed.

The costs of the entropy-only alternative:

- **BIP39 passphrase.** Like the built-in verifiers of short sources in suite 3, this alternative filters MHFE password guesses without a BIP39 passphrase; the filter widths and generation costs differ. Without a passphrase and with an identifiable public wallet history, a candidate phrase can already be compared with that history, and both comparisons are negligible beside the Argon2id work of a guess. With an independent passphrase, the check filters MHFE password guesses separately, and the pair search is no longer a product. In the example of Composition with the BIP39 passphrase, the expected time falls from about 12,195 years to about 17.2 years with $k = 16$, close to the roughly 17.0 years for a 12-word source. Roughly 8,192 false mnemonic matches still require their own passphrase searches, adding about 68 days beyond the MHFE password search in that model; the costs do not separate completely. A 24-word source created without the mode keeps the strongest combination with a passphrase. With the check over the seed described below, the search still runs over pairs.
- **Decoys.** If the adversary knows that this creation mode was used, a recovered decoy must pass its check too. The described independent password search takes about 2^k recoveries on average. If run sequentially at the measured 70-second recovery time on the reference laptop, the search takes about 53 days at $k = 16$ and about five hours at $k = 8$, where a wrong password passes once in 256. This is neither a fixed completion time nor a lower bound on every way to prepare a decoy. Both conditioning the source entropy and selecting a decoy by its recovered phrase change the assumptions of the existing theorems. Those theorems do not establish deniability for this mode; that would need a separate experiment and proof.
- **Recognition.** For a fixed MHFE-specific tag, a passing source is statistical evidence of preparation, not proof of provenance: an ordinary uniformly random source passes with probability about 2^{-k} too.
- **New wallets only.** An arbitrary existing source cannot be made to satisfy this fixed predicate without changing its mnemonic and wallet, although it may already pass by chance. A recovery tool can always report whether the recovered phrase passes. A failure indicates a wrong input only for a wallet created in this mode.

A check over the seed with the passphrase. With a BIP39 passphrase, the check can instead be computed over the BIP39 seed that the phrase and the passphrase give together: the generator draws E again until a digest of the seed begins with k zero bits. The owner then gets one check after recovery that covers the MHFE password, the settings and the passphrase together. A pass is statistical evidence, not proof: a wrong password or passphrase passes with probability about 2^{-k}, about one in 65,536 at $k = 16$, and a pass never identifies the wallet. A failure means something only for a wallet created with this check.

In the search described under Composition with the BIP39 passphrase, a password guess can be tested only together with a passphrase guess, because the check needs both. That search still runs over pairs, as without any check. In that example its modeled expected time stays at about 12,195 years instead of falling to about 17.2 years. Theorem 5 bounds seed-recovery probability for given budgets in the random-oracle model, with explicit error terms. The approximate budget for success probability one half follows while those terms are negligible. The theorem assumes independent secrets, a generation that meets its selection rule, all funds under the passphrase, no data that identifies the phrase without the passphrase, and no funded wallet that another MHFE password opens on the same container.

- Against an attacker who knows the wallet's addresses or its fingerprint, the bound is that of an ordinary 24-word phrase with the same passphrase, apart from error terms.
- Against an attacker without wallet data, the check is a 16-bit pair filter that an ordinary phrase does not give. The bound still holds, but such an attacker gains from the check.

Testing a pair costs about as much as comparing it with the wallet's addresses. Without a passphrase, this variant filters MHFE password guesses alone, like the check over the entropy. Its own costs:

- For a given passphrase, about $\mathrm{ENT} - k$ bits of entropy remain: about 240 of 256 for a 24-word phrase at $k = 16$.

- Each trial at creation needs one PBKDF2-HMAC-SHA512 with 2,048 iterations. A quick measurement gave about 0.76 ms on one core of the reference laptop, so $k = 16$ takes about a minute on average on one core and $k = 12$ a few seconds; a browser is slower. This is not a promised completion time.
- The passphrase is fixed at creation: changing it normally loses the check, though another passphrase can pass by chance or after a search.
- Another passphrase normally fails the check, but anyone who searches for a passphrase that passes with a given phrase finds one after about 2^k trials. A decoy passphrase is therefore a separate strategy, and the check neither rules one out nor makes one safe. A funded wallet under a decoy passphrase that the attacker knows, for example after its disclosure, identifies the phrase without the real passphrase. Theorem 5 then no longer applies (A.7), and an attacker who knows that wallet can test MHFE passwords alone, as in Proposition G8.
- If the adversary knows that this profile was used, a disclosed decoy must also pass with its disclosed passphrase. For a fixed decoy passphrase, the described independent MHFE password search takes about 2^k recoveries on average; this is not a lower bound on every preparation strategy.
- The deniability theorems assume a uniformly random phrase. They do not by themselves cover a phrase drawn to pass this check; that would need a new experiment or proof.
- For a wallet with a nonempty passphrase, the check gives a meaningful result only when that passphrase is entered with it. The empty-passphrase variant needs no second secret and offers only the MHFE password's protection, with no BIP39 passphrase layer.

Relation to suite 3's short-source checks. With the empty passphrase, the profile gives the owner and the attacker a recovery filter without a wallet-identity reference, as do suite 3's built-in verifiers. This is the shared convenience, not an equivalence of strength or cost: the profile uses 16 bits, conditions the source entropy and requires a seed search at creation. Its exact definition and recommended use are in the specification and are not repeated here.

The entropy-only and shorter-source alternatives would each need their own profile. For wallets created without a check, a trusted reference for the actual wallet confirms recovery without restricting the source's entropy. When a BIP39 passphrase is used, that reference must be derived with the same passphrase to preserve the composition comparison. A reference derived without it would itself supply a separate mnemonic check.

A private check word for generated passwords

The specification now defines this rule as the optional profile MHFE-PASSWORD-CHECK-1. The profile has only five words and a check word; the analysis below explains its choices. A generator draws five words independently and uniformly from the EFF large list [6] and appends a private check word. For word indexes d_1 through d_5, each from 0 to 7,775, the rule is

$$c = (d_1 + 5 \cdot d_2 + 7 \cdot d_3 + 11 \cdot d_4 + 13 \cdot d_5) \bmod 7776$$

For this profile, the specification fixes the list ordering, the coefficients and the exact password serialization. The MHFE password is the full string, including the sixth word. Since each coefficient is coprime to 7,776, any single substitution by a different list word is detected, and one erased word at a known position is recovered uniquely from the remaining words. This also recovers the check word when only that word is missing. A single substitution at an unknown position cannot be corrected uniquely: each position admits a possible repair.

Transpositions are not always detected. For independent uniform data words, a swap of the positions with weights u and v passes with probability $\gcd(u - v, 7776) \,/\, 7776$, including swaps of identical words. Conditioned on different words, the probability is $(\gcd(u - v, 7776) - 1) \,/\, 7775$. With the profile's weights, swaps of adjacent data words therefore pass with probabilities $3 \,/\, 7775$, $1 \,/\, 7775$, $3 \,/\, 7775$ and $1 \,/\, 7775$ when the words differ. The check-word

position has coefficient -1, so exchanging the check word with the preceding word uses the weight difference 14; under the same condition, this swap passes with probability $1 / 7775$. These figures are averages for random words, not measured human-error rates. For example, exchanging indexes 0 and 1944 at the first two positions is a concrete undetected swap.

The generation entropy remains $5 \cdot \log_2(7776)$, about 64.6 bits. The check word adds redundancy, not randomness, and it adds a word to remember. It must remain secret: publishing it removes one word's worth of uncertainty, about 12.9 bits. Passing this check confirms the password's form, not that the password opens the intended container. Recovery still needs the exact password form and the usual verifier or wallet-identity reference. Decoy passwords must follow the same generation and selection rules. The profile changes neither suite's password encoding.

The profile does not define a variant with six random words and a seventh check word. Such a variant would give about 77.5 bits; six words is the length EFF itself suggests [6]. It would, however, need its own coefficients, because with a fifth coefficient of 13 no sixth coefficient can keep both transpositions of the sixth word with its neighbours as rare as above.

This password check is separate from the check words for derived wallets. Those check words compare the recovered seed with a previously trusted value, while this one shows only that the typed words fit together. A check word computed from the wallet would not provide the same cheap, unique password repair: testing one missing word could require up to 7,776 expensive recoveries, and the checksum rule would no longer establish a unique answer.

Alternative formats for multiple password openings with recovery checks

This section asks whether one checksum-valid, 24-word BIP39 container can support several different passwords, each of which opens a wallet and passes an internal recovery check. The question is a natural one to study next to MHFE, because such a format would keep the physical backup constraint and address the lack of a verifier for additional full-width openings. The proposals below replace the construction rather than extend the algorithms of suites 3 and 4. They have no assigned suite identifier, no conformance vectors and no reviewed implementation, and the MHFE security arguments and deniability theorems do not apply to them.

Shared constraint and password-derived wallets

A 24-word BIP39 container has 256 payload bits, and that payload determines its eight checksum bits [3]. One candidate layout splits the payload into a fresh public salt S of 64 bits and a coded record Z of 192 bits. Argon2 derives key material for each password. HKDF with separate context labels could derive the masks, equation coefficients, verification keys and wallet outputs [64]. The eventual format would bind the role, word count and settings into the relevant derivations and checks. Independent access requires independently generated strong passwords. Changing only a public role prefix does not isolate wallets if their random password core is reused.

An arbitrary existing 12-word wallet requires 128 bits of source entropy. Three recovery checks of 16 bits each give the illustrative information budget $64 + 128 + 3 \cdot 16 = 240$, which leaves 16 bits beyond those constraints. This budget alone does not establish that an encoding always exists. Two arbitrary existing 12-word wallets already require 256 bits at fixed independent passwords, which leaves no room for this salt and checking budget.

An additional wallet need not store its entropy. Instead, it can be computed from its own password and S, with a distinct wallet-output context, and encoded as 12, 15, 18, 21 or 24 BIP39 words. For example, one existing 12-word wallet and two newly derived 24-word wallets need not store another 512 bits.

The record confirms the intended opening, while the derived wallet can be recovered without that record if the password, salt and derivation profile survive. Losing the salt loses this property, and changing the salt changes the derived wallets. A longer output does not create additional secret entropy: with public S, password guessing remains a

recovery route. A 16-bit check is only a consistency filter, with a modeled accidental acceptance probability of 2^{-16}. It is not authentication or a guarantee of wallet identity.

Affine equations and Gaussian elimination

For opening i, password-derived coefficients and a separately derived offset could impose equations over $\mathrm{GF}(2)$ of the form

$$A_i \cdot Z = b_i \oplus \mathrm{payload}_i$$

For the existing wallet, the payload includes its entropy and check bits. For a password-derived wallet, only the check constraints need to be encoded. Stacking the equations and solving them by Gaussian elimination avoids an exponential search. With one 128-bit stored wallet and three 16-bit checks, the illustrative system has 176 equations on 192 unknown bits. If the rows are independent, the system has 16 free variables, which are sampled uniformly. Independently pseudorandom offsets are essential: homogeneous checks alone would accept an all-zero record for every password.

Linear oblivious key-value stores (OKVS) provide a relevant foundation [65]. In that work, Definition 2 in Section 2.1 compares encodings of two key sets of the same cardinality with uniformly random values. Section 2.2 explains how uniformly random right-hand sides and uniform sampling of a full-rank linear system's solutions yield a uniform encoding. Linearity is thus compatible with formal key hiding, and this route is a reasonable first candidate for a prototype and proof. Investigating it does not require inventing a new block-cipher primitive.

The proposed application is not already covered by that definition. Its checks have known expected values, an existing wallet can be fixed, and a disclosure reveals correlated password and wallet information. Separate offsets might mask these payloads into pseudorandom right-hand sides, but their independence from the coefficients needs an explicit assumption. Hidden numbers of openings, rank failures and retry rules, and security after disclosures need a separate experiment. A useful proof target is that, after specified openings are disclosed, the record's distribution is independent of additional undisclosed openings.

Several records that share a salt and constraints may reveal their common affine subspace, and affine combinations can also preserve checks. Single-record uniformity therefore establishes neither security for several versions nor authentication. If every valid container necessarily contains a stored main wallet, disclosing a visibly derived opening would itself reveal that another opening exists. A deniable format would therefore also need to permit an honest container without an undisclosed stored main wallet.

XOR masks and nonlinear HMAC checks

A candidate for comparison derives separate XOR masks and HMAC keys after the expensive password KDF. In the simplest 12-word sketch, the raw block is $E_i \| U_i \| T_i$: 128 bits of wallet entropy, 48 bits of free padding and a 16-bit tag. A candidate check is

$$T_i = \mathrm{Trunc}_{16}(\text{HMAC-SHA-256}(V_i, \mathrm{domain} \| S \| E_i \| U_i))$$
$$Z = (E_i \| U_i \| T_i) \oplus \mathrm{Mask}_i$$

The existing wallet E_1 is held fixed. Its tag is computed for each trial padding, and the same record is tested under the other passwords. Once the keys are derived, this search uses HMAC rather than another Argon2 call per trial. For n total openings and t-bit tags, independent ideal check functions suggest $2^{(n-1)\cdot t}$ trials on average. With three openings, $t = 16$ and a 48-bit tail, this means about 2^{32} candidates per solution, and the tail admits an expected 2^{16} solutions. The estimate of trials per solution assumes that the tail admits many solutions. With four openings, the same tail admits one solution on average and none with probability about 37%. With five openings, it admits a solution with probability

of only about one in 65,536. Each candidate needs at least two HMAC evaluations; later checks can be skipped after an earlier failure. These estimates are model-dependent. They do not guarantee a completion time or prove a lower bound on every method.

In this original sketch, another wallet's entropy is determined by E_1 and the mask difference. Varying the padding does not let the owner select an arbitrary second existing wallet. Nor does the sketch automatically give a 24-word wallet, or a wallet recoverable from the password and salt alone. Adding the separate wallet-output derivation described above requires its own definition of roles, checking and decoding. HMAC's nonlinearity does not by itself prove greater security than affine coding. The search must avoid a publicly recognizable padding rule, and the resulting record distribution needs analysis. Reusing the same salt and password mask to encrypt different raw payloads exposes their XOR, so changes and retained versions need explicit rules.

A worked research sketch of the affine route

A design-and-attack exercise on 2026-10-05 developed the affine route into a detailed research sketch, called MHFE-MW here. Within the same exercise, three AI-assisted review passes attacked an earlier version, and a revision addressed their findings. The intended development is a separate program and publication, with its own format and analysis. It would not be integrated into MHFE or added as a suite of the current specification. This outline records the research. It is not an interoperable format definition or an expert-reviewed construction.

- **Layout.** The 256 bits hold 54 bits of public salt material S, a 10-bit tweak T and 192 bits Z solved over $\mathrm{GF}(2)$. The exact serialization and the salt bytes passed to Argon2 still need a complete definition.
- **Nominal recovery work.** The sketch derives a key through twelve sequential Argon2id calls with the suite 3 work parameters, using salts derived from S and prior outputs. Ordinary recovery therefore has the same nominal call count and profile as suite 3. This is not a lower bound on every password test or a claim of equal cryptographic security: suite 3's state-derived salts and its known-pair filter differ from this chain. One candidate chain can serve the cheap tests of all opening roles. With w independent passwords from the same uniform space, an attack seeking any one of them can gain about $\log_2(w)$ bits compared with one target: about 2.6 bits for six passwords and 3.6 for twelve. These are conditional multi-target estimates, not general security deductions. The first call has a public salt, so the trace-comparison procedure under Observers on the same computer gives about 8.2 bits of modeled work reduction, or 3.5 bits if that call uses Argon2i and the next uses Argon2id. Neither estimate bounds all side-channel attacks.
- **Salt and reuse.** Independent uniform 54-bit salt material has a birthday collision scale near 2^{27} containers. At a fixed password and settings, versions or containers that share S reuse the same expensive chain, and changing T alone does not restore separate KDF work. This reuse needs analysis alongside deliberate salt retention when derived wallets are preserved.
- **Rows and checks.** Password-derived, domain-separated material and T select equation rows and offsets. Each opening has a t_H-bit header that encodes its role and word count. If a wrong opening produces a uniform header in the model, the seven accepted codes give probability $7 \cdot 2_H^{-t}$ per trial. This is consistency checking, not authentication. For Q wrong trials the union bound is $Q \cdot 7 \cdot 2_H^{-t}$, capped at 1. Automatic searches need a stated budget and must not silently choose one of several survivors.

- **Check size and capacity.**
  - With $t_H = 32$, a modeled wrong opening passes about once in 614 million, or $2^{-29.2}$, seven times more often than a single 32-bit check. A million independent wrong trials give about 0.163% probability of a false match. A pass does not establish wallet identity. The capacity budget permits one stored 12-word wallet and one derived wallet, one stored 15-word wallet, or six derived wallets.
  - With $t_H = 16$, the per-trial estimate is about 1 in 9,400, and a thousand independent wrong trials give about 10.13% probability of a false match. The capacity budget permits one stored 12-word wallet and three derived wallets, one stored 15-word wallet and one derived wallet, or twelve derived wallets. The earlier review passes did not examine this later parameter choice.
  - The maxima are full-rank capacity limits with no free solution bits. A stored wallet of 18 to 24 words does not fit either layout. Both check sizes require a defined search limit and an independent wallet-identity reference when confirmation of the intended wallet is needed. Written references linked to hidden wallets may be external evidence; remembering them or guarding them separately has different operational costs.

- **Rank and version selection.** A uniform square binary matrix of this size has full rank with probability about 0.2888, so the capacity limits require retries. The sketch proposes to evaluate all 1,024 tweaks in eight classes of 128. It would select a usable class cyclically from a start class and then, within that class, select a full-rank tweak by a priority derived from the owner's dice. Creation would choose the start class from the dice; an update would start after the current class. A first-successful-tweak rule in a publicly fixed order can leak the layout: an almost-surely full-rank layout selects the first tweak, whereas a square system does so with probability about 0.2888, a difference near 0.711. The proposed class rule needs fully defined ordering, ties, exhaustion and class-reuse behavior. The claimed eight-version guarantee is a proof target, not established by this summary.
- **Covert-channel mitigation.** A dishonest encoder can choose a salt or solution so that a public function of the container reveals a password hash tag, which enables a cheap filter before Argon2. The proposed mitigation makes all random choices reproducible from at least 50 fresh, private rolls of the owner's fair six-sided dice, about 129.25 bits of input entropy. The canonical generator, salt-retention rules, equation ordering, priorities and solution sampling must all be fixed. A trusted implementation from an independent source must reproduce the entire container from the same inputs; checking the funding addresses with it also guards against output substitution. Matching results do not establish that either program retained no secrets or that a shared computer was trustworthy. Retained dice can help distinguish additional openings by reproducing a disclosed-only encoding, so their retention and disclosure need analysis. Erasing the record cannot undo a copy already made by compromised software.
- **Passphrase protection.** For new wallets, the profile of the intended separate application would require a strong BIP39 passphrase generated independently of the opening password before funding. Finding the mnemonic would then still leave that passphrase to guess. An existing wallet keeps its original BIP39 passphrase: adding or changing one creates a different wallet and requires migration. The header supplies an early opening-password filter, so protection need not equal a product of the two search spaces. In a full-space search containing w genuine openings and N_{wrong} wrong passwords, let $N_P = w + N_{\text{wrong}}$. Assuming equally sized passphrase spaces, a straightforward procedure is modeled by $N_P \cdot C_{\text{chain}} + (w + 7 \cdot N_{\text{wrong}} \,/\, 2^t_H) \cdot N_Q \cdot C_{\text{BIP39}}$: false survivors also require passphrase searches. This is a procedure estimate, not an optimal-attack bound. Separated search does not remove the passphrase's protection against disclosure of the mnemonic itself.
- **Disclosure and changes.** A password invented without preparation normally fails the header. Accepted accidental openings can be searched for, and credible wallet history still needs preparation. Demonstrating a custom MW opening identifies the use of the MW format, but a password alone does not prove the format or another hidden opening. A header match is not certain identification of a correct hidden password. Ordinary re-encoding that preserves chosen openings needs their constraints, derived within a trusted workflow; the passwords may be supplied one after another rather than all at once. Retaining derived constraints instead creates sensitive recovery data. If S changes, all wallets derived from it change, even if their passwords do not.

- **Partial loss.** With the password, settings and profile known, a derived wallet can be computed from S alone. Knowing container words 1 to 3 exposes 33 of its 54 bits, which leaves at most 2^{21} salt candidates for the described chain search. For a known stored 12-word role with a 32-bit header, four missing words wholly within Z, at known positions, leave about 16 candidates after the header and BIP39-checksum constraints in the ideal model. That estimate assumes that the role's 32 header equations, restricted to the 44 bits of the missing words, have rank 32. Full rank of the whole system does not ensure this. The estimate also treats the eight BIP39 checksum bits in the intact last word as an ideal-hash filter. It does not guarantee unique recovery. Once the role's password chain is derived, these candidate tests need no further Argon2 calls. In contrast, repairing a suite 3 container without an MHFE-REPAIR-1 card needs a full recovery for each remaining candidate. The first five words cover all of S. Losing every recoverable copy of that salt removes password-only recovery of derived wallets. Another verified backup can preserve them.
- **Status and prior art.** The earlier review exercise reported no attack on its modeled core but found serious operational issues, including the covert channel. No independent cryptographer has reviewed the revision. The exercise proposed a residual deniability estimate near 2^{-45} for 2^{80} BLAKE2b queries, excluding hidden-password guesses. The estimate assumes fresh erased dice, independent passwords, explained changes and an appropriate honest-owner comparison. That estimate and the eight-version claim remain unverified here: this document contains neither their complete game and generator nor their proof. Fifty rolls alone do not establish either bound, and the model must also cover rank selection, conditioned layouts and disclosures. The first steganographic file-system construction of Anderson, Needham and Shamir [66] stores files as password-selected XOR combinations, adds them by solving linear equations and permits partial disclosure. Linear OKVS [65] give related formal hiding definitions, not a proof of this wallet application. Aezeed [57] already stores salt and a password-checked seed using 24 words from the BIP39 list with a different encoding. A limited literature search does not establish novelty.

Both routes need a complete format, independent vectors, and analysis of password guessing, salt collisions and reuse, multiple disclosures, and records of funded wallets. The 64-bit salt and 16-bit checks of the first two sketches are capacity examples, not recommended security parameters. The worked sketch's sizes are not frozen either, and its stored salt material has only 54 bits of diversity. RFC 9106's allowance for a 64-bit salt under space constraints [4] concerns byte-string length, not the safety of this distribution or the new construction. Neither that comparison nor collision estimates establish its security. These alternatives are recorded to investigate the trade-offs. They do not change current MHFE recovery, and this document does not claim that a new construction is ready to protect funds.

Appendix: proofs of the guessing bounds

This appendix presents proofs of the three guessing bounds of Part I: Theorem 3 for one container, Theorem 4 for a known pair and Theorem 5 for a 24-word source with an unknown BIP39 passphrase, with or without the wallet check. All three are theorems in the random-oracle model of the security model, with explicit budgets for Argon2id calls, for BIP39 seed computations in Theorem 5 and for the cheaper functions. They concern suite 3, and they say nothing about the concrete primitives beyond the assumptions of A.1.

Theorems 1 and 2 are the deniability results of Part I. The lemmas here are Lemmas G1 to G5, K1 to K4 and W1 to W4, so that they are not confused with Lemmas 1 to 3 of Deniability, and the games are Γ_0 to Γ_2, Δ_0 to Δ_2 and Θ_0 to Θ_2.

Independent review is needed to assess the correctness of these proofs and, separately, the applicability of the model to the concrete primitives. The arguments use sequences of games and the fundamental lemma of game playing [49], [67]: two games that behave identically until an event Bad occurs give any event the same probability up to the probability of Bad.

A.1 Model

Fix suite 3, the memory level and the PIM; the round prefix $C_i = \mathrm{BE32}(\mathrm{MEM}) \| \mathrm{BE32}(\mathrm{PIM}) \| \mathrm{BE32}(i)$ is then a function of the round index i. Here $\oplus$ is XOR and $\|$ concatenation, written $\oplus$ and || elsewhere in this document; $1[\cdot]$ is 1 if its condition holds and 0 otherwise, and $\mathrm{Exp}[\cdot]$ is an expectation. Four independent, uniformly random functions stand for the primitives:

- $A : \mathrm{Pw} \times \{0,1\}^{128} \to \{0,1\}^{256}$, Argon2id with the fixed settings, of the normalized password encoding and the salt; Pw is the set of normalized password encodings;
- $B : \{0 \ldots 11\} \times \{0,1\}^{128} \to \{0,1\}^{128}$, the salt $\mathrm{Trunc}_{128}(\mathrm{BLAKE2b\text{-}256}(\mathrm{DS_SALT} \| C_i \| R))$;
- $H : \{0,1\}^{256} \times \{0 \ldots 11\} \times \{0,1\}^{128} \to \{0,1\}^{128}$, the mask $\mathrm{Trunc}_{128}(\mathrm{HMAC\text{-}SHA\text{-}256}(K, \mathrm{DS_MASK} \| C_i \| R))$;
- $G : \{0,1\}^{\mathrm{ENT}} \to \{0,1\}^{256}$, SHA-256 of the source entropy.

The assumptions are:

1. A is a uniform function of (P_{enc}, S) for the fixed settings. This contains the salt-separation hypothesis A1, which is not established for the real Argon2id.
2. The four functions are independent, although Argon2id uses BLAKE2b internally and HMAC uses SHA-256. Independence is an explicit idealization; it does not follow from domain separation or from the use of different input formats.
3. HMAC-SHA-256 restricted to 32-byte keys is a random function of key and message. HMAC with keys of a fixed length below $d - 1$ bits, where d is the block length of the hash (512 for SHA-256), is indifferentiable from a keyed random oracle when the compression function is ideal [68].
4. The owner's computations, including the creation check, the checks of other source lengths and the generation loop of the wallet-check profile, reveal nothing beyond what the game gives the adversary: no memory addresses, timings, call log or number of trials.
5. For Theorem 3 and its extensions with an unknown source, the source entropy is uniform and independent of the password and the functions. Theorem 4 instead fixes a known state X independent of the password and A, B, H; it does not require a uniform choice of X. Theorem 5 generates a 24-word source as the wallet-check profile prescribes, or uniformly for Corollary W6, and its BIP39 passphrase is independent of the password and of the functions. In all games the password distribution is independent of the functions.
6. For Corollary G10 only: public wallet data of the original seed phrase with the empty or a known BIP39 passphrase is idealized as a uniformly random function of the phrase, independent of the four functions above and of the password. This is an assumption about the derivation through PBKDF2-HMAC-SHA512, BIP32 and the address hashes, not a claim that their serialized outputs or tuples of dependent fields are uniform bit strings. Corollary G10 states the output model.
7. For Theorem 5 only: the BIP39 seed of a phrase and passphrase, the wallet-check digest of a seed and the wallet data derived from a seed are idealized as uniformly random functions S, T and Ω, independent of each other, of the four functions above and of both secrets. A.6 defines them; the caveats of assumption 6 apply to Ω.

The adversary knows the suite, the settings and the source length ENT. Knowing ENT only strengthens it: SHA-256 at the other entropy lengths is a separate, unprogrammed part of G that it can simulate itself, and if the password distribution depends on ENT, the theorems hold with that distribution conditioned on ENT. Its budgets are k_A queries

to A and q_B, q_H, q_G queries to B, H, G, and in Theorem 5 also k_S, q_T and q_Ω queries to S, T and Ω. Without loss of generality, it repeats no query. A is the expensive function, and in Theorem 5 S is a second one, far cheaper than A; the others are cheap but counted.

The passwords are numbered $w_1, w_2, \ldots$ so that $p_1 \geq p_2 \geq \ldots$, where $p_i = \Pr[P = w_i]$, and $\mathrm{top}(m) = p_1 + \ldots + p_m$. For a set C of passwords chosen independently of P, $\Pr[P \in C] = \mathrm{Exp}[\sum_{w \in C} \Pr[P = w]] \leq \mathrm{Exp}[\mathrm{top}(|C|)]$.

Packing is $\mathrm{pack}(E) = E \| \mathrm{Trunc}_r(G(E))$ with $r = 256 - \mathrm{ENT}$. For a password p and a state $W_i = (L_i, R_i)$ of two 128-bit halves, round i is

$$\begin{aligned} S_i &= B(i, R_i), & K_i &= A(p, S_i), \\ M_i &= H(K_i, i, R_i), & W_{i+1} &= (R_i, L_i \oplus M_i). \end{aligned}$$

and $\mathrm{Perm}_p(X) = W_{12}$ for $W_0 = X$. The inverse of round i sets $R_i = L_{i+1}$, recomputes S_i, K_i, M_i and sets $L_i = R_{i+1} \oplus M_i$; Inv_p is the inverse permutation Perm_p^{-1}. Perm_p depends on A, B, H only, not on G, and for fixed A, B, H it is a permutation of $\{0,1\}^{256}$. For every ENT, L_0 is the first 128 bits of E. The H points of the twelve rounds carry the twelve different round indices, so the twelve masks of one evaluation are values of H at twelve different points.

A.2 Theorem 3: one container

Game Γ_0. P is drawn from the password distribution, $E \leftarrow \{0,1\}^{\mathrm{ENT}}$, $X = \mathrm{pack}(E)$ and $Y = \mathrm{Perm}_P(X)$, for a short source, $\mathrm{ENT} \in \{128, 160, 192, 224\}$. The adversary receives Y, uses the four functions within its budgets and outputs p^*. It wins if $p^* = P$.

Theorem 3. For every adversary in Γ_0,

$$\begin{aligned} \Pr[\mathrm{win}] &\leq \mathrm{top}\left(\left\lfloor \frac{k_A}{12} \right\rfloor + 1\right) + \varepsilon, \\ \varepsilon &= \frac{q_G + q_B + 12 p_1 k_A + 66}{2^{128}} + \frac{q_H}{2^{256}}. \end{aligned}$$

An adversary that has made k_A Argon2id calls succeeds no better than one that tests the $\lfloor k_A \,/\, 12 \rfloor$ most likely passwords by full recovery without false verifier passes and guesses one more; the difference is at most ε. Partial work, such as screening R_0 after eleven calls, gains at most ε in total. For $k_A < 12$ the bound is $p_1 + \varepsilon$.

The bound improves on the trivial bound 1 when $\mathrm{top}(\lfloor k_A \,/\, 12 \rfloor + 1) + \varepsilon < 1$. It shows that cheap work changes little against plain guessing when $\varepsilon \ll \mathrm{top}(\lfloor k_A \,/\, 12 \rfloor + 1)$. If the constant $66 \,/\, 2^{128}$ is negligible compared with the main term, so are the terms in k_A and q_H for feasible k_A and q_H. The second condition then reads $q_G + q_B \ll 2^{128} \cdot \mathrm{top}(\lfloor k_A \,/\, 12 \rfloor + 1)$. A password of five EFF words has probability about $2^{-64.6}$, and for such passwords the second condition holds while $q_G + q_B$ stays well below 2^{64} times the number of tested passwords. Beyond that, the bound remains valid but no longer shows that cheap work is useless.

No attack is known that gains from the q_B term. Lemma G3 charges this term whenever a fresh half of the chain hits an earlier B query with the same round index; such a hit yields a salt, not a key.

Proposition G8 gives an attack that spends twelve calls per tested password and falls short of the main term only through false verifier passes, so the bound matches that strategy up to the stated terms. The theorem bounds the probability of success for given budgets. It does not forbid partial filters, whose total effect is inside ε. Corollary G10 extends the bound to an attacker who also checks candidates against wallet data, such as a known address, with the empty or a known BIP39 passphrase, for every source length.

A.3 Proof of Theorem 3

Re-sampling the container. In Game Γ_1, P is drawn as before, $Y \leftarrow \{0,1\}^{256}$, the functions A, B, H, G' are uniform, $x^* = \mathrm{Inv}_P(Y)$, $e^* = x^*[0 : \mathrm{ENT}]$ and $v^* = x^*[\mathrm{ENT} : 256]$. G equals G', except that the first r output bits of $G(e^*)$ are set to v^*; its other $256 - r$ output bits at e^* are those of G'. Γ_0 and Γ_1 use eagerly sampled functions, so the computations of the challenger in them are part of the functions and need no separate account.

Lemma G1. (P, A, B, H, Y, G) has the same joint distribution in Γ_0 and Γ_1.

Proof. The marginal distribution of (P, A, B, H) is the same in both games, and in each it is sampled independently of the other inputs: E and G in Γ_0, Y and G' in Γ_1. Condition on $P = p$ and on A, B, H. Then Perm_p is a fixed permutation. For $y \in \{0,1\}^{256}$ write $\mathrm{Inv}_p(y) = e_y \| v_y$ with $|e_y| = \mathrm{ENT}$. For a function g:

- in Γ_0, $\Pr[Y = y, G = g] = \sum_e 2^{-\mathrm{ENT}} \cdot \Pr[G = g] \cdot 1[\mathrm{Perm}_p(e \| \mathrm{Trunc}_r g(e)) = y]$. Since Perm_p is a bijection, the bracket is 1 exactly when $e = e_y$ and $\mathrm{Trunc}_r g(e_y) = v_y$, so the sum is $2^{-\mathrm{ENT}} \cdot \Pr[G = g] \cdot 1[\mathrm{Trunc}_r g(e_y) = v_y]$;
- in Γ_1, $\Pr[Y = y, G = g] = 2^{-256} \cdot \Pr[G' \text{ agrees with } g \text{ outside the first } r \text{ bits at } e_y] \cdot 1[\mathrm{Trunc}_r g(e_y) = v_y]$, and the middle factor is $2^r \cdot \Pr[G = g]$, since those r bits of G' are free.

Both are $2^{-\mathrm{ENT}} \cdot \Pr[G = g] \cdot 1[\mathrm{Trunc}_r g(e_y) = v_y]$, because $\mathrm{ENT} + r = 256$. The adversary's run is a function of (Y, A, B, H, G) and its own coins, so every event of Γ_0 has the same probability in Γ_1.

So Y is uniform and independent of (P, A, B, H). It is not independent of G, which carries the one relation $\mathrm{Trunc}_r G(e^*) = v^*$.

Removing the programmed point. Game Γ_2 is Γ_1 with $G = G'$, that is, without the programmed point. **Bad** is the event that the adversary queries G at e^*. Γ_1 and Γ_2 differ only in the first r bits of G at e^*, and e^* and the winning condition are defined identically in both. So the two games are identical until Bad: $\Pr_{\Gamma_1}[\mathrm{win} \wedge \neg \mathrm{Bad}] = \Pr_{\Gamma_2}[\mathrm{win} \wedge \neg \mathrm{Bad}]$ and $\Pr_{\Gamma_1}[\mathrm{Bad}] = \Pr_{\Gamma_2}[\mathrm{Bad}]$. Hence

$$\Pr_{\Gamma_0}[\mathrm{win}] = \Pr_{\Gamma_1}[\mathrm{win}] \le \Pr_{\Gamma_2}[\mathrm{win} \vee \mathrm{Bad}]. \tag{1}$$

A lazy implementation of Γ_2. In Γ_2, $G = G'$ does not depend on e^*, so nothing the adversary sees depends on x^*. The following implementation gives the same joint distribution to P, Y, the adversary's run and the function values at the points the adversary queries and at the chain of P below, and so the same probability to every event determined by them, among them win, Bad and the event of Corollary G6. Γ_2 can therefore be run as follows:

1. draw P and $Y \leftarrow \{0,1\}^{256}$; P is not used during the run;
2. run the adversary with tables T_A, T_B, T_H, T_G that start empty; a query at a new point receives a fresh uniform answer, which is recorded, and a repeated point returns the recorded answer; the game evaluates no point itself;

3. after the adversary halts, a **finalization**, which the adversary does not see, evaluates the chain of P point by point with the same tables, sampling each point not yet in its table, and obtains x^*, e^* and Bad from the recorded G queries Q_G.

The chain of P. With L_{12} and R_{12} the halves of Y, the chain is the sequence of 36 points of the inverse computation

$$\begin{aligned} c_1 &= (11, R_{11}), \quad c_2 = (P, S_{11}), \qquad c_3 = (K_{11}, 11, R_{11}),\\ c_4 &= (10, R_{10}), \quad \dots \\ c_{35} &= (P, S_0), \qquad c_{36} = (K_0, 0, R_0). \end{aligned}$$

of types B, A, H repeated, where $R_{11} = L_{12}$ and, for $i \le 10$, $R_i = L_{i+1} = R_{i+2} \oplus M_{i+1}$. c_1 is fixed by Y, and c_{t+1} is a function of Y, P and the answers at $c_1 \dots c_t$. Then $\mathrm{Inv}_P(Y) = (L_0, R_0)$ with $L_0 = R_1 \oplus M_0$, and e^* begins with the 128 bits L_0.

The oracle type is part of a point's identity: points in different tables are distinct even if their input encodings coincide.

Defined. c_1 is defined from the start. For $t \ge 1$, c_{t+1} becomes defined at the first moment at which c_t is defined and the answer at c_t has been sampled.

Complete chain. $\mathrm{Comp}(P)$ holds if the adversary itself queried all twelve A points $c_2, c_5, \dots, c_{35}$ of the chain.

Early_P**.** For some $t \in \{2, \dots, 36\}$, c_t is already in its table at the moment it becomes defined. Early_t denotes this event at step t.

Lemma G2 (order). On $\neg\mathrm{Early}_P$, for every $t \ge 2$: if the adversary queried c_t, its first query of c_{t-1} came before its first query of c_t. Hence, if the adversary did not query some chain point, it queried no later chain point.

Proof. On $\neg\mathrm{Early}_P$, every c_s with $s \ge 2$ is outside its table when it becomes defined, so its answer is sampled at its first query after that moment; c_1 is defined from the start. The answer at c_{t-1} is sampled at its first query, made either by the adversary or by finalization. At that moment c_t becomes defined and, by $\neg\mathrm{Early}_t$, is not yet in its table, so any query of c_t by the adversary comes later. If the first query of c_{t-1} is made by finalization, it comes after the adversary halted, so the adversary never queried c_t.

Lemma G2 is the sequential-chain argument of time-lock puzzles and proofs of sequential work [69], [70], here for a chain whose points depend on the Feistel state rather than on an encoded step.

Lemma G3. $\Pr_{\Gamma_2}[\mathrm{Early}_P] \le (12 \cdot p_1 \cdot k_A + 66 + q_B) \,/\, 2^{128} + q_H \,/\, 2^{256}$.

Proof. Write $\neg\mathrm{Early}_{<t}$ for the event that no Early_s with $s < t$ holds. Then $\Pr[\mathrm{Early}_P] = \sum_t \Pr[\mathrm{Early}_t \wedge \neg\mathrm{Early}_{<t}]$. Every chain point is eventually sampled, by the adversary or by finalization. On $\neg\mathrm{Early}_{<t}$ the points $c_1 \dots c_{t-1}$ were sampled in chain order, each after it became defined, so c_t becomes defined at the moment τ_t at which the answer at c_{t-1} is sampled fresh. At τ_t the tables hold the adversary's queries made so far and, if τ_t falls in finalization, the chain points among $c_1 \dots c_{t-1}$ that finalization sampled. Finalization queries no other point, so it adds no dependence beyond these values.

Let n_t be the number of points in the table of c_t's type that agree with c_t in every component except the one set by the sample at c_{t-1}, counted just before that sample ($n_t = 0$ outside $\neg\mathrm{Early}_{<t}$). The table of c_t's type differs from that of c_{t-1} (B to A, A to H, H to B), so the sample does not change it. $\neg\mathrm{Early}_{<t}$, n_t and the other components of c_t are

determined by the history before the sample. The sample is uniform given that history, whatever the adversary did before. Once the other components are fixed, c_t is an injective function of the sample. Hence

$$\begin{aligned}\Pr[\mathrm{Early}_t \wedge \neg\mathrm{Early}_{<t}] &= \mathrm{Exp}[\mathbf{1}[\neg\mathrm{Early}_{<t}] \cdot \Pr[c_t \in \mathrm{table} \mid \mathrm{history}]] \\ &\le \frac{\mathrm{Exp}[n_t]}{V_t}.\end{aligned}$$

where V_t is the number of values of the sample: 2^{128} for a salt or a mask, 2^{256} for a key.

By the type of c_{t-1}:

- **A B point** (i, R_i), sample $S_i \in \{0,1\}^{128}$, next $c_t = (P, S_i)$. The matching points are the A points with password P: at most N_P^A adversary points, where N_P^A is the number of the adversary's A queries with password P, and, if τ_t falls in finalization, the $11 - i$ points of rounds $11 \ldots i+1$ of this chain. In Γ_2 the run is independent of P, so $\mathrm{Exp}[N_P^A] = \sum_w \Pr[P = w] \cdot \mathrm{Exp}[N_w^A] \le p_1 \cdot k_A$. Summing over $i = 0 \ldots 11$ gives $(12 \cdot p_1 \cdot k_A + 66) \,/\, 2^{128}$.
- **An A point**, sample $K_i \in \{0,1\}^{256}$, next $c_t = (K_i, i, R_i)$. Only H points with round index i can match. The chain's other H points carry other indices, so at most N_i^H adversary points can, where N_i^H is the number of the adversary's H queries with index i. Summing $\mathrm{Exp}[N_i^H] \,/\, 2^{256}$ over i gives at most $q_H \,/\, 2^{256}$, since each query has one index.
- **An H point** with $i \ge 1$, sample $M_i \in \{0,1\}^{128}$, next $c_t = (i-1, R_{i+1} \oplus M_i)$ with a uniform second component. The chain's B points sampled so far carry indices $\ge i$, so only the adversary's N_{i-1}^B B queries with index $i-1$ can match. Summing $\mathrm{Exp}[N_{i-1}^B] \,/\, 2^{128}$ over i gives at most $q_B \,/\, 2^{128}$.

Counts taken at τ_t are at most the final counts, and expectations add, so the bound holds for adaptive adversaries without enumerating strategies.

An $A(P, S)$ query carries no round index, so its salt can match a fresh salt in any of the twelve rounds. Each B or H query carries one round index, so those matching counts are partitioned by round. This explains the factor 12 on the adversary's A count, and the factor 24 for the two chains in Lemma K3.

Lemma G4. $\Pr_{\Gamma_2}[\mathrm{Bad} \wedge \neg\mathrm{Comp}(P) \wedge \neg\mathrm{Early}_P] \le q_G \,/\, 2^{128}$.

Proof. On $\neg\mathrm{Early}_P \wedge \neg\mathrm{Comp}(P)$, some A point of the chain was not queried by the adversary, so by Lemma G2 the adversary did not query $c_{36} = (K_0, 0, R_0)$. The chain's other H points carry other round indices, so c_{36} is not in T_H when finalization reaches it. Let N be this event, "c_{36} is not in T_H when finalization reaches it". Then M_0 is sampled at that moment τ, after the adversary halted. N, R_1 and Q_G are determined by the history before τ, and M_0 is uniform given it. Bad implies that $L_0 = R_1 \oplus M_0$ equals the first 128 bits of some point of Q_G, at most q_G values, so $\Pr[\mathrm{Bad} \wedge N] \le q_G \,/\, 2^{128}$; and $\neg\mathrm{Early}_P \wedge \neg\mathrm{Comp}(P) \subseteq N$.

Lemma G5. On $\neg\mathrm{Early}_P$ the 36 points of the chain are pairwise distinct. In particular, on $\mathrm{Comp}(P) \wedge \neg\mathrm{Early}_P$ the adversary made at least twelve A queries with password P.

Proof. Suppose $c_s = c_t$ with $s < t$; then $s < t - 1$, because consecutive points have different types. By the definition of "defined", when c_t becomes defined every c_u with $u < t$ has been defined and sampled, c_s included, so c_t is already in its table: this is Early_t.

Lemma G5 is the step that excludes repeated chain values, which an earlier analysis of iterated key derivation [71] assumed away, as Bellare, Ristenpart and Tessaro point out [47, App. B]. It concerns the real chain only. The chain of another password can repeat a salt and be completed with fewer than twelve A queries, with probability about $66 \cdot 2^{-128}$ per password tried. This does not enter the bound, because the set below is independent of P and a repeat in the real chain is Early_P.

Assembly. Let C' be the set of passwords with which the adversary made at least twelve A queries. Then $|C'| \le \lfloor k_A \,/\, 12 \rfloor$, and C' and p^* are functions of the adversary's run, which in Γ_2 is independent of P. Every outcome of $\mathrm{win} \vee \mathrm{Bad}$ lies in one of three sets:

- $\{P \in C' \cup \{p^*\}\}$: a win gives $P = p^*$, and $\mathrm{Bad} \wedge \mathrm{Comp}(P) \wedge \neg\mathrm{Early}_P$ gives $P \in C'$ by Lemma G5; $\Pr_{\Gamma_2}[P \in C' \cup \{p^*\}] \le \mathrm{top}(\lfloor k_A \,/\, 12 \rfloor + 1)$;
- Early_P: Lemma G3;
- $\mathrm{Bad} \wedge \neg\mathrm{Comp}(P) \wedge \neg\mathrm{Early}_P$: Lemma G4.

With (1) this gives Theorem 3.

The terms of ε come from these events:

Term	Event
$q_G \,/\, 2^{128}$	L_0 of the real chain lands in the first 128 bits of a G query (G4)
$12 \cdot p_1 \cdot k_A \,/\, 2^{128}$	a fresh salt of the real chain hits an A query with password P (G3)
$66 \,/\, 2^{128}$	a fresh salt hits an earlier A point of the same chain (G3)
$q_B \,/\, 2^{128}$	a fresh half hits a B query with the same round index (G3)
$q_H \,/\, 2^{256}$	a fresh key hits an H query with the same round index (G3)

Nothing else enters: the game makes no calls during the run, and finalization evaluates only the 36 points of the real chain, whose own coincidences are the $66 \,/\, 2^{128}$ term.

The ciphertext-only analysis describes a screen that asks, after eleven calls, whether R_0 has a SHA-256 preimage. This screen needs G queries at the unknown L_0 and is charged by the first term in the table. For $\mathrm{ENT} > 128$ the screen asks whether some L_0 of the 2^{128} values satisfies the r-bit verifier together with the known source bits in R_0. For a fixed R_0, no value fits with probability $(1 - 2^{-r})^{2^{128}}$, about $\exp(-2^{\mathrm{ENT}-128})$, so the screen passes almost every candidate.

A.4 Consequences

Corollary G6 (source recovery). If the adversary outputs e_{out} and wins when $e_{\mathrm{out}} = E$,

$$\Pr[\mathrm{win}] \le \mathrm{top}\left(\left\lfloor \frac{k_A}{12} \right\rfloor\right) + \frac{q_G + 1 + q_B + 12 p_1 k_A + 66}{2^{128}} + \frac{q_H}{2^{256}}.$$

Proof. e^* is the same function of (P, A, B, H, Y) as E in Γ_0, so by Lemma G1 $\Pr_{\Gamma_0}[e_{\text{out}} = E] = \Pr_{\Gamma_1}[e_{\text{out}} = e^*] \le \Pr_{\Gamma_2}[e_{\text{out}} = e^* \vee \mathrm{Bad}]$. Treat e_{out} as one more G point: Lemma G4 holds with $Q_G \cup \{e_{\text{out}}\}$, and the assembly holds without p^*, since a password guess without a complete chain yields no E.

Corollary G7 (depth). Let the adversary submit its queries in batches, each answered only after the whole batch is submitted. Let Early_t also hold when c_t becomes defined while a query of it is in the current batch. A batch is fixed before any of its answers is sampled, so each sample is still uniform given a history that includes the batch. Early_P defined this way contains the sequential one, and the counts of matching points are unchanged. Lemma G2 then holds with "earlier" read as "in a strictly earlier batch", Lemma G3 with the same bound, and Lemma G4 unchanged, since finalization follows the run. On $\mathrm{Comp}(P) \wedge \neg\mathrm{Early}_P$ the twelve A points of the chain therefore lie in twelve batches in chain order. If the adversary's A queries occupy at most eleven batches, of any width, $\mathrm{Comp}(P)$ implies Early_P, so $\mathrm{win} \vee \mathrm{Bad} \subseteq \{p^* = P\} \cup \mathrm{Early}_P \cup (\mathrm{Bad} \wedge \neg\mathrm{Comp}(P) \wedge \neg\mathrm{Early}_P)$ and

$$\Pr[\mathrm{win}] \le p_1 + \frac{q_G + q_B + 12 p_1 k_A + 66}{2^{128}} + \frac{q_H}{2^{256}}.$$

k_A counts every parallel A query, so the corollary says something only while k_A, q_B and q_G stay far below 2^{128}. It concerns the real password and does not bound how many other chains are completed in parallel.

Proposition G8 (matching attack). For a password w write $x(w) = \mathrm{Inv}_w(Y)$, $e(w) = x(w)[0 : \mathrm{ENT}]$ and $v(w) = x(w)[\mathrm{ENT} : 256]$; w passes if $\mathrm{Trunc}_r G(e(w)) = v(w)$. Let $m \ge 0$. The attack inverts Y under $w_1 \dots w_m$, tests each result with one G query, outputs the first password that passes, and otherwise outputs w_{m+1}. Its budget is $k_A \le 12m$, $q_B \le 11m + 1$ (the first B point $(11, L_{12})$ is shared by every password), $q_H \le 12m$ and $q_G = m$. With $\delta = 2^{-r} + 2^{-128} + 12 \cdot 2^{-256}$, its success probability is at least

$$\sum\nolimits_{j \le m+1} p_j \cdot (1 - (j-1) \cdot \delta) \ge \mathrm{top}(m+1) \cdot (1 - m \cdot \delta).$$

Proof. Work in Γ_1 (Lemma G1), where P is independent of (Y, A, B, H, G') and $e(w)$, $v(w)$ are functions of (Y, A, B, H). G itself depends on P through the programmed point. The real P always passes, because G is programmed at $e(P)$. So the attack succeeds on $P = w_j$, $j \le m+1$, unless some w_i with $i < j$ passes, and its success probability is at least $\sum_j (p_j - \sum_{i<j} \Pr[P = w_j \wedge w_i \text{ passes }])$. Condition on $P = w_j$; for $i \ne j$:

- $\Pr[P = w_j \wedge e(w_i) \ne e(w_j) \wedge w_i \text{ passes }] = p_j \cdot \Pr[e(w_i) \ne e(w_j)] \cdot 2^{-r} \le p_j \cdot 2^{-r}$: given $P = w_j$ and $e(w_i) \ne e(w_j)$, $G(e(w_i)) = G'(e(w_i))$ is uniform and independent of $v(w_i)$ and of that event;
- $\Pr[P = w_j \wedge e(w_i) = e(w_j)] = p_j \cdot \Pr[e(w_i) = e(w_j)] \le p_j \cdot (2^{-128} + 12 \cdot 2^{-256})$. Sample the chain of w_j first and then that of w_i. Each of the at most twelve distinct A points of the chain of w_i carries password w_i and is fresh, so $K_0(w_i) = K_0(w_j)$ has probability at most $12 \cdot 2^{-256}$. Otherwise the two index-0 H points differ, $M_0(w_i)$ is fresh after $R_1(w_i)$ is fixed, and $L_0(w_i) = L_0(w_j)$ has probability 2^{-128}.

Each term is at most $p_j \cdot \delta$, which gives the bound.

Together with Theorem 3 for $k_A = 12m$, the success of the best adversary lies between $\mathrm{top}(m+1)(1 - m \cdot \delta)$ and $\mathrm{top}(m+1) + \varepsilon$. For $r = 32$ (21-word sources) the relative loss $m \cdot 2^{-32}$ matters only as m approaches 2^{32} tested passwords.

Remark G9 (creation refusal). Creation refuses $Y = X$ and asks for a different password, PIM or memory level. For every fixed x, password w and functions A, B, the twelve H points of the forward path from x carry distinct round indices, so the path's twelve masks are independent and uniform. (M_0, M_1) maps to W_2 bijectively, and for fixed later masks the later rounds map W_2 to W_{12} bijectively. So W_{12} is uniform and, over the choice of H, $\Pr[\mathrm{Perm}_w(x) = x] = 2^{-256}$ exactly.

Couple each game with refusal and the same game without refusal (Γ_0, Δ_0 with the same fixed X, or the game of Corollary G10) on the same password, source and functions. The two games coincide unless the first attempt is refused, whatever the owner does after a refusal. So Theorems 3 and 4 and Corollaries G6, G7 and G10 hold with refusal after adding 2^{-256} to their bounds.

Corollary G10 (wallet data). Let $\Omega : \{0,1\}^{\mathrm{ENT}} \to \{0,1\}^{\lambda}$ be a further uniformly random function, independent of A, B, H, G and the password. It idealizes the data used to check a wallet derived from the original seed phrase with the empty or a known BIP39 passphrase, for example a receiving address or a master key fingerprint (assumption 6). Here λ is the width of the ideal uniform output, not the length of a serialized wallet record. A tuple is covered by this model only if its joint output satisfies the same uniformity and independence assumptions. The upper bound does not depend on λ, but still requires this output model. If a known passphrase depends on the password, the p_i are the probabilities given that passphrase. Let the adversary in Game Γ_0 also receive $\Omega(E)$ and make q_Ω queries to Ω, and allow every suite 3 source length, $\mathrm{ENT} \in \{128, 160, 192, 224, 256\}$; for $\mathrm{ENT} = 256$ there is no verifier and G does not enter the container. Then

$$\Pr[\mathrm{win}] \le \mathrm{top}\left(\left\lfloor \frac{k_A}{12} \right\rfloor + 1\right) + \frac{q_G + q_\Omega + q_B + 12 p_1 k_A + 66}{2^{128}} + \frac{q_H}{2^{256}}.$$

Proof. In Game Γ_1 also draw $\omega^* \leftarrow \{0,1\}^{\lambda}$, give it to the adversary, and set Ω equal to a uniform Ω' except $\Omega(e^*) = \omega^*$. In Γ_0 the joint probability of Lemma G1 gains the factor $\Pr[\Omega = f] \cdot 1[f(e_y) = \omega]$. In Γ_1 it gains $\Pr[\Omega' \text{ agrees with } f \text{ outside } e_y] \cdot 2^{-\lambda} \cdot 1[f(e_y) = \omega] = \Pr[\Omega = f] \cdot 1[f(e_y) = \omega]$, the same factor, so $(P, A, B, H, Y, G, \Omega, \Omega(E))$ has the same distribution in Γ_0 and Γ_1. In Γ_2 neither G nor Ω is programmed, ω^* is uniform and independent of P, and Bad is a query of G or Ω at e^*. Lemmas G2, G3 and G5 are unchanged. In Lemma G4 the first 128 bits of an Ω query must equal L_0, which adds q_Ω / 2^{128}. For $\mathrm{ENT} = 256$, $\mathrm{pack}(E) = E$ and the same steps apply with Ω alone.

A real derivation reads the original phrase, whose last word holds the first $\mathrm{ENT} / 32$ bits of $G(E)$. The same proof covers $\Omega(e) = D(\mathrm{phrase}(e, \mathrm{Trunc}_{\mathrm{ENT}/32} G(e)))$ for a random function D independent of the others, with D programmed at the phrase of e^*, Bad including a D query at any phrase whose entropy is e^*, and q_Ω counting D queries. The coupling of Remark G9 applies unchanged.

For every source length the bound matches the twelve-call strategy up to the error terms and false-match losses in this model. Proposition G8 holds for the attack that tests each inverted source against the verifier, if any, and against Ω, with $q_\Omega = m$ and $\delta = 2^{-(r+\lambda)} + 2^{-128} + 12 \cdot 2^{-256}$. This holds because, in Γ_1 with Ω also programmed at e^*, the values of G' and Ω' at a point other than e^* are uniform and independent of the values compared with them; the factor $2^{-(r+\lambda)}$ uses this joint uniformity.

Encoding prefixes, checksums and dependent fields do not add independent bits to λ: two copies of one 32-bit fingerprint still supply one 32-bit check, not a 64-bit check. Applying the matching-attack calculation to structured wallet records requires a justified joint false-match probability and a model of their derivation; summing field lengths is insufficient.

Without wallet data, for $\mathrm{ENT} = 256$, Γ_0 already equals Γ_2: the container alone gives no information about the password, and the adversary wins with probability at most p_1. Wallet data that depends on an unknown BIP39 passphrase is not a function of E alone and is not covered here; Theorem 5 treats it for 24-word sources.

A.5 Theorem 4: a known pair

Game Δ_0. Fix $X \in \{0,1\}^{256}$, chosen independently of the functions and of the password, for example the packed state of a known source. Packing uses SHA-256, which is independent of A, B, H. P is drawn and $Y = \mathrm{Perm}_P(X)$. The adversary receives (X, Y), uses A, B, H within its budgets and outputs p^*. It wins if $p^* = P$. G plays no role.

Fractional top. For real $x \geq 0$, $T(x) = \mathrm{top}(\lfloor x \rfloor) + (x - \lfloor x \rfloor) \cdot p_{\lfloor x \rfloor + 1}$. It is the largest value of $\sum_w \Pr[P = w] \cdot u_w$ over weights $0 \leq u_w \leq 1$ with $\sum_w u_w \leq x$, since the optimum fills the weights greedily in order of decreasing probability. It satisfies $\mathrm{top}(\lfloor x \rfloor) \leq T(x) \leq \mathrm{top}(\lceil x \rceil)$.

Theorem 4. For every adversary in Δ_0, for every source length of suite 3,

$$\Pr[\mathrm{win}] \leq T\left(1 + \frac{k_A}{11}\right) + \varepsilon_K \leq \mathrm{top}\left(\left\lfloor \frac{k_A}{11} \right\rfloor + 2\right) + \varepsilon_K,$$
$$\varepsilon_K = \frac{24 p_1 k_A + 2 q_B + 419}{2^{128}} + \frac{2 q_H + 12}{2^{256}}.$$

With a known pair, an adversary that has made k_A Argon2id calls succeeds no better than one that tests the $\lfloor k_A \,/\, 11 \rfloor$ most likely passwords exactly, guesses one more, and gets credit $(k_A \bmod 11) \,/\, 11$ for one partly tested password, up to ε_K. The fractional credit comes from the averaging below; no known attack uses a partial chain. The filter of Observation 2 uses eleven calls, saving one round. The attack of Proposition K5 succeeds with probability within a relative loss of about $\lfloor k_A \,/\, 11 \rfloor \cdot 2^{-128}$ of $\mathrm{top}(\lfloor k_A \,/\, 11 \rfloor + 1)$. So the bound matches the eleven-call strategy up to the fractional credit for one password and the small terms.

Programming two adjacent masks. Fix a round $j \in \{0, \ldots, 10\}$. In Game $\Delta_{1,j}$, P is drawn, A, B, H' are uniform and $Y \leftarrow \{0,1\}^{256}$. Using A, B, H':

- evaluate rounds $0 \ldots j-1$ of Perm_P forward from X, giving $W_j = (L_j, R_j)$ (for $j = 0$ nothing is evaluated and $W_0 = X$), and set $h_j = (A(P, B(j, R_j)), j, R_j)$;
- evaluate rounds $11 \ldots j+2$ backward from Y, giving $W_{j+2} = (L_{j+2}, R_{j+2})$ (for $j = 10$ nothing is evaluated and $W_{12} = Y$), set $R_{j+1} = L_{j+2}$ and $h_{j+1} = (A(P, B(j+1, R_{j+1})), j+1, R_{j+1})$.

H equals H' except $H(h_j) = L_{j+2} \oplus L_j$ and $H(h_{j+1}) = R_{j+2} \oplus R_j$. The two points differ, since their round indices differ. The forward rounds $0 \ldots j-1$ use H only at indices below j, and the backward rounds $j+2 \ldots 11$ only at indices above $j+1$. The A and B points of rounds j and $j+1$ are not programmed. So, given P, A and B, the two points and the two required values are functions of Y and of H at round indices other than j and $j+1$, and these evaluations are the same under H and H'.

Lemma K1. For every j, (P, A, B, H, Y) has the same joint distribution in Δ_0 and $\Delta_{1,j}$.

Proof. The marginal distribution of (P, A, B) is the same in both games. In each game the triple is sampled independently of the other inputs: H in Δ_0, and H' and Y in $\Delta_{1,j}$. Condition on $P = p$ and on A, B. Fix a function h and $y \in \{0,1\}^{256}$, and write Perm_p^h for Perm_p computed with $H = h$. Compute the two points and the required

values from y and from h at the other indices. If H' agrees with h outside these two points, it agrees with h at every other index, so it yields the same points and values.

- If $\mathrm{Perm}_p^h(X) = y$, the path passes through W_j and W_{j+2}. Round j gives $W_{j+1} = (R_j, L_j \oplus M_j)$ and round $j+1$ gives $W_{j+2} = (L_j \oplus M_j, R_j \oplus M_{j+1})$. So $M_j = L_{j+2} \oplus L_j$ and $M_{j+1} = R_{j+2} \oplus R_j$; that is, h takes the required values at the two points.
- Conversely, with these two values, round j maps W_j to (R_j, L_{j+2}). Its right half is $R_{j+1} = L_{j+2}$, so round $j+1$ uses the point h_{j+1} and maps to $(L_{j+2}, R_{j+2}) = W_{j+2}$. The backward rounds then lead to y.

So $1[\mathrm{Perm}_p^h(X) = y]$ equals the indicator that h takes the required values at the two points. In Δ_0, $\Pr[H = h, Y = y] = \Pr[H = h] \cdot 1[\mathrm{Perm}_p^h(X) = y]$. In $\Delta_{1,j}$, $\Pr[H = h, Y = y] = 2^{-256} \cdot \Pr[H' \text{ agrees with } h \text{ outside the two points }] \cdot 1[h \text{ takes the required values }]$, and the middle factor is $2^{256} \cdot \Pr[H = h]$, since two 128-bit outputs are free. The two expressions are equal. The adversary's run is a function of (X, Y, A, B, H) and its coins.

Lemma K1 is an exact single-path form of the fact that two Feistel rounds with fresh round functions make an output uniform [30], [72]. Lemma G1 is its one-point analogue.

Averaging over the programmed round. Game $\Delta_{2,j}$ is $\Delta_{1,j}$ with $H = H'$. Bad_j is the event that the adversary queries H at h_j or at h_{j+1}. $\Delta_{1,j}$ and $\Delta_{2,j}$ differ only at those two points, which are defined identically in both games. So the games are identical until Bad_j, and $\Pr_{\Delta_{1,j}}[\mathrm{win}] \le \Pr_{\Delta_{2,j}}[\mathrm{win} \vee \mathrm{Bad}_j]$. The game $\Delta_{2,j}$ does not depend on j, so write Δ_2 for the game and Bad_j for its eleven events, all on one probability space. $\Pr_{\Delta_0}[\mathrm{win}]$ does not depend on j, so by Lemma K1 $\Pr_{\Delta_0}[\mathrm{win}] \le \Pr_{\Delta_2}[\mathrm{win} \vee \mathrm{Bad}_j]$ for every j, and averaging over the eleven values,

$$\begin{aligned} \Pr_{\Delta_0}[\mathrm{win}] &\le \mathrm{Exp}_{\Delta_2}\left[\frac{1}{11}\sum_j \mathbf{1}[\mathrm{win} \vee \mathrm{Bad}_j]\right] \\ &= \mathrm{Exp}_{\Delta_2}\left[\min\left(1, \mathbf{1}[p^* = P] + \frac{n_{\mathrm{bad}}}{11}\right)\right]. \end{aligned} \tag{2}$$

where n_{bad} is the number of $j \in \{0, \ldots, 10\}$ for which Bad_j holds. The equality follows because the quantities under the two expectations agree: both are 1 if the adversary wins, and otherwise both are $n_{\mathrm{bad}} \,/\, 11$. In Δ_2 the run is independent of P.

Two chains. Δ_2 is implemented lazily as in A.3: P is drawn first and not used during the run, Y is uniform, the tables start empty, and after the adversary halts a finalization evaluates

1. the **forward chain** of P from X: $d_1 = (0, R_0)$ [B], $d_2 = (P, S_0)$ [A], $d_3 = (K_0, 0, R_0)$ [H], $d_4 = (1, R_1)$ [B], …, $d_{36} = (K_{11}, 11, R_{11})$ [H], with $R_{i+1} = L_i \oplus M_i$; then
2. the **backward chain** $c_1 \ldots c_{36}$ of A.3.

h_j is the forward H point of round j, d_{3j+3}, and h_{j+1} is the backward H point of round $j+1$. In Δ_2 the two chains share the tables but need not be consistent with each other. A point of one chain may coincide with a point of the other; it is then sampled once, possibly during the other chain's finalization. "Defined" has the meaning of A.3, for each chain separately, and d_1 and c_1 are defined from the start. Early_P holds if some point of either chain is already in its table at the moment it becomes defined. This is a freshness event. An insertion by either finalization can trigger it even when the adversary has not queried the point. Its probability is bounded in Δ_2, where the two paths need not match. **Cross** holds if some forward A point of round i equals some backward A point of a round $i' \neq i$.

Lemma K2.

1. On $\neg\mathrm{Early}_P$, for each chain, if the adversary queried a point of the chain, it queried the preceding point of the same chain strictly earlier. The proof of Lemma G2 applies per chain. That proof concerns point values, so it holds whichever chain's query first sampled the preceding point.
2. On $\neg\mathrm{Early}_P$, the 36 points of each chain are pairwise distinct, by the proof of Lemma G5 per chain. Across the chains, only A points of different rounds are claimed distinct, and only on $\neg\mathrm{Cross}$. Points of the same round may coincide.
3. $\Pr_{\Delta_2}[\mathrm{Cross} \wedge \neg\mathrm{Early}_P] \le 132 \,/\, 2^{128}$.

Proof of 3. Fix rounds $i \neq i'$; there are 132 ordered pairs. The two A points are equal exactly when the salts $S_i^f = B(i, R_i^f)$ and $S_{i'}^b = B(i', R_{i'}^b)$ are equal. Their B points carry different round indices, so those are different points. Let τ be the moment at which the later of the two is sampled if each was sampled after it became defined, and never otherwise; this moment is a stopping time. On $\neg\mathrm{Early}_P$ neither point was in its table when it became defined, so this moment occurs. The answer sampled at τ is uniform given the history before τ, which contains the other salt. Each pair contributes at most 2^{-128}.

Distinctness of A points of different rounds across the chains genuinely needs $\neg\mathrm{Cross}$. Both first salts, $B(0, R_0)$ and $B(11, L_{12})$, are computable from (X, Y). They coincide with probability 2^{-128}. In that case the keys of rounds 0 and 11 coincide for every password, and a filter that skips one round costs ten calls. Cross charges such events.

Lemma K3. $\Pr_{\Delta_2}[\mathrm{Early}_P] \le (24 \cdot p_1 \cdot k_A + 2 \cdot q_B + 287) \,/\, 2^{128} + (2 \cdot q_H + 12) \,/\, 2^{256}$.

Proof. $\Pr[\mathrm{Early}_P] \le \sum_{\text{chain}} \sum_t \Pr[\mathrm{Early}_t \wedge \neg\mathrm{Early}_{<t}]$, where Early_t and $\mathrm{Early}_{<t}$ refer to the steps of the same chain. Within one chain the steps are ordered in time, so $\neg\mathrm{Early}_{<t}$ is determined before the sample at step t. Each term is bounded as in Lemma G3 by $\mathrm{Exp}[n_t] \,/\, V_t$. A step of one chain can take place during the other chain's finalization, through a shared point. In that case the tables hold a subset of the points counted below, so the counts remain upper bounds. N_P^A is the number of the adversary's A queries with password P, and $\mathrm{Exp}[N_P^A] \le p_1 \cdot k_A$ since the run is independent of P. N_i^B and N_i^H count the adversary's B and H queries with round index i.

The forward chain is finalized first. Besides the adversary's points, the tables then hold at most the forward chain's earlier points:

- the B step of round i (sample S_i, next (P, S_i)) meets N_P^A adversary points and i earlier forward A points; summed over $i = 0 \ldots 11$, $(12 \cdot N_P^A + 66) \,/\, 2^{128}$;
- the A step of round i (sample K_i, next (K_i, i, R_i)) meets only adversary H points with index i; summed, $q_H \,/\, 2^{256}$;
- the H step of round $i \le 10$ (sample M_i, next $(i+1, R_{i+1})$ with a uniform half) meets only adversary B points with index $i+1$; summed, $q_B \,/\, 2^{128}$.

The backward chain is finalized second, and the tables may then also hold the 36 forward points:

- the B step of round i meets N_P^A adversary points, the 12 forward A points and $11 - i$ earlier backward A points; summed over i, $(12 \cdot N_P^A + 144 + 66) \,/\, 2^{128}$;
- the A step of round i meets adversary H points with index i and the one forward H point with that index; summed, $(q_H + 12) \,/\, 2^{256}$;

- the H step of round $i \geq 1$ (next $(i-1, R_{i-1})$) meets adversary B points with index $i-1$ and the one forward B point with that index; summed, $(q_B + 11) \,/\, 2^{128}$.

Adding these and using $\mathrm{Exp}[N_P^A] \leq p_1 \cdot k_A$ gives the bound, with $66 + 144 + 66 + 11 = 287$.

Lemma K4. On $\neg\mathrm{Early}_P \wedge \neg\mathrm{Cross}$, $n_{\mathrm{bad}} \leq \min(11, N_P^A)$.

Proof. Let f be the number of forward rounds whose H point the adversary queried and b the number of backward rounds whose H point it queried. By Lemma K2(1), these are the forward rounds $0 \ldots f-1$ and the backward rounds $11 \ldots 12-b$, and the adversary queried the forward A points of rounds $0 \ldots f-1$ and the backward A points of rounds $12-b \ldots 11$. Bad_j holds through h_j exactly when $j \leq f-1$, and through h_{j+1} exactly when $j+1 \geq 12-b$, so $n_{\mathrm{bad}} \leq \min(11, f+b)$.

- If $f + b \leq 12$, the two sets of rounds are disjoint. The forward points are pairwise distinct and so are the backward points, by Lemma K2(2). A forward and a backward point come from different rounds and are distinct on $\neg\mathrm{Cross}$. So $N_P^A \geq f + b \geq n_{\mathrm{bad}}$.
- If $f + b \geq 13$, then $12 - b \leq f - 1$, and the forward A points of rounds $0 \ldots f-1$ together with the backward A points of rounds $f \ldots 11$ are twelve points of twelve different rounds, distinct for the same reasons. So $N_P^A \geq 12 > 11 \geq n_{\mathrm{bad}}$.

Assembly. From (2), Lemma K4, Lemma K2(3) and Lemma K3,

$$\begin{aligned}\Pr_{\Delta_0}[\mathrm{win}] \leq \mathrm{Exp}_{\Delta_2}\left[\min\left(1, \mathbf{1}[p^* = P] + \frac{N_P^A}{11}\right)\right] \\ + \Pr_{\Delta_2}[\mathrm{Early}_P] + \Pr_{\Delta_2}[\mathrm{Cross} \wedge \neg\mathrm{Early}_P].\end{aligned}$$

In Δ_2, P is independent of the run, so the expectation equals $\mathrm{Exp}[\sum_w \Pr[P = w] \cdot u_w]$ with $u_w = \min(1, 1[p^* = w] + N_w^A \,/\, 11)$. The weights lie between 0 and 1 and sum to at most $1 + k_A \,/\, 11$, so the expectation is at most $T(1 + k_A \,/\, 11)$. With $287 + 132 = 419$ this gives Theorem 4.

Proposition K5 (matching attack). Let $m \geq 0$. For each of $w_1 \ldots w_m$, the attack evaluates rounds $11 \ldots 1$ backward from Y and accepts the password if the resulting L_1 equals R_0, the right half of X; round 0 copies this half unchanged into L_1. The attack outputs the first accepted password, or w_{m+1} if none is accepted. Its budget is $k_A \leq 11m$, $q_B \leq 10m + 1$ (the first B point $(11, L_{12})$ is shared) and $q_H \leq 11m$. With $\delta_K = 2^{-128} + 11 \cdot (m+1) \cdot 2^{-256}$, its success probability in Δ_0 is at least

$$\sum\nolimits_{j \leq m+1} p_j \cdot (1 - (j-1) \cdot \delta_K) \geq \mathrm{top}(m+1) \cdot (1 - m \cdot \delta_K).$$

Proof. The real password is always accepted, so the attack succeeds on $P = w_j$, $j \leq m+1$, unless some w_i with $i < j$ is accepted. Condition on $P = w_j$ and on the values at the points of the real path. For a tested $w_i \neq w_j$, the A points of its chain carry password w_i, so each key of its chain is a fresh uniform value when first sampled. An H point of its chain coincides with an earlier sampled H point of the same index (one on the real path and at most one per earlier tested password) only if both have the same right half, hence the same salt, and the same key. Given the earlier paths, the key of w_i at that salt is uniform even if its chain repeats a salt, so such a coincidence has probability at most

$(m+1)\cdot 2^{-256}$. Over eleven rounds, this gives $11(m+1)\cdot 2^{-256}$. Otherwise M_1 is fresh after R_2 is fixed, and $L_1 = R_2 \oplus M_1 = R_0$ has probability 2^{-128}. So $\Pr[P = w_j \wedge w_i \text{ accepted }] \le p_j \cdot \delta_K$, and summing over $i < j$ gives the bound.

With $m = \lfloor k_A \,/\, 11 \rfloor$, the best success therefore lies between $\mathrm{top}(m+1)(1 - m\cdot\delta_K)$ and $T(1 + k_A \,/\, 11) + \varepsilon_K \le \mathrm{top}(m+2) + \varepsilon_K$. The eleven calls need not be sequential: a filter can evaluate a forward part from X and a backward part from Y in parallel, so about six sequential calls suffice per password. No depth bound is claimed for a known pair.

A.6 Theorem 5: a 24-word source with an unknown BIP39 passphrase

The specification says that with a strong, independent BIP39 passphrase the wallet check requires both secrets, so that an attacker repeats the expensive recovery for MHFE password candidates and tests passphrase candidates for each. The composition section gives the expected cost of that pair search. Theorem 5 gives an upper bound on seed-recovery probability for any attack in the model, with explicit error terms. Proposition W7 states when pair search approaches that bound and how false matches can leave a gap.

The danger to exclude is an intermediate value that depends on one secret only and can be recognized. Such a value, or an integrity check on it, lets each secret be guessed separately (A.8). For a 24-word source, the intermediate value is the recovered entropy. In the model below, it cannot be recognized without the passphrase: every 256-bit value is a valid phrase, the wallet check is computed only after the passphrase enters, and the adversary has no other data that identifies the phrase alone.

The main term is the total probability of the most likely password and passphrase pairs that the budgets allow, counting twelve Argon2id calls per password and one seed computation per pair. The 16-bit check does not enter that main term. It enters two error terms, through the selection of the source and through queries at the unknown seed.

Model. Fix suite 3 with $\mathrm{ENT} = 256$ (24 words, $r = 0$, no verifier, $X = E$) and the settings. A, B, H and assumptions 1 to 4 are those of A.1; G does not enter. By assumption 7 there are three further independent random functions:

- $S : \{0,1\}^{256} \times \mathrm{Qw} \to \{0,1\}^{512}$, the BIP39 seed $\mathrm{PBKDF2\text{-}HMAC\text{-}SHA512}(\mathrm{UTF8}(\mathrm{NFKD}(M(e))), \texttt{"mnemonic"} \| \mathrm{UTF8}(\mathrm{NFKD}(q)), 2048, 512)$ computed from the phrase $M(e)$ of e and the passphrase q. Qw is the set of normalized passphrases. $M(e)$ contains the checksum $\mathrm{Trunc}_8(\mathrm{SHA\text{-}256}(e))$ and is an injective encoding of e, so modelling S as a function of (e, q) loses nothing. One query stands for one seed computation of 2,048 iterations, and the budget k_S counts these computations as the second expensive resource.
- $T : \{0,1\}^{512} \to \{0,1\}^{256}$, the check digest $\mathrm{SHA\text{-}256}(\texttt{"MHFE-WALLET-CHECK-SEED-1"} \| \mathrm{BE32}(256) \| \mathrm{seed})$; the check passes when the first 16 bits are zero. Budget q_T.
- $\Omega : \{0,1\}^{512} \to \{0,1\}^{\lambda}$, the data of the funded wallet, derived from the seed, that the adversary may know, such as receiving addresses or the master key fingerprint. The caveats of Corollary G10 on the width λ apply. Budget q_Ω.

S_i remains the salt of round i, and T_A, T_B, T_H the tables of A.3; S and T without an index are the seed and check functions.

Keys are not in the model. An attack that obtains a spending key without the seed would have to exploit the derivation from the seed to keys and addresses, which assumption 7 idealizes.

Secrets. The MHFE password $P \in \mathrm{Pw}$ and the BIP39 passphrase $Q \in \mathrm{Qw}$ are independent of each other and of the functions. Their probabilities are $p_1 \geq p_2 \geq \ldots$ for passwords $w_1, w_2, \ldots$ and $s_1 \geq s_2 \geq \ldots$ for passphrases $v_1, v_2, \ldots$. Each is the probability of a normalized value, the total over the inputs that normalize to it.

Pair top. For integers $m, n \geq 0$, $\mathrm{top2}(m, n)$ is the largest sum $\sum p_i \cdot s_j$ over a set of at most n index pairs (i, j) with $i \leq m$: the sum of the n largest such products, or of all of them if there are fewer.

Lemma W1. Every set Π of at most n password and passphrase pairs that uses at most m distinct passwords has $\sum_{(w,v)\in\Pi} \Pr[P = w] \cdot \Pr[Q = v] \leq \mathrm{top2}(m, n)$.

Proof. Pairs of probability 0 contribute nothing and are dropped. Number the passwords of Π as $u_1, u_2, \ldots$ by decreasing probability; then $\Pr[P = u_k] \leq p_k$. The map $(u_k, v_j) \mapsto (k, j)$ is injective, its images have $k \leq m$, and it does not decrease the products, so the sum is at most a sum of at most n distinct products $p_k \cdot s_j$ with $k \leq m$.

Game Θ_0 (real).

1. Draw P and Q.
2. Generate the source as the profile prescribes: repeat $e \leftarrow \{0,1\}^{256}$ and $d = S(e, Q)$ until $\mathrm{Trunc}_{16}(T(d)) = 0^{16}$. Let E be the passing e and $\mathrm{seed}^* = S(E, Q)$.
3. Create the container $Y = \mathrm{Perm}_P(E)$, with the creation refusal of Remark G9: if $Y = E$, the owner changes the password, PIM or memory level.
4. Let $\omega^* = \Omega(\mathrm{seed}^*)$. The adversary receives Y and ω^* and knows that the profile was used. It queries A, B, H, S, T, Ω within its budgets $k_A, q_B, q_H, k_S, q_T, q_\Omega$ and repeats no query.
5. When it halts, it outputs a string σ and a pair (p^*, q^*). It wins the seed game if $\sigma = \mathrm{seed}^*$, and the pair game if $(p^*, q^*) = (P, Q)$.

Data that identifies the phrase alone. The adversary receives Y and the target wallet reference ω^* specified in Θ_0, but no additional phrase-identifying data. The reference ω^* is allowed even when Q is empty or independently known. In particular:

- When Q is unknown to the adversary, the wallet with the empty passphrase remains unused: the adversary knows no address, fingerprint or transaction history of it. A zero balance is not enough, since past history also identifies the phrase.
- All funds stay under the passphrase. A funded wallet of the same phrase under another passphrase, such as a decoy passphrase, is outside Θ_0 (A.7). If the adversary knows or guesses that passphrase, the wallet identifies the phrase on its own.
- No other reference identifies the original seed phrase without the passphrase.

The address computations of an unused wallet are S and Ω queries like any others. A funded wallet that another MHFE password opens on the same container is outside Θ_0 for a different reason: it adds a second relation on Y and A, B, H.

Selection rule. Step 2 is the specification's sequential rule. Theorem 5 also covers any rule that meets four conditions:

- it runs independent uniform trials;
- it selects a passing trial by the order of the trials and their pass/fail outcomes alone, with any coins independent of the trial values;
- it stops with probability 1 whenever a passing value exists;

- it reveals nothing (assumption 4).

Such a rule reduces to the sequential one.

- Treat S, T and Q as fixed; the random functions can be sampled in full in advance. Let $U = \{e : \mathrm{Trunc}_{16} T(S(e, Q)) = 0^{16}\}$ and suppose U is not empty.
- Given the sequence of outcomes, the trials are independent, each uniform on U if it passed and on its complement otherwise.
- The selected index depends only on the outcomes and on independent coins, so $\Pr[E = e | S, T, Q] = 1 \,/\, |U|$ for every $e \in U$, as for the first success.
- The owner's trials only read the functions. Everything the adversary sees is a function of $P, Q, A, B, H, S, T, \Omega$, E and its own coins.
- So the two rules give the same joint distribution of all of these, and Lemma W2 below applies unchanged. If U is empty, neither rule stops; Lemma W2 gives that event an arbitrary outcome.

A search on parallel workers that takes the first result found meets these conditions if neither the scheduling of the workers nor the order in which their trials finish depends on the trial values. The order in which the trials finish can depend on the values even when the scheduling does not, if the duration of a trial depends on its value. Two kinds of rule are not covered: a rule that depends on the trial values, such as the smallest passing value in a batch, and a generation that reveals its history or duration.

Theorem 5. For every adversary in Θ_0,

$$
\begin{aligned}
\text{(a)} \quad & \Pr[\sigma = \mathrm{seed}^*] \le \mathrm{top2}\left(\left\lfloor \frac{k_A}{12} \right\rfloor , k_S \right) + \varepsilon_W + 2^{-512}, \\
\text{(b)} \quad & \Pr[(p^*, q^*) = (P, Q)] \le \min\Big\{ \mathrm{top2}\left(\left\lfloor \frac{k_A}{12} \right\rfloor + 1, k_S + 1 \right), \\
& \qquad\qquad \mathrm{top2}\left(\left\lfloor \frac{k_A}{12} \right\rfloor , k_S \right) + p_1 s_1 \Big\} + \varepsilon_W, \\
& \varepsilon_W = \frac{s_1 k_S + q_B + (1 + 12 p_1) k_A + 66}{2^{128}} + \frac{q_H + k_A}{2^{256}} \\
& \qquad + \frac{q_T + q_\Omega}{2^{512}} + \delta_{\mathrm{gen}} + 2^{-256}, \\
& \delta_{\mathrm{gen}} = \frac{2^{18}(k_S + 1)}{2^{256}} + \frac{2^{18} q_T}{2^{512}}.
\end{aligned}
$$

The term 2^{-256} accounts for the creation refusal of Remark G9.

Reading for independent uniform secrets. Take N_1 equally likely passwords and N_2 equally likely passphrases. Then $\mathrm{top2}(m, n) = \min(n, \min(m, N_1) \cdot N_2) \,/\, (N_1 \cdot N_2)$, and part (a) gives at most $\min(k_S, \lfloor k_A \,/\, 12 \rfloor \cdot N_2) \,/\, (N_1 \cdot N_2) + \varepsilon_W + 2^{-512}$.

- Ignoring the explicit error terms, seed recovery with probability one half needs both resources: $k_A \ge 12 \cdot \lceil N_1 \,/\, 2 \rceil$ Argon2id calls and $k_S \ge \lceil N_1 \cdot N_2 \,/\, 2 \rceil$ seed computations. These are approximate thresholds only when $\varepsilon_W + 2^{-512}$ is negligible. The Argon2id cost does not multiply with N_2; the passphrase adds seed computations, each far cheaper than an Argon2id call.

- The error terms must be compared with the main term at the chosen budgets.
  - With $s_1 = 1 \,/\, N_2$, the seed term is $s_1 \cdot k_S \,/\, 2^{128} = k_S \,/\, (N_2 \cdot 2^{128})$. At these budgets, where $k_S \le \lfloor k_A \,/\, 12 \rfloor \cdot N_2$, the main term is $k_S \,/\, (N_1 \cdot N_2)$, and the seed term is smaller whenever $N_1 < 2^{128}$.
  - For example, an MHFE password and a passphrase of five EFF words each give about $2^{129.25}$ pairs. At $k_S = N_1 \cdot N_2 \,/\, 2$ the seed term is about $4 \cdot 10^{-20}$; without the factor s_1 it would be about 1.19, and the bound would say nothing.
  - The seed-query part of δ_{gen}, $2^{18} \cdot (k_S + 1) \,/\, 2^{256}$, is small while k_S stays well below 2^{238}. For larger spaces, such as $N_1 = 2^{64}$ and $N_2 = 2^{192}$ at $k_S = N_1 \cdot N_2 \,/\, 2$, it is about 2^{17}, and the bound says nothing. Its other part, $2^{18} \cdot q_T \,/\, 2^{512}$, must be included too. The remaining terms depend on their own query budgets, and even an error that is small in absolute value must be compared with the main term.

- With these budgets, Proposition W7 gives a matching attack that tests the check first and the wallet data after a pass. It recovers the seed with probability at least $(1 \,/\, 2) \cdot (1 - (\lceil N_1 \cdot N_2 \,/\, 2 \rceil - 1) \cdot 2^{-(\lambda+16)}) - \delta_{\mathrm{gen}} - 2^{-256}$. The bound is therefore tight up to this false-match loss and the error terms when $\lambda + 16$ is well above $\log_2(N_1 \cdot N_2)$.
  - $\lambda + 16 \ge \log_2(N_1 \cdot N_2) + 6$ keeps the loss below 1%.
  - A 32-bit fingerprint meets this only for $\log_2(N_1 \cdot N_2) \le 42$. An address, idealized at about 160 bits, meets it for every feasible budget.
  - This concerns how close the attack comes to the bound, not the bound itself. It matters only when $N_1 \cdot N_2 \,/\, 2$ seed computations are feasible.
  - When the tests are narrow, W7 shows little. With $N_1 = N_2 = 2^{10}$ and the check as the only test, the matching attack succeeds with probability about $1 \,/\, 16$, while Theorem 5 allows $1 \,/\, 2$.

One cost budget. In Θ_2 below, the pair set Z is independent of (P, Q). Conditioning on the run and applying Lemma W1 to Z gives $\Pr[(P, Q) \in Z] \le \mathrm{Exp}_{\Theta_2}[\mathrm{top2}(|C'|, |Q_S|)]$. Now let the attacker divide one budget c between the two resources as it goes, paying C_A per Argon2id call and C_S per seed computation, with total cost at most c on every execution. For such an attacker, the proof of Theorem 5 together with this bound gives $\Pr[\sigma = \mathrm{seed}^*] \le \mathrm{Exp}_{\Theta_2}[\mathrm{top2}(|C'|, |Q_S|)] + \varepsilon_W + 2^{-512}$, with ε_W evaluated at $k_A = \lfloor c \,/\, C_A \rfloor$ and $k_S = \lfloor c \,/\, C_S \rfloor$.

- For uniform secrets, $m = |C'|$ and $n = |Q_S|$ satisfy $12 \cdot C_A \cdot m + C_S \cdot n \le c$ on every run. For $x = \min(n, m \cdot N_2)$ this gives $x \cdot (12 \cdot C_A + N_2 \cdot C_S) \,/\, N_2 \le c$, so $\mathrm{top2}(m, n) \le x \,/\, (N_1 \cdot N_2) \le c \,/\, (N_1 \cdot (12 \cdot C_A + N_2 \cdot C_S))$.
- Such an attacker recovers the seed with probability at most $c \,/\, (N_1 \cdot (12 \cdot C_A + N_2 \cdot C_S)) + \varepsilon_W + 2^{-512}$.
- Therefore, success probability at least one half requires $c \ge (1 \,/\, 2 - \varepsilon_W - 2^{-512}) \cdot N_1 \cdot (12 \cdot C_A + N_2 \cdot C_S)$, with the error terms evaluated at the budgets allowed by c as above. This approaches half the full-search cost only while those terms are negligible. It gives no positive lower bound when their sum is at least one half.

A known passphrase. Suppose Q is known: the empty-passphrase form of the profile, or a passphrase that leaked independently of P (otherwise the p_i are the probabilities given the leak). Then $s_1 = 1$ and $\mathrm{top2}(m, n) = \mathrm{top}(\min(m, n))$, and part (b) gives $\mathrm{top}(\min(\lfloor k_A \,/\, 12 \rfloor, k_S) + 1) + \varepsilon_W$. This is the analogue of Corollary G10 for a 24-word source generated under the profile, which G10 does not cover because the profile conditions E.

- The wallet reference ω^* in Θ_0 is allowed. Additional phrase-identifying data and funded openings under other MHFE passwords remain outside that experiment even when Q is known.
- The main term has $\min(\lfloor k_A \,/\, 12 \rfloor, k_S)$, because each tested password also needs a seed computation.
- G10's term $q_\Omega \,/\, 2^{128}$, for queries that read E, becomes $s_1 \cdot k_S \,/\, 2^{128} = k_S \,/\, 2^{128}$. Ω reads the seed and contributes $q_\Omega \,/\, 2^{512}$.
- The terms $k_A \,/\, 2^{128}$ and $k_A \,/\, 2^{256}$ of Lemma W4, $q_T \,/\, 2^{512}$ and δ_{gen} are new, and q_G disappears.

What the check gives an attacker. The bound does not depend on λ. Up to δ_{gen} and $q_T \,/\, 2^{512}$, it equals the bound of Corollary W6 for an ordinary 24-word phrase with the same passphrase, against an attacker who knows the same wallet data. Against an attacker without wallet data, an ordinary phrase is better protected:

- Every attacker has the check, a 16-bit filter on pairs.
  - With wallet data of width λ, it adds 16 bits to the λ-bit wallet-data filter: in the model, a wrong candidate seed $\sigma \neq \text{seed}^*$ passes both with probability $2^{-(\lambda+16)}$, and a wrong pair whose seed equals seed^* passes both.
  - Without wallet data, it is the only filter. The bound still holds, because it already covers an attacker with wallet data.
- Example: $N_1 = N_2 = 2^{10}$ and no ω^*.
  - Without the profile and ignoring the creation refusal, the container is independent of (P, Q), and the best success in the pair game is 2^{-20}.
  - With the profile, the attacker inverts all 2^{10} passwords and computes all 2^{20} seeds. The real pair passes, together with about 16 false passes, so a random passing pair is right with probability about $(1 - e^{-16}) \,/\, 16 \approx 0.06$.
  - Theorem 5 allows this, since for these budgets $\text{top2} = 1$.

Corollary W5 (phrase and passphrase). If the adversary outputs $(e_{\text{out}}, q_{\text{out}})$ and wins when it equals (E, Q), then $\Pr[\text{win}] \le \text{top2}(\lfloor k_A \,/\, 12 \rfloor, k_S + 1) + \varepsilon_W$, with $k_S + 1$ in place of k_S in ε_W.

Corollary W6 (ordinary 24-word source). For a uniform E generated without the profile, with an unknown passphrase and wallet data $\omega^* = \Omega(S(E, Q))$, Theorem 5 and Corollary W5 hold with $\delta_{\text{gen}} = 0$ and without the term $q_T \,/\, 2^{512}$, since T is not programmed.

Proof of Theorem 5. The proof follows A.3 and adds two steps: the generation by rejection is replaced by one programmed point, and the seed queries are assigned to passwords.

Creation refusal. By Remark G9, Θ_0 with refusal and Θ_0 without refusal coincide unless the first attempt is refused. For every E, P, A and B, that refusal has probability 2^{-256} over the choice of H, and H is independent of the generation. From here on Θ_0 has no refusal, and the bound gains 2^{-256}.

Game Θ_0'. Θ_0, except for the generation:

1. $E \leftarrow \{0,1\}^{256}$, $\text{seed}^* \leftarrow \{0,1\}^{512}$ and $\eta^* \leftarrow \{0,1\}^{240}$;
2. S and T are uniform except $S(E, Q) = \text{seed}^*$ and $T(\text{seed}^*) = 0^{16} \| \eta^*$.

Lemma W2. The joint distribution of P, Q, E, seed^*, A, B, H, Ω and the adversary's view differs between Θ_0 and $\Theta_0{}'$ by at most δ_{gen} in statistical distance.

Proof. Implement S and T lazily. The loop of Θ_0 reads S only at its trial points (e_i, Q) and T only at $d_i = S(e_i, Q)$. Given the loop's transcript, S and T are uniform at every other point. Let J be the number of trials, $E = e_J$ and $\mathrm{seed}^* = d_J$; the trials $i < J$ are rejected. A rejected e_i differs from E, and a rejected d_i from seed^*, since equal points give equal check results.

Let ${\Theta_0}''$ be Θ_0 in which, after generation, the values of S at the rejected points (e_i, Q) and of T at the rejected d_i are replaced by fresh uniform values. Θ_0 and ${\Theta_0}''$ are identical until the adversary queries S at a rejected (e_i, Q) or T at a rejected d_i.

${\Theta_0}''$ and ${\Theta_0}'$. In ${\Theta_0}''$, S and T are uniform except $S(E, Q) = \mathrm{seed}^*$ and $T(\mathrm{seed}^*) = 0^{16} \| \eta^*$, where η^* is the rest of the passing check value. The loop enters the public functions only through $(E, \mathrm{seed}^*, \eta^*)$. Condition on $Q = q$. Three kinds of permutation leave the loop's distribution unchanged, preserve J and every check outcome, and each changes one coordinate of $(E, \mathrm{seed}^*, \eta^*)$:

- a permutation π of $\{0,1\}^{256}$, applied to the points of S at q and to the trials;
- a permutation ρ of $\{0,1\}^{512}$, applied to the outputs of S at q and to the inputs of T;
- a permutation of the last 240 bits of the outputs of T.

Together they act transitively on $\{0,1\}^{256} \times \{0,1\}^{512} \times \{0,1\}^{240}$. So, given $Q = q$ and J, the triple is uniform, and it is independent of P, A, B, H, Ω, which the loop does not use. Hence ${\Theta_0}''$ and ${\Theta_0}'$ give the same joint distribution, except when the loop runs past 2^{80} trials (below). That case, including a loop that never stops, may be given any outcome.

The rejected points. The same permutations, restricted to those that fix E or seed^*, give the distribution of the rejected points. Given $(Q, J, E, \mathrm{seed}^*, \eta^*)$, each rejected e_i is uniform on $\{0,1\}^{256} \setminus \{E\}$ and each rejected d_i is uniform on $\{0,1\}^{512} \setminus \{\mathrm{seed}^*\}$. In ${\Theta_0}''$ the adversary's view is a function of $(Q, E, \mathrm{seed}^*, \eta^*)$ and of the values of S and T away from the programmed points, together with P, A, B, H, Ω and the coins. Given the loop's transcript, those function values are uniform and independent of it. So, given $(Q, J, E, \mathrm{seed}^*, \eta^*)$, the rejected points are independent of the view, and a query hits a given rejected point with probability at most $2 \,/\, 2^{256}$ for S and $2 \,/\, 2^{512}$ for T.

The number of trials. This part is for the sequential rule; the selection rule above reduces the other covered rules to it. A trial whose e_i and d_i are new points passes with probability exactly 2^{-16}. Among the first 2^{80} trials, a point repeats with probability at most $2^{81} \,/\, 2^{256}$ per trial, so each such trial passes with probability at least 2^{-17} given the earlier ones. Hence $\sum_{i \le 2^{80}} \Pr[J > i] \le 2^{17}$ and $\Pr[J > 2^{80}] \le (1 - 2^{-17})^{2^{80}} < \exp(-2^{63}) < 2^{-256}$.

Total. The adversary queries a rejected point in ${\Theta_0}''$ with probability at most
$2^{17} \cdot (2 \cdot k_S \,/\, 2^{256} + 2 \cdot q_T \,/\, 2^{512}) + 2^{-256}$. The distance between Θ_0 and ${\Theta_0}'$ is therefore at most
$2^{18} \cdot k_S \,/\, 2^{256} + 2^{18} \cdot q_T \,/\, 2^{512} + 2 \cdot 2^{-256} \le \delta_{\mathrm{gen}}$.

Re-sampling the container. In Game Θ_1, draw P and Q, $Y \leftarrow \{0,1\}^{256}$, $\omega^* \leftarrow \{0,1\}^{\lambda}$, $\mathrm{seed}^* \leftarrow \{0,1\}^{512}$ and uniform A, B, H, S', T', Ω'. Let $e^* = \mathrm{Inv}_P(Y)$. S, T and Ω equal S', T' and Ω', except:

- $S(e^*, Q) = \mathrm{seed}^*$;
- the first 16 bits of $T(\mathrm{seed}^*)$ are zero, and its other 240 bits are those of $T'(\mathrm{seed}^*)$;
- $\Omega(\mathrm{seed}^*) = \omega^*$.

Lemma W3. $(P, Q, A, B, H, Y, S, T, \Omega, \omega^*, E, \mathrm{seed}^*)$ has the same joint distribution in $\Theta_0{}'$ and Θ_1, where $E = e^*$ in Θ_1.

Proof. Condition on $P = p$, $Q = q$ and A, B, H; then Perm_p is a fixed permutation of $\{0,1\}^{256}$. In $\Theta_0{}'$, E is uniform, so $Y = \mathrm{Perm}_p(E)$ is uniform and $E = \mathrm{Inv}_p(Y) = e^*$. The two games then agree point by point:

- both set S at the point (e^*, q) to a uniform seed^* and leave S uniform elsewhere;
- both make $T(\mathrm{seed}^*)$ uniform with its first 16 bits zero and leave T uniform elsewhere;
- in $\Theta_0{}'$, $\omega^* = \Omega(\mathrm{seed}^*)$ for a uniform Ω, and in Θ_1, Ω' is uniform and set to a uniform ω^* at seed^*. In both, ω^* is uniform and Ω is uniform with $\Omega(\mathrm{seed}^*) = \omega^*$, as in the proof of Corollary G10.

Removing the programmed points. Game Θ_2 is Θ_1 with $S = S'$, $T = T'$ and $\Omega = \Omega'$. Nothing is programmed, and ω^* and seed^* are independent uniform values. **Bad** is the event that the adversary queries S at (e^*, Q), T at seed^* or Ω at seed^*; Bad_S is the first and $\mathrm{Bad}_T\Omega$ the other two. Θ_1 and Θ_2 differ only at those three points, and e^*, seed^* and both winning conditions are defined identically in both, so the two games are identical until Bad. With Lemmas W2 and W3 and the refusal step, for the seed game and for the pair game,

$$\begin{aligned} \Pr_{\Theta_0}[\mathrm{win}] &\le \Pr_{\Theta_1}[\mathrm{win}] + \delta_{\mathrm{gen}} + 2^{-256} \\ &\le \Pr_{\Theta_2}[\mathrm{win} \vee \mathrm{Bad}] + \delta_{\mathrm{gen}} + 2^{-256}. \end{aligned} \tag{3}$$

In Θ_2 the adversary's run is a function of $(Y, \omega^*, A, B, H, S', T', \Omega')$ and its coins, which are independent of (P, Q, seed^*). Θ_2 is run lazily as in A.3:

1. P and Q are drawn first and not used during the run.
2. The tables for A, B, H, S, T and Ω start empty.
3. After the adversary halts, finalization evaluates the chain $c_1 \dots c_{36}$ of P from Y, obtains $e^* = \mathrm{Inv}_P(Y)$ and decides Bad from the recorded queries.

Early_P, $\mathrm{Comp}(P)$ and Lemmas G2, G3 and G5 concern only A, B, H, the run and the chain of P, so they carry over unchanged; the tables of S, T and Ω are separate. Q_S is the set of the adversary's S queries.

The seed. seed^* is uniform and used nowhere in Θ_2, so $\Pr_{\Theta_2}[\mathrm{Bad}_T\Omega] \le (q_T + q_\Omega) \,/\, 2^{512}$.

The seed query. Let N be the event of Lemma G4: $c_{36} = (K_0, 0, R_0)$ is not in T_H when finalization reaches it. N holds exactly when the adversary did not query c_{36}, because the chain's other H points carry other round indices. There are three cases.

- Early_P**.** Lemma G3 bounds it by $(12 \cdot p_1 \cdot k_A + 66 + q_B) \,/\, 2^{128} + q_H \,/\, 2^{256}$.

- $\mathrm{Bad}_S \wedge \neg\mathrm{Early}_P \wedge N$**.** As in Lemma G4, M_0 is sampled at finalization, after Q_S is fixed, and $L_0 = R_1 \oplus M_0$ is uniform given the history.
  - Bad_S needs an S query (e, v) whose first 128 bits equal L_0 and whose passphrase v equals Q.
  - Q is drawn independently of P and the functions. Neither the run nor finalization uses it before Bad is decided, so it is independent of the history and of M_0.
  - For each query the probability is therefore $2^{-128} \cdot \Pr[Q = v] \le 2^{-128} \cdot s_1$. Summed over the queries, it is at most $s_1 \cdot k_S \,/\, 2^{128}$.
  - This term is bounded in Θ_2, where the run does not depend on Q. In Θ_1 the run can depend on Q through the programmed point (e^*, Q). No separate transfer of this event to Θ_1 is claimed: the final bound passes to Θ_1 and Θ_0 through (3), which needs only $\Pr_{\Theta_2}[\mathrm{win} \vee \mathrm{Bad}]$. δ_{gen} needs no factor s_1: in $\Theta_0{}''$ the view depends on Q, but the bound of Lemma W2 holds for each fixed Q.
- $\mathrm{Bad}_S \wedge \neg\mathrm{Early}_P \wedge \neg N$**.** The adversary queried c_{36}, so by Lemma G2 it queried every chain point, each after the previous one. Then $\mathrm{Comp}(P)$ holds, and by Lemma G5 $P \in C'$, the set of passwords with which the adversary made at least twelve A queries, so $|C'| \le \lfloor k_A \,/\, 12 \rfloor$. The assignment below handles this case.

Completed chains. For a password u, its chain from Y is defined as that of P, with u in place of P; its first point $c_1 = (11, L_{12})$ is the same for all passwords. Let $\tau(u)$ be the first moment before the adversary halts at which all 36 points of the chain of u are in the tables of A, B, H. All of them then come from the adversary's queries. If there is no such moment, $\tau(u) = \infty$; finalization does not change $\tau(u)$. If $\tau(u) < \infty$, the tables at $\tau(u)$ determine $e^*(u) = \mathrm{Inv}_u(Y)$. Since τ ranges over the run only, $\tau(u)$ and, when it is finite, $e^*(u)$ are functions of the run.

Assignment. For each $(e, v) \in Q_S$, let $u(e)$ be the password $u \in C'$ with $\tau(u) < \infty$ and $e^*(u) = e$ for which $(\tau(u),\ \text{rank of } u)$ is least, if there is one. The rank orders passwords as $w_1, w_2, \ldots$. Let

$$Z = \{(u(e), v) : (e, v) \in Q_S \text{ and } u(e) \text{ is defined}\}.$$

Z is a function of the run and hence independent of (P, Q). It has at most k_S pairs, and its passwords lie in C'.

On $\mathrm{Bad}_S \wedge \neg\mathrm{Early}_P \wedge \neg N$, $\tau(P)$ is the moment of the adversary's first query of c_{36}, $P \in C'$, and $(e^*, Q) \in Q_S$, so $u(e^*)$ is defined. Either $u(e^*) = P$ and $(P, Q) \in Z$, or the following event holds.

Coll_P: some password $u \ne P$ has $\tau(u) \le \tau(P)$ and $e^*(u) = e^*(P)$.

Lemma W4. $\Pr_{\Theta_2}[\mathrm{Coll}_P \wedge \neg\mathrm{Early}_P \wedge \neg N] \le k_A \,/\, 2^{128} + k_A \,/\, 2^{256}$.

Proof. On $\neg\mathrm{Early}_P \wedge \neg N$, c_{36} was not in T_H when it became defined, and the adversary queried it. So M_0 is sampled fresh at the adversary's first query of c_{36}, at the moment $\tau(P)$. R_1 and R_0 are determined before $\tau(P)$, and $e^*(P) = (R_1 \oplus M_0, R_0)$. There are two cases.

- $\tau(u) < \tau(P)$. The tables before $\tau(P)$ determine $e^*(u)$. Each such u has at least one A query with password u, so there are at most k_A of them. The first halves of their inverted entropies form a set of at most k_A values determined before $\tau(P)$. M_0 is uniform given the history before $\tau(P)$, so $R_1 \oplus M_0$ lies in that set with probability at most $k_A \,/\, 2^{128}$.

- $\tau(u) = \tau(P)$. The query at $\tau(P)$ completes the chain of u, so c_{36} is a point of that chain. It is an H point with round index 0, so it is the last point of the chain of u. The A points $(u, S_0(u))$ and (P, S_0) differ because $u \neq P$, yet they have the same answer K_0. On $\neg\mathrm{Early}_P$, K_0 was sampled fresh at the adversary's first query of c_{35}. For each of the at most k_A other A answers, whichever of the two was sampled later equals the other with probability 2^{-256}, so the total is at most $k_A \,/\, 2^{256}$.

Freshness is needed only for the real chain. Chains of other passwords may have been computed in any order, may repeat a salt or may never be completed. They enter only through the at most k_A first halves that L_0 of the real chain must miss, and through Z, whose size is counted by the S queries. The term $k_A \,/\, 2^{128}$ is not tight: counting only completed chains of passwords that end in C' would need a separate argument for chains that repeat a salt.

Assembly. Z uses at most $\lfloor k_A \,/\, 12 \rfloor$ passwords and has at most k_S pairs, and it is independent of (P, Q), so by Lemma W1 $\Pr_{\Theta_2}[(P, Q) \in Z] \leq \mathrm{top2}(\lfloor k_A \,/\, 12 \rfloor, k_S)$. Every outcome of $\mathrm{win} \vee \mathrm{Bad}$ in Θ_2 lies in one of these sets:

- $\{(P, Q) \in Z\}$, which contains $\mathrm{Bad}_S \wedge \neg\mathrm{Early}_P \wedge \neg N \wedge \neg\mathrm{Coll}_P$;
- the event that the output wins in Θ_2 without Bad: in the seed game $\sigma = \mathrm{seed}^*$, of probability at most 2^{-512}, since seed^* is independent of the run; in the pair game $(p^*, q^*) = (P, Q)$, where (p^*, q^*) is a function of the run;
- Early_P, $\mathrm{Bad}_S \wedge \neg\mathrm{Early}_P \wedge N$, $\mathrm{Coll}_P \wedge \neg\mathrm{Early}_P \wedge \neg N$ and $\mathrm{Bad}_T\Omega$, bounded above.

For the pair game, $\Pr[(P, Q) \in Z \cup \{(p^*, q^*)\}]$ is at most $\mathrm{top2}(\lfloor k_A \,/\, 12 \rfloor + 1, k_S + 1)$ by Lemma W1, and at most $\mathrm{top2}(\lfloor k_A \,/\, 12 \rfloor, k_S) + p_1 \cdot s_1$ by a union bound, since (p^*, q^*) is independent of (P, Q). With (3) this gives Theorem 5.

The terms of ε_W come from these events:

Term	Event
$s_1 \cdot k_S \,/\, 2^{128}$	L_0 of the real chain, sampled after the run, lands in the first 128 bits of an S query with the real passphrase
$k_A \,/\, 2^{128}$	L_0 of the real chain, sampled fresh, equals that of another completed chain (Lemma W4)
$k_A \,/\, 2^{256}$	the real chain's last key equals another password's key (Lemma W4)
$(12 \cdot p_1 \cdot k_A + 66 + q_B) \,/\, 2^{128}$	Early_P: a fresh salt or half of the real chain hits an earlier query (Lemma G3)
$q_H \,/\, 2^{256}$	Early_P: a fresh key of the real chain hits an H query (Lemma G3)
$(q_T + q_\Omega) \,/\, 2^{512}$	a T or Ω query at the unknown seed
δ_{gen}	a query at a rejected generation trial, or the loop running past 2^{80} trials (Lemma W2)
2^{-256}	the creation refusal (Remark G9)

Proof of Corollary W5. $E = e^*$ by Lemma W3, so the event becomes $(e_{\mathrm{out}}, q_{\mathrm{out}}) = (e^*, Q)$ in Θ_1 and Θ_2. Treat $(e_{\mathrm{out}}, q_{\mathrm{out}})$ as one more S query at the end of the run. The event then implies Bad_S for a run with at most $k_S + 1$ queries, and the assembly without an output gives the bound.

Proof of Corollary W6. Without the profile, S is uniform and $\text{seed}^* = S(E, Q)$, which has the same distribution as a uniform S with $S(E, Q)$ set to a uniform seed^*. So Θ_0 has the same distribution as $\Theta_0{}'$ without the programming of T, and Lemma W2 is not needed. Since T is not programmed, Bad includes no T query. The rest of the proof is unchanged.

Proposition W7 (matching attack). For a password w write $x(w) = \text{Inv}_w(Y)$. Let $m \geq 1$ and $n \geq 1$, and let Π be a set of at most n pairs, with passwords among $w_1 \dots w_m$, whose total probability is $\text{top2}(m, n)$. The attack:

1. inverts Y under $w_1 \dots w_m$;
2. goes through the pairs (w, v) of Π in decreasing order of $\Pr[P = w] \cdot \Pr[Q = v]$, computes $\sigma = S(x(w), v)$ for each, and at the first pair with $\Omega(\sigma) = \omega^*$ outputs that pair and σ and stops;
3. outputs an arbitrary pair and string if no pair matches.

Its budget is $k_A \leq 12m$, $q_B \leq 11m + 1$, $q_H \leq 12m$, $k_S \leq n$, $q_\Omega \leq n$ and $q_T = 0$. Let $\delta_X = 2^{-128} + 12 \cdot 2^{-256}$ and write $\text{rank}(w, v)$ for the position of (w, v) in the order. Its success probabilities are at least

$$\begin{aligned}
\text{Seed game: } & \sum_{(w,v) \in \Pi} \Pr[P = w] \Pr[Q = v] \left(1 - (\text{rank}(w, v) - 1)2^{-\lambda}\right) \\
& \quad - \delta_{\text{gen}} - 2^{-256} \\
& \geq \text{top2}(m, n) \left(1 - (n - 1)2^{-\lambda}\right) - \delta_{\text{gen}} - 2^{-256}, \\
\text{Pair game: } & \text{top2}(m, n) \left(1 - (n - 1)(2^{-\lambda} + 2^{-512}) - (m - 1)\delta_X\right) \\
& \quad - \delta_{\text{gen}} - 2^{-256}.
\end{aligned}$$

Proof. By the refusal step and Lemmas W2 and W3 it suffices to work in Θ_1, at a cost of $\delta_{\text{gen}} + 2^{-256}$. In Θ_1, (P, Q) is independent of $(Y, A, B, H, S', T', \Omega', \omega^*)$, and each $x(w)$ is a function of (Y, A, B, H). Condition on $P = w_j$ and $Q = v_k$ with $(w_j, v_k) \in \Pi$. The real pair always matches: $S(x(w_j), v_k) = S(e^*, Q) = \text{seed}^*$ and $\Omega(\text{seed}^*) = \omega^*$. The attack fails only if it stops earlier, at a pair $(w, v) \neq (w_j, v_k)$ with $\Omega(\sigma) = \omega^*$ for $\sigma = S(x(w), v)$. There are two cases:

- If $(x(w), v) \neq (e^*, Q)$, then $\sigma = S'(x(w), v)$, a value at an unprogrammed point.
  - If $\sigma = \text{seed}^*$, which has probability 2^{-512}, the seed game is won and the pair game lost.
  - Otherwise $\Omega(\sigma) = \Omega'(\sigma)$ is uniform and independent of ω^*, and the pair matches with probability $2^{-\lambda}$.
- If $(x(w), v) = (e^*, Q)$, then $v = Q$ and $x(w) = x(w_j)$ with $w \neq w_j$. Then $\sigma = \text{seed}^*$: the seed game is won and the pair game lost. At most $m - 1$ pairs (w, Q) precede the real pair, and for each $\Pr[x(w) = x(w_j)] \leq \delta_X$, as in the proof of Proposition G8.

A union bound over the pairs before the real one gives both bounds.

Other tests. The same proof applies to other predicates. With the 16-bit check $\text{Trunc}_{16} T(\sigma) = 0^{16}$ in place of the test on Ω ($q_T \leq n$, $q_\Omega = 0$), the real pair passes because T is programmed at seed^*, and 2^{-16} replaces $2^{-\lambda}$. When the attack tests T first and Ω only after a pass ($q_T \leq n$, q_Ω the number of passes), a wrong pair whose candidate seed is $\sigma \neq \text{seed}^*$ passes both with probability $2^{-(\lambda+16)}$, which replaces $2^{-\lambda}$. A wrong pair with $(x(w), v) = (e^*, Q)$ yields seed^* and passes both, as in the proof.

Comparison with Theorem 5. With $k_A = 12m$ and $k_S = n$, Theorem 5 gives at most $\mathrm{top2}(m, n) + \varepsilon_W + 2^{-512}$ in the seed game. The best attack lies between the two bounds, which differ only by the false-match loss and the error terms. That loss matters when the test is narrow:

- With the check alone, the union bound of W7 becomes vacuous beyond about 2^{16} tested pairs.
- The attack's success does not fall; it grows with n towards a limit. Take uniform secrets, N equally likely pairs, the first n of them tested and $p = 2^{-16}$. Treat false passes as independent, and ignore coinciding entropies or seeds and the final guess. Then the first-match attack with the check alone succeeds with probability about $(1 - (1 - p)^n) \,/\, (N \cdot p)$. For $N_1 = N_2 = 2^{10}$ this is 0.06136, 0.06248 and 0.06250 at $n = 2^{18}$, 2^{19} and 2^{20}.
- What falls is the conditional probability of choosing the right pair when it is among those tested, $(1 - (1 - p)^n) \,/\, (n \cdot p)$: 0.245, 0.125 and 0.0625 at the same n.
- For the success probability itself, Theorem 5 allows up to $1 \,/\, 2$ or 1 at the two larger budgets. A posterior calculation suggests that no attack does much better with the check alone, since a tested pair that passes is 2^{16} times as likely as an untested pair. This is not proved here.

A.7 What the proofs do not cover

- **The wallet check profile.** A source generated under `MHFE-WALLET-CHECK-SEED-1` depends on the seed and check functions, which the adversary can evaluate: given these functions and the passphrase, it is uniform only on the values that pass the check. So Theorem 3 and Corollary G10 do not cover it. Theorem 5 does cover it, with an unknown or a known passphrase. The target wallet reference ω^* in Θ_0 is allowed even when the passphrase is empty or independently known. Theorem 5 does not cover:
  - additional phrase-identifying data outside Θ_0. When Q is unknown, examples are an address, fingerprint or history of the wallet with the empty passphrase, or a funded wallet under another passphrase that the adversary knows or guesses. Such data identifies E without Q, so a matching attack in the manner of Proposition G8 tests passwords alone. A funded wallet under another passphrase that stays unknown is also outside Θ_0, but testing with it still needs that passphrase. No upper bound like Corollary G10 is proved for this conditioned source with these additional data;
  - a funded wallet that another MHFE password opens on the same container, such as a funded decoy: it adds a second relation on Y and A, B, H that Θ_0 does not contain, so the proof does not cover it;
  - an unknown passphrase related to the password, chosen with it or stored with it;
  - a generation that selects by the values of its trials or reveals its history or duration;
  - deniability: a decoy would also have to pass the check, and the deniability theorems assume a uniform source.
- **Non-uniform or partly known sources.** Theorem 3 and its unknown-source extensions require uniform source entropy independent of the password. Those results do not cover sources that people choose or that have partly leaked. Theorem 4 treats a fully known state separately. It holds for any fixed X independent of the password and A, B, H, and does not require a uniform choice.
- **An unknown BIP39 passphrase with a shorter source.** Theorem 5 covers 24-word sources. For 12 to 21 words, wallet data depends on the passphrase, so Corollary G10 does not apply. The verifier tests the password alone, so in general no bound of the form of Theorem 5 holds: when the password space is not much larger than 2^r, the costs add rather than multiply. A password bound like Theorem 3 with an added $s_1 \cdot k_S \,/\, 2^{128}$ should follow by the argument of A.6, but it has not been written.

- **Suite 4.** Its halves have $h = \mathrm{ENT} / 2$ bits, so the terms that charge a guessed half ($q_B / 2^{128}$ in Lemma G3, $(2 \cdot q_B + 11) / 2^{128}$ in Lemma K3 and $q_\Omega / 2^{128}$ in Corollary G10) would have 2^h in place of 2^{128}. For $h = 64$, the salt function can be tabulated using only B queries, which makes Early_P certain without giving a key. Even for one round this takes 2^{64} evaluations and 2^{68} bytes (256 EiB) to store one 128-bit salt per input. The point is the kind of oracle call, not the amount of work: the tabulation needs no Argon2id call. No suite 4 proof has been written for any width. Such a proof needs an order lemma that charges a guessed half only through the A query at its salt.
- **Several containers or pairs.** Containers with equal L_{12} share the first inverse salt, so one A call per password serves both. A statement about many containers needs its own game, in the manner of multi-instance security [47]: in a composable model, per-session confidentiality of password-based encryption is unachievable when salting bounds guesses only globally rather than per session [48].
- **Preprocessing.** Precomputation within the budgets is counted. Advice retained from unbounded preprocessing [73], [74] is not covered.
- **Memory and time.** The theorems count calls to an ideal A. Turning calls into memory and time needs a resource framework [75] and a proven memory-hardness bound for Argon2id, which this document neither proves nor assumes. For data-independent functions, a separate work [76] connects memory cost and unrecoverability.
- **Quantum attackers.** The games allow classical queries only.
- **The assumptions themselves.** Assumptions 1 to 7 of A.1 are assumed, not proved. They include the salt-separation hypothesis A1, the independence of the functions, HMAC-SHA-256 as a random function, the BIP39 seed, the check digest and the wallet derivation as random functions, and the absence of side channels. S stands for PBKDF2 with 2,048 iterations. The cost of one call is a property of PBKDF2, not of the model.

A.8 Related proofs and techniques

Charging each tested password a complete chain of expensive calls is the accounting of Bellare, Ristenpart and Tessaro for iterated password hashing [47]. Query-complexity amplification [77] proves the same effect for collision-free iteration and charges premature guesses of chain values much as Early_P does. Farshim and Tessaro extend such arguments to preprocessing and warn that chains must not overlap [74]. Collision-free iteration [77] separates chains by encoding the input and the step into every query. By contrast, the c-fold iterated hash of $\mathrm{pw} \| \mathrm{sa}$ analysed in [47] enters the password and the salt in the first call only. In MHFE, the salts are derived from the Feistel state and are not stored, the adversary can also walk backward from the container, and the expensive and cheap functions are separate. Lemmas G5 and K2 and the event Cross are the steps that these differences require.

Meet-in-the-middle key recovery on Feistel schemes is known [78], and ideal-cipher lower bounds exist for cascades [67], [79]. None of these treats one low-entropy key passed through an expensive ideal function in every round. Theorem 4's contribution is the matching lower bound, not the eleven-call filter.

Zaverucha used an all-or-nothing transform to build password-based encryption in which every guess must process the whole ciphertext [80], and Biryukov and Khovratovich made such encryption memory-hard [81]. Those designs store a salt and do not preserve a format. MHFE stores nothing and keeps a valid BIP39 phrase, and the theorems above state its cost per guess in the stated model, up to the terms given. No published bound of this kind was found for SLIP-0039's encryption, which derives its round salts from the half in the same way with four PBKDF2 rounds.

Lemma W2's hybrids follow the analysis of a stateless rejection sampler by Dedić, Itkis, Reyzin and Russell [82]: outputs generated lazily, repeated samples and a limit on the number of tries. Their rejection function is a pseudorandom function under a full-entropy key. They replace this pseudorandom function by a random function that the adversary cannot access. In Theorem 5, the adversary queries $S(\cdot, q)$ at guessed low-entropy passphrases, the real one included, so only the structure of their hybrids carries over. In Section 3.3.4 of [83], the share-format attack of

Scarlata, Backendal and Haller on threshold MFKDF shows that an intermediate value with a recognizable format, or with an integrity check, lets the factors be guessed separately. Theorem 5 shows that, in its model, a 24-word source with an independent passphrase has no such route: the recovered entropy is always a valid phrase, and the wallet check reads the seed, which needs both secrets.

References

Citation numbers refer to the single reference list of the specification, `README.md`. Numbering follows first citation in the specification, then first citation of additional sources in this supplement.